

Projeto de Controle Preditivo baseado em Modelos (MPC) utilizando ferramentas de **Hardware in the loop.**

Prof. Dr. Edson Antonio Batista

Ementa:

Estudo e aplicação do controle preditivo baseado em modelo (Model Predictive Control – MPC) em sistemas dinâmicos discretos, com foco em implementação prática utilizando plataformas de Hardware-in-the-Loop (HiL). Revisão de conceitos fundamentais de álgebra matricial, análise de sistemas no espaço de estados e discretização de modelos. Formulação do problema de otimização no MPC. Estratégias de MPC com restrições e com funções de base de Laguerre. Implementação computacional, simulação e testes de controladores preditivos em ambiente HiL utilizando ferramentas como MATLAB/Simulink e FPGAs. Análise de desempenho, robustez e tempo real em aplicações industriais e de energia.

Objetivo:

1. Compreender os fundamentos teóricos do controle preditivo baseado em modelo discreto;
2. Aplicar técnicas de modelagem e controle preditivo em sistemas lineares e não lineares;
3. Desenvolver e implementar controladores MPC em plataformas digitais e embarcadas.;
4. Integrar metodologias de simulação e validação via Hardware-in-the-Loop;
5. Explorar aplicações do MPC em controle de processos, sistemas elétricos, telecomunicações, eletrônicos e energéticos.

Conteúdo Programático

1. Revisão Matemática e Modelagem

- Operações matriciais e propriedades fundamentais - **VHDL**.
- Modelagem de sistemas dinâmicos lineares e não lineares no espaço de estados - Matlab.
- Discretização de sistemas contínuos - Matlab.
- Representações equivalentes e análise de controlabilidade e observabilidade - Matlab.

1. Fundamentos de Controle Preditivo (MPC)

- Estrutura geral do MPC.
- Formulação do problema de otimização quadrática (QP).
- Função custo, horizonte de predição e controle.
- Restrições em variáveis de estado e de controle
- MPC com Restrições e Robustez
- Controle preditivo sob restrições de amplitude e taxa.
- Técnicas de suavização e tuning de parâmetros.
- Estratégias para tratamento de incertezas e ruídos.

Conteúdo Programático

3. MPC com Funções de Laguerre

- Representação compacta da dinâmica por funções de Laguerre.
- Redução de complexidade computacional.
- Implementação numérica e análise de desempenho.

4. Implementação Computacional e Hardware-in-the-Loop (HiL)

- Introdução ao conceito e arquitetura de sistemas HiL.
- Ferramentas para desenvolvimento e teste (MATLAB/Simulink, FPGA).
- Integração entre modelos simulados e hardware real.
- Testes em malha fechada e validação experimental.

5. Estudo de Casos e Aplicações

- Aplicações em processos tecnológicos.
- Análise comparativa de desempenho entre abordagens clássicas e MPC.
- Projeto final: implementação de um controlador MPC em ambiente HiL.

Critério de Avaliação:

- Exercícios e relatórios de laboratório (30%)
- Seminários e apresentações técnicas (20%)
- Projeto final de implementação em HiL (50%)

Diagrama de bloco do Controlador Preditivo

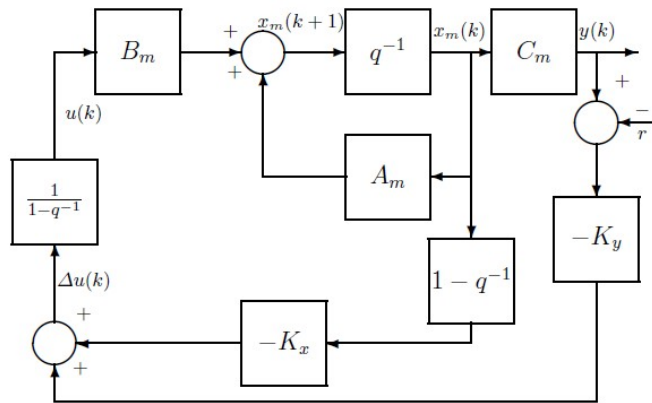
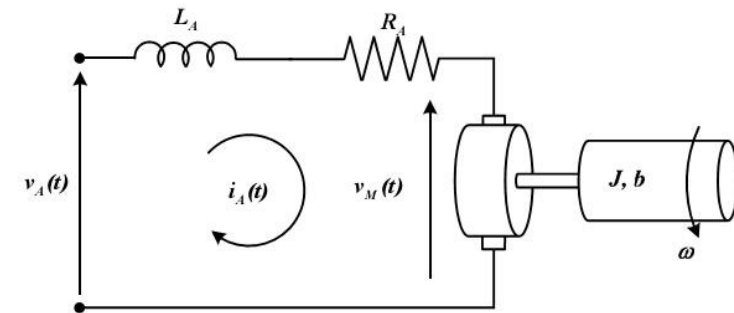
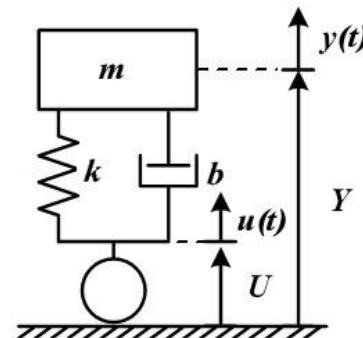
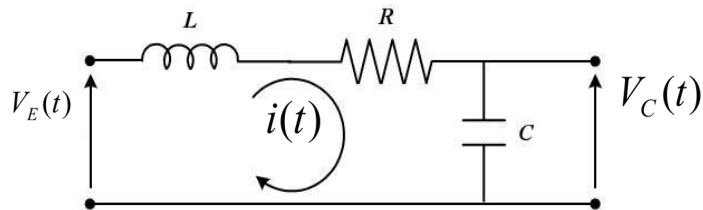
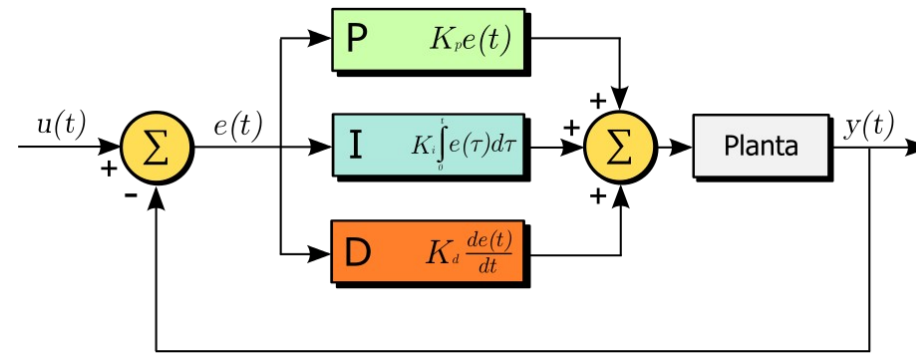


Diagrama de bloco do Controlador PID



Model Predictive Control - MPC

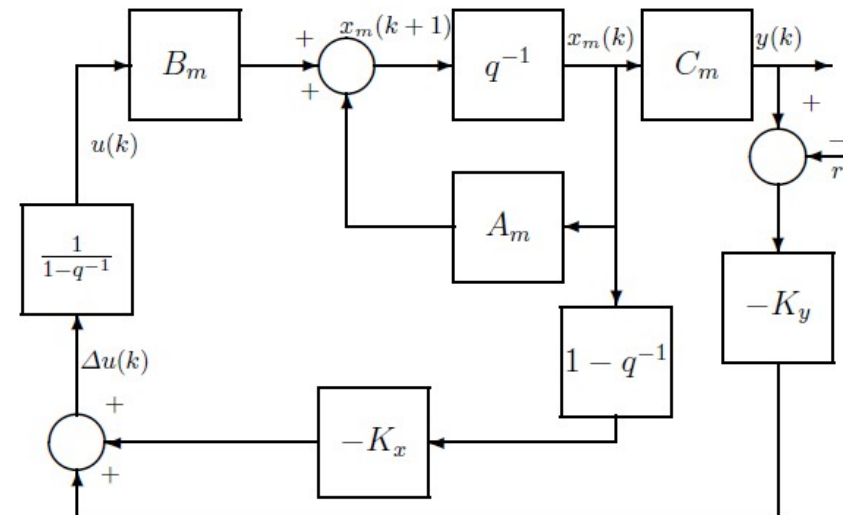
- Em formato matricial:

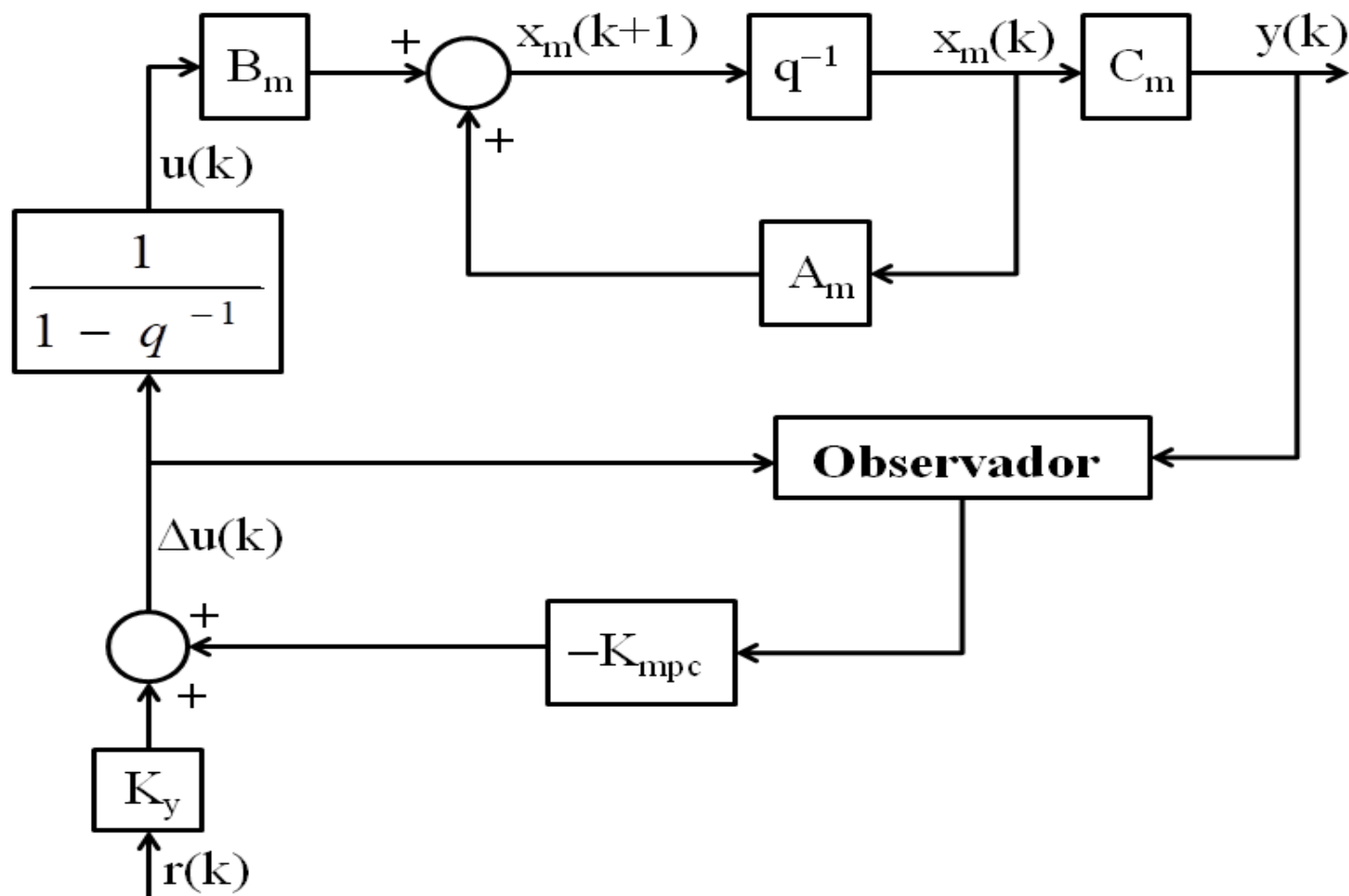
$$Y = Fx(k_i) + \varphi\Delta U$$

$$\begin{bmatrix} y(k+1) \\ y(k+2) \\ \vdots \\ y(k+n) \end{bmatrix} = \begin{bmatrix} CA \\ CA^2 \\ CA^3 \\ \vdots \\ CA^{N_p} \end{bmatrix} x(k_i) + \begin{bmatrix} CB & 0 & 0 & \dots & 0 \\ CAB & CB & 0 & \dots & 0 \\ CA^2B & CAB & CB & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ CA^{N_p-1}B & CA^{N_p-2}B & CA^{N_p-3}B & \dots & CA^{N_p-N_c}B \end{bmatrix} \Delta U$$

$$K_{mpc} = [1 \quad 0 \quad \dots \quad 0](\varphi^T \varphi + \bar{R})^{-1}(\varphi^T F)$$

$$K_y = [1 \quad 0 \quad \dots \quad 0](\varphi^T \varphi + \bar{R})^{-1}(\varphi^T \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix})$$





Exemplo

Considere o circuito elétrico RLC mostrado na Figura 1, composto por um indutor L , um resistor R e um capacitor C , com entrada $V_E(t)$, corrente de malha $i(t)$ e saída $V_C(t)$, correspondente à tensão no capacitor.

O objetivo desta modelagem é obter:

- a equação diferencial no domínio do tempo;
- a representação no domínio de Laplace;
- a função de transferência;
- o modelo em espaço de estados;

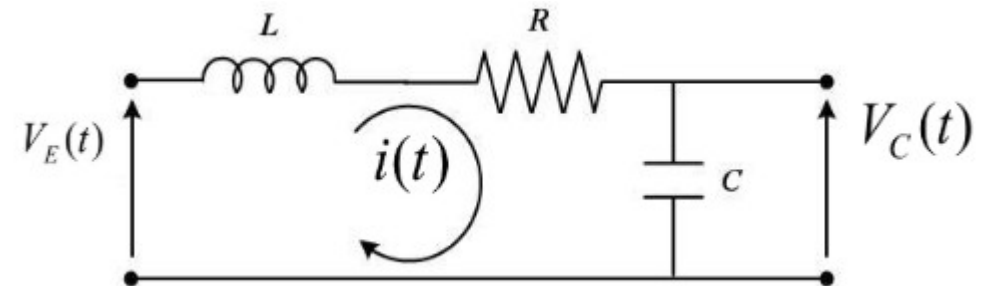


Figura 1: Circuito RLC série com saída no capacitor.

Aplicando a Lei das Tensões de Kirchhoff (LKT) ao circuito, tem-se:

$$V_E(t) = V_L(t) + V_R(t) + V_C(t) \quad (1)$$

Sabendo que:

$$V_L(t) = L \frac{di(t)}{dt} \quad (2)$$

$$V_R(t) = Ri(t) \quad (3)$$

e que, para o capacitor,

$$i(t) = C \frac{dV_C(t)}{dt} \quad (4)$$

substitui-se na equação da malha:

$$V_E(t) = L \frac{di(t)}{dt} + Ri(t) + V_C(t) \quad (5)$$

Como $i(t) = C \frac{dV_C(t)}{dt}$ (6), então $\frac{di(t)}{dt} = C \frac{d^2V_C(t)}{dt^2}$ (7).

Substituindo essas relações na equação da malha, obtém-se:

$$V_E(t) = LC \frac{d^2V_C(t)}{dt^2} + RC \frac{dV_C(t)}{dt} + V_C(t) \quad (8)$$

Portanto, a equação diferencial do sistema é dada por:



$$LC \frac{d^2V_C(t)}{dt^2} + RC \frac{dV_C(t)}{dt} + V_C(t) = V_E(t) \quad (9)$$

Aplicando a Transformada de Laplace à Equação (9), assumindo condições iniciais nulas, resulta em:

$$LCs^2V_C(s) + RCsV_C(s) + V_C(s) = V_E(s) \quad (10)$$

Colocando $V_C(s)$ em evidência:

$$V_C(s)(LCs^2 + RCs + 1) = V_E(s) \quad (11)$$

Logo, a função de transferência entre a entrada $V_E(s)$ e a saída $V_C(s)$ é:



$$\frac{V_C(s)}{V_E(s)} = \frac{1}{LCs^2 + RCs + 1} \quad (12)$$

Para obter a representação em espaço de estados, define-se o vetor de estados como:

$$x_1(t) = V_C(t) \quad (13)$$

$$x_2(t) = i(t) \quad (14)$$

A entrada do sistema é definida como:

$$u(t) = V_E(t) \quad (15)$$

e a saída como:

$$y(t) = V_C(t) \quad (16)$$

A partir da relação do capacitor,

$$i(t) = C \frac{dV_C(t)}{dt} \quad (17)$$

segue que:

$$\dot{x}_1(t) = \frac{1}{C} x_2(t) \quad (18)$$

Da equação da malha,

$$V_E(t) = L \frac{di(t)}{dt} + Ri(t) + V_C(t) \quad (19)$$

isolando $\frac{di(t)}{dt}$, obtém-se:

$$L \frac{di(t)}{dt} = V_E(t) - Ri(t) - V_C(t) \quad (20)$$

|

$$\frac{di(t)}{dt} = -\frac{1}{L} V_C(t) - \frac{R}{L} i(t) + \frac{1}{L} V_E(t) \quad (21)$$

ou seja,



$$\dot{x}_2(t) = -\frac{1}{L} x_1(t) - \frac{R}{L} x_2(t) + \frac{1}{L} u(t) \quad (22)$$

Assim, o modelo em espaço de estados contínuo é dado por:

$$\dot{x}(t) = Ax(t) + Bu(t) \quad (23)$$

$$y(t) = Cx(t) + Du(t) \quad (24)$$

com:



$$A = \begin{bmatrix} 0 & \frac{1}{C} \\ -\frac{1}{L} & -\frac{R}{L} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{1}{L} \end{bmatrix} \quad (25)$$

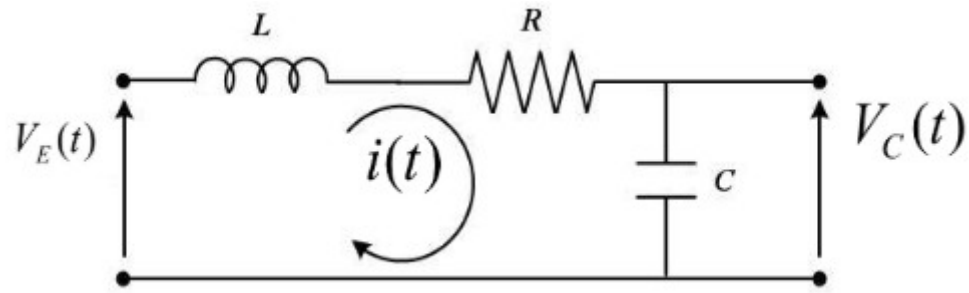
$$C = [1 \quad 0], \quad D = [0] \quad (26)$$

Portanto,



$$\dot{x}(t) = \begin{bmatrix} 0 & \frac{1}{C} \\ -\frac{1}{L} & -\frac{R}{L} \end{bmatrix} x(t) + \begin{bmatrix} 0 \\ \frac{1}{L} \end{bmatrix} u(t) \quad (27)$$

$$y(t) = [1 \quad 0] x(t) \quad (28)$$



$$A = \begin{bmatrix} 0 & \frac{1}{C} \\ -\frac{1}{L} & -\frac{R}{L} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{1}{L} \end{bmatrix}$$

$$C = [1 \quad 0]$$

$$K_{mpc} = [1 \quad 0 \quad \dots \quad 0](\varphi^T \varphi + \bar{R})^{-1}(\varphi^T F)$$

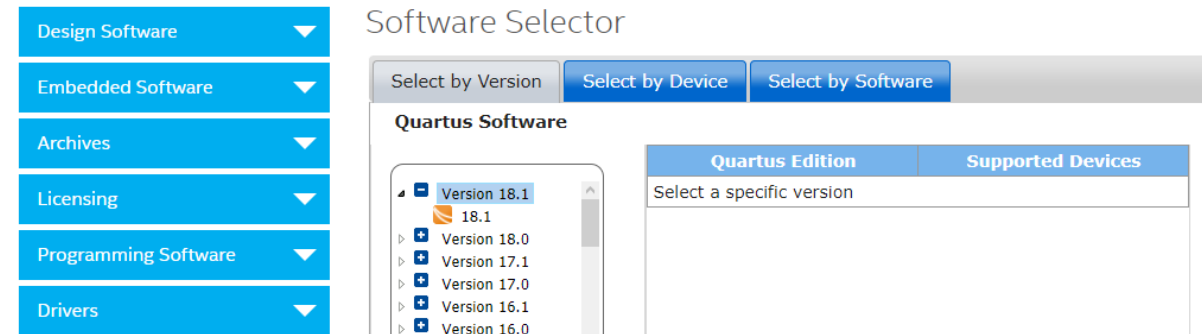
$$K_y = [1 \quad 0 \quad \dots \quad 0](\varphi^T \varphi + \bar{R})^{-1}(\varphi^T \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix})$$

Parte Prática:

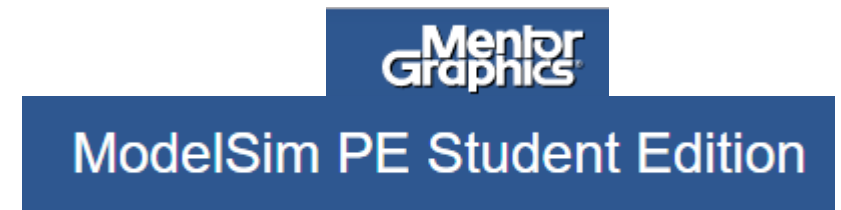
- Quartus II – Intel Corporation
- ModelSim
- VHDL
- **FPGA**



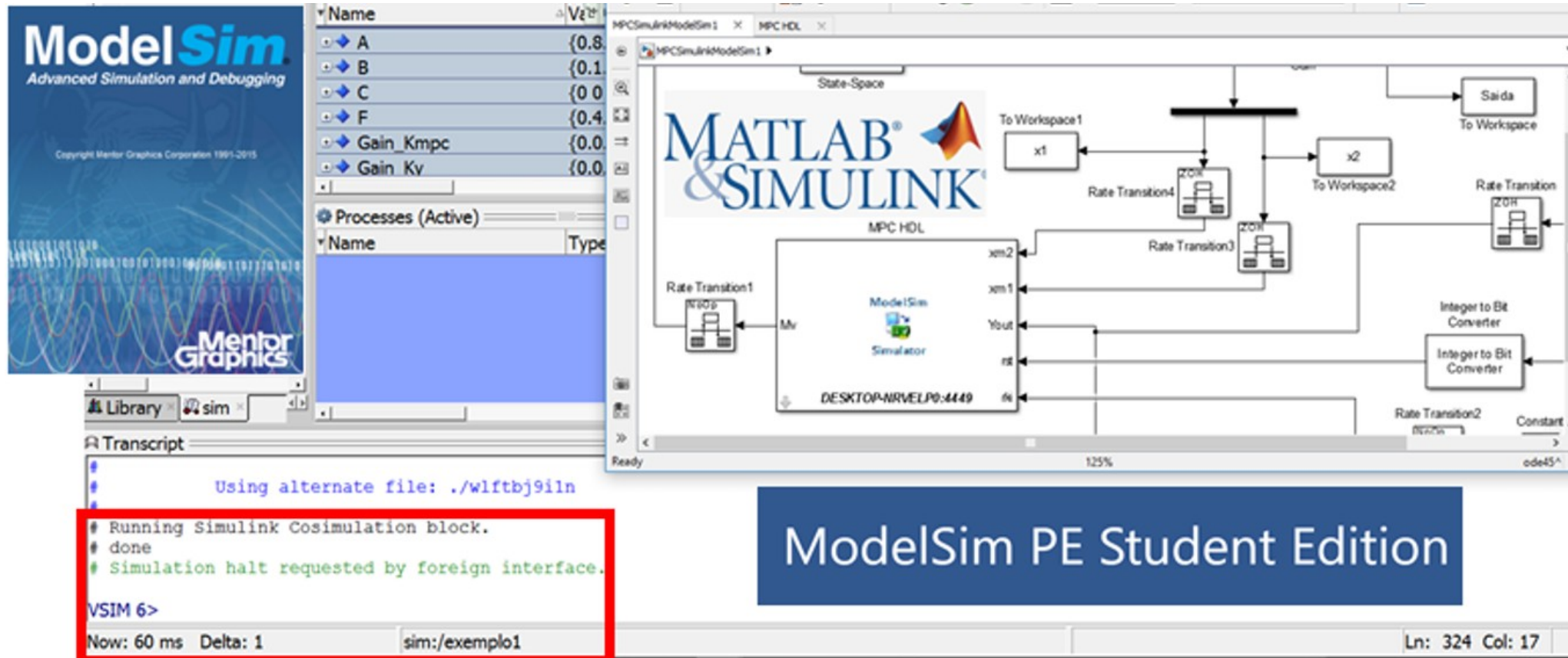
Download Center for FPGAs



ModelSim
Altera Edition



Tecnologia utilizada para desenvolver e testar MPC



ModelSim PE Student Edition

Transcript:

```

Using alternate file: ./wlftbj9i1n
# Running Simulink Cosimulation block.
# done
# Simulation halt requested by foreign interface.
VSIM 6>
Now: 60 ms Delta: 1 sim:/exemplo1

```

Block Diagram Details:

- State-Space** block connected to **MATLAB & SIMULINK** block.
- MATLAB & SIMULINK** block contains **MPC HDL** and **ModelSim Simulator**.
- Rate Transition1** block connected to **Mv** input.
- Rate Transition2** block connected to **Constant** block.
- Rate Transition3** and **Rate Transition4** blocks connected to **Rate Transition1** and **Rate Transition2** respectively.
- Integer to Bit Converter** blocks connected to **Rate Transition3** and **Rate Transition4**.
- To Workspace1** and **To Workspace2** blocks connected to **Rate Transition3** and **Rate Transition4** respectively.
- Saida** block connected to **Rate Transition4**.

Arquitetura básica de hardware desenvolvido em VHDL

ENTITY *entity_name* **IS**

 generics

 port declarations

END *entity_name*;

ARCHITECTURE *arch_name* **OF** *entity_name* **IS**

 enumerated data types

 internal signal declarations

 component declarations

BEGIN

 signal assignment statements

 process statements

 component instantiations

END *arch_name*;

- **LIBRARY IEEE;**
 - Contem o seguintes pacotes:
 - `std_logic_1164`
 - `std_logic_arith`
 - `std_logic_signed`
 - `std_logic_unsigned`


```
LIBRARY ieee; -- declaração de biblioteca (fundamental)
USE ieee.std_logic_1164.all; -- Definição do pacote a se utilizado
```

```
ENTITY Reg1 IS -- nome do componente
  PORT -- definição das entradas e saídas
  ( -- informa o início da definição das portas de entrada e saída
    Dado : IN STD_LOGIC; -- o comando "IN" define como entrada e o comando "STD_LOGIC" ==> 0 e 1
    Clock : IN STD_LOGIC; -- A cada linha deve-se colocar ";"
    Clear : IN STD_LOGIC;
    Preset : IN STD_LOGIC;
    Saida : OUT STD_LOGIC -- o comando "OUT" define como saída e o comando "STD_LOGIC" ==> 0 e 1
  ); -- informa o fim das portas de entrada e saída
END Reg1; -- fecha a definição das portas
```

```
ARCHITECTURE bdf_type OF Reg1 IS -- definição do comportamento do componente
```

```
SIGNAL DFF_inst : STD_LOGIC; -- sinal, pode ser utilizado para manipular as informações entre entrada e saída
```

```
ENTITY entity_name IS
  generics
  port declarations
END entity_name;
```

```
ARCHITECTURE arch_name OF entity_name IS
  enumerated data types
  internal signal declarations
  component declarations
```

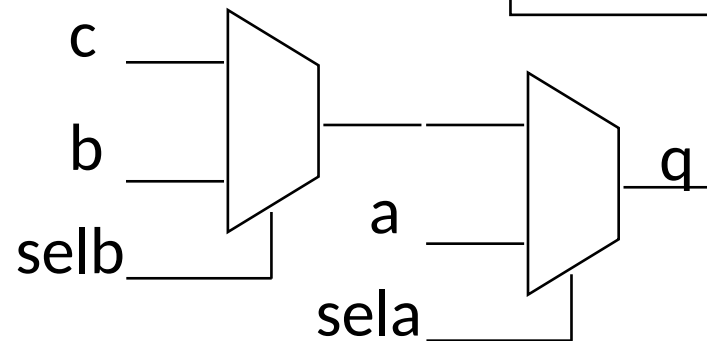
```
BEGIN
  signal assignment statements
  process statements
  component instantiations
END arch_name;
```

Sintaxe

```
IF condition1 THEN
    {sequence of statement(s)}
ELSIF condition2 THEN
    {sequence of statement(s)}
    .
    .
ELSE
    {sequence of statement(s)}
END IF;
```

Exemplo

```
PROCESS(sela, selb, a, b, c)
BEGIN
    IF sela='1' THEN
        q <= a;
    ELSIF selb='1' THEN
        q <= b;
    ELSE
        q <= c;
    END IF;
END PROCESS;
```



Sintaxe

```

signal_name <= signal/value when condition1 else
    signal/value when condition2 else
        .
        .
    signal/value when condition3 else
    signal/value;

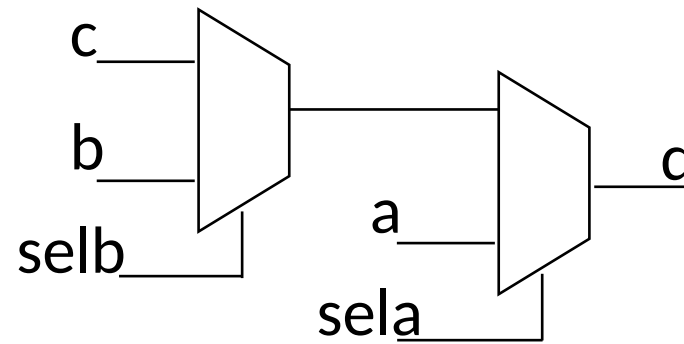
```

Exemplo

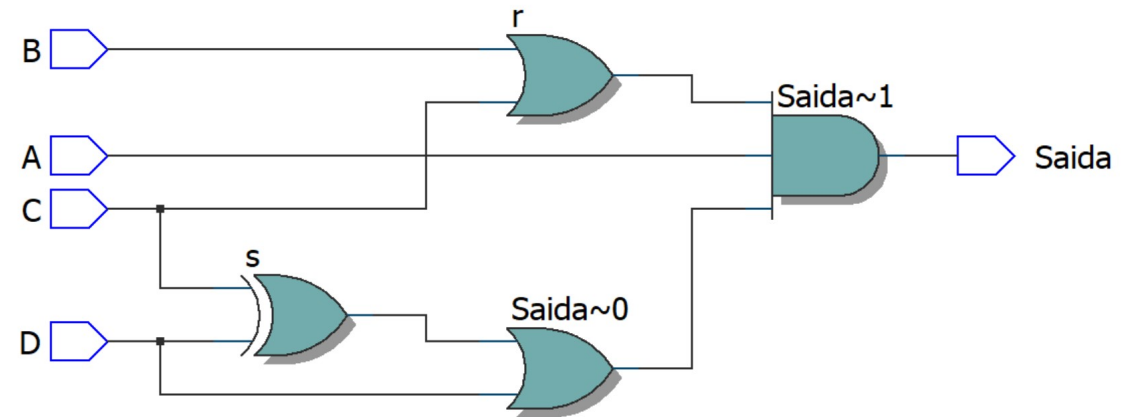
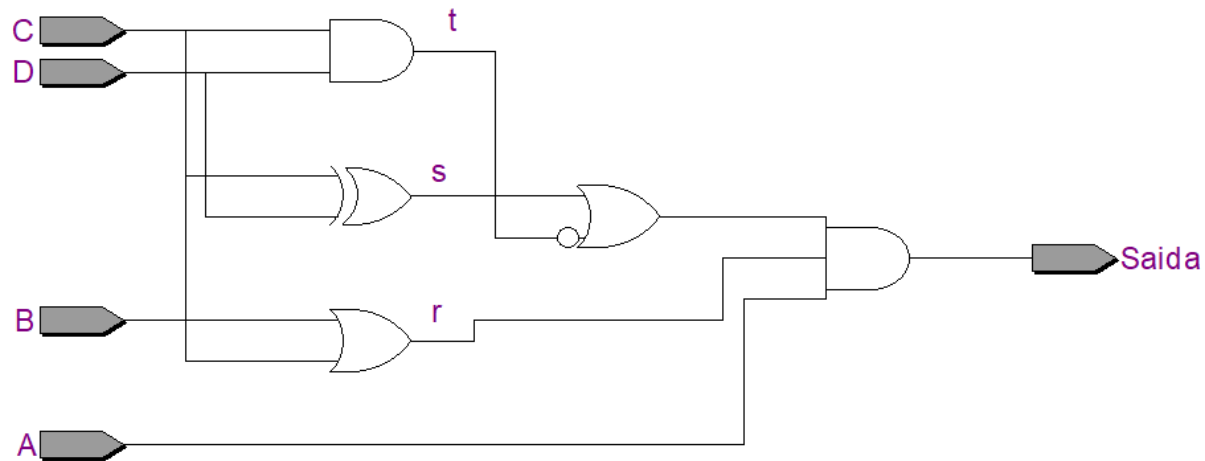
```

q <=  a WHEN sela = '1' ELSE
      b WHEN selb = '1' ELSE
      c;

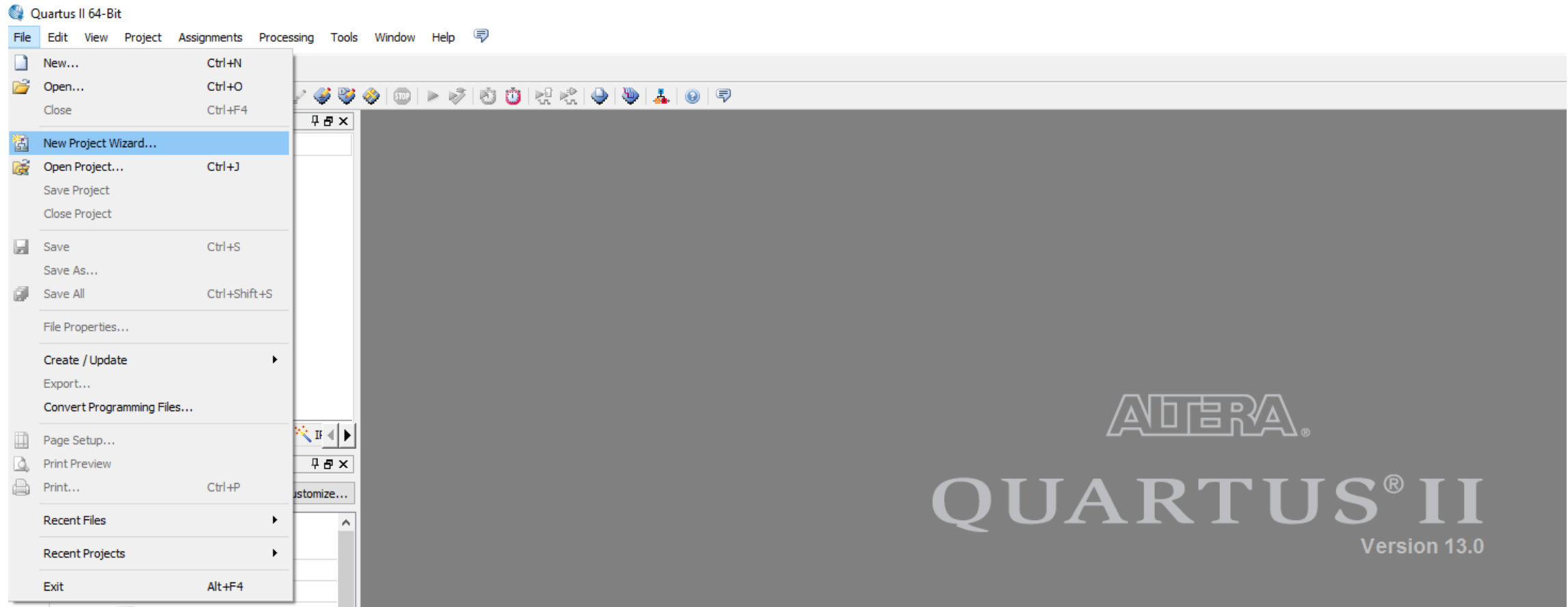
```

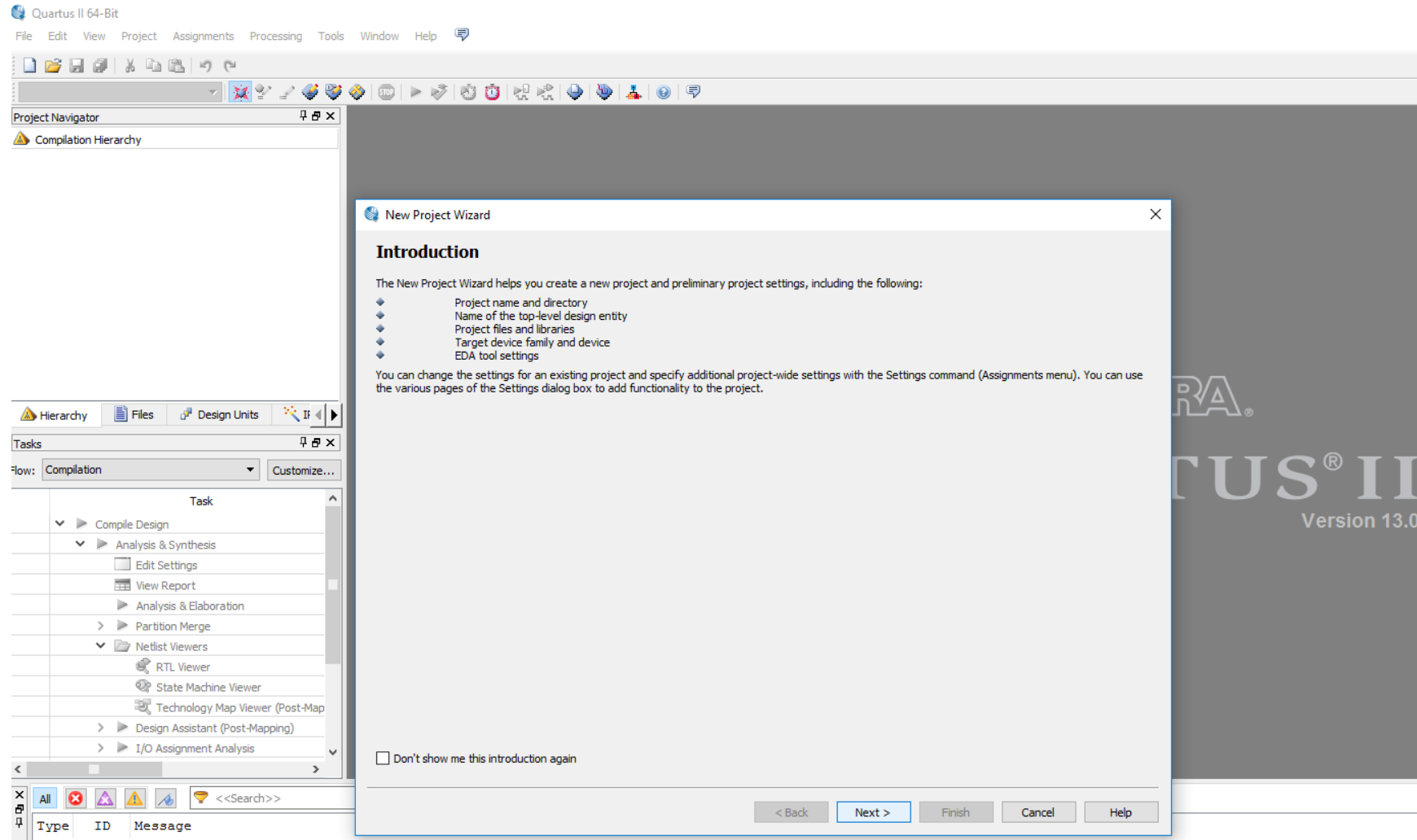


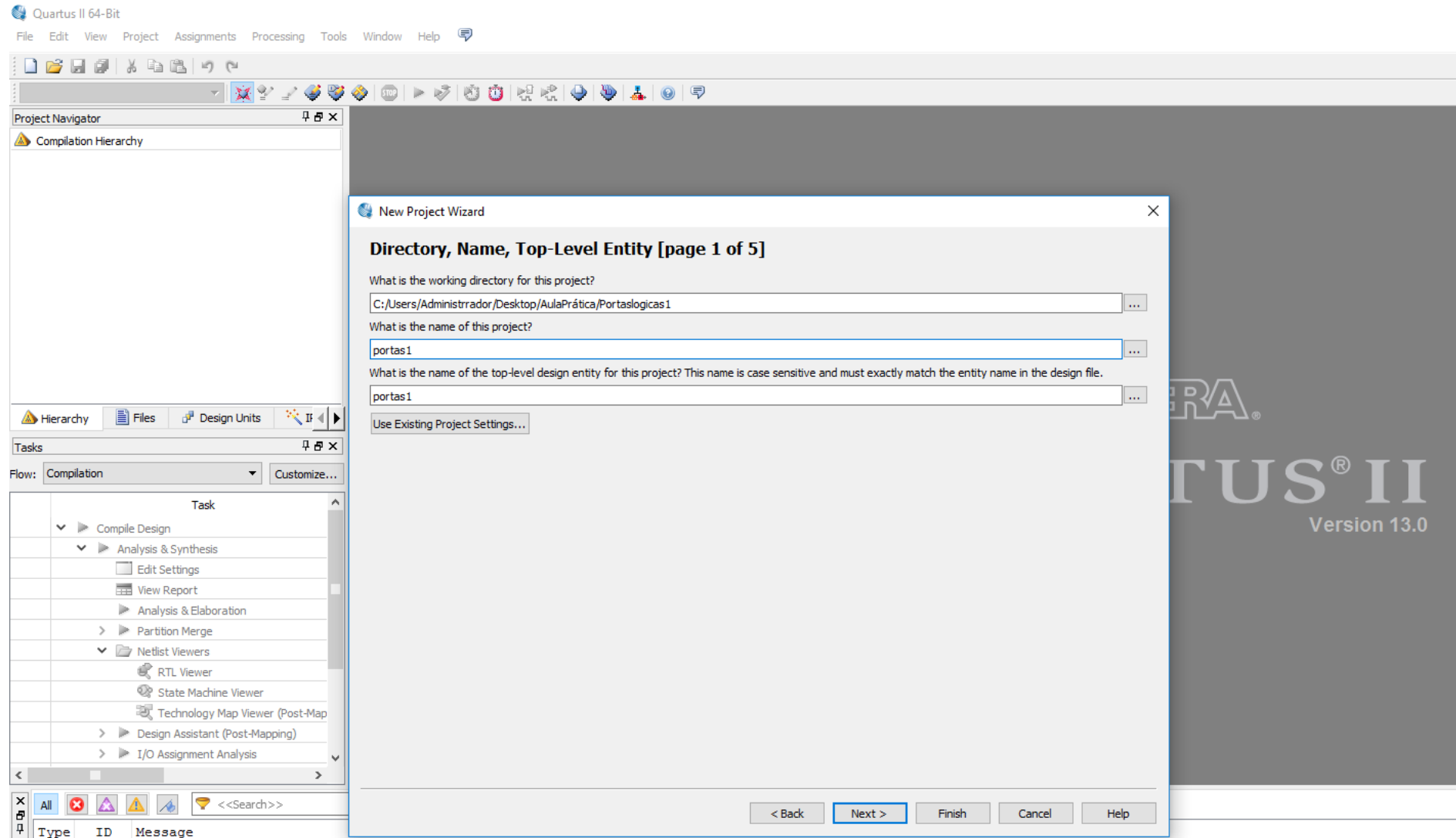
Exemplo: Descrever em VHDL o comportamento do circuito abaixo. Simule o funcionamento do hardware no ModelSim.



1. Abra o Quartus II;
2. Selecione o New Project;
3. Selecione uma pasta para armazenar o projeto;
4. Atribua um nome ao projeto (evite caracteres especiais);
5. Next;
6. Next;
7. Selecione o dispositivo;
8. Conclua;
9. Selecione New;
10. Selecione Block Diagram;
11. Monte o circuito lógico utilizando a biblioteca de dispositivos;
12. Após montar o circuito salve o arquivo com o mesmo nome atribuído no item 4;
13. Start compilation o projeto;
14. Caso 100%;
15. Gere o respectivo projeto em VHDL – CreateHDL.







Quartus II 64-Bit

File Edit View Project Assignments Processing Tools Window Help

Project Navigator

Compilation Hierarchy

Tasks

Flow: Compilation Customize...

Task

- Compile Design
 - Analysis & Synthesis
 - Edit Settings
 - View Report
 - Analysis & Elaboration
 - Partition Merge
 - Netlist Viewers
 - RTL Viewer
 - State Machine Viewer
 - Technology Map Viewer (Post-Map)
 - Design Assistant (Post-Mapping)
 - I/O Assignment Analysis

New Project Wizard

Family & Device Settings [page 3 of 5]

Select the family and device you want to target for compilation.
You can install additional device support with the Install Devices command on the Tools menu.

Device family

Family: Cyclone II

Devices: All

Show in 'Available devices' list

Package: Any

Pin count: 672

Speed grade: 6

Name filter:

Target device

☐ Auto device selected by the Fitter

☒ Specific device selected in 'Available devices' list

☐ Other: n/a

☒ Show advanced devices ☐ HardCopy compatible only

Available devices:

Name	Core Voltage	LEs	User I/Os	Memory Bits	Embedded multiplier 9-bit elements	PLL	bal Clo
EP2C35F672C6	1.2V	33216	475	483840	70	4	16
EP2C50F672C6	1.2V	50528	450	594432	172	4	16
EP2C70F672C6	1.2V	68416	422	1152000	300	4	16

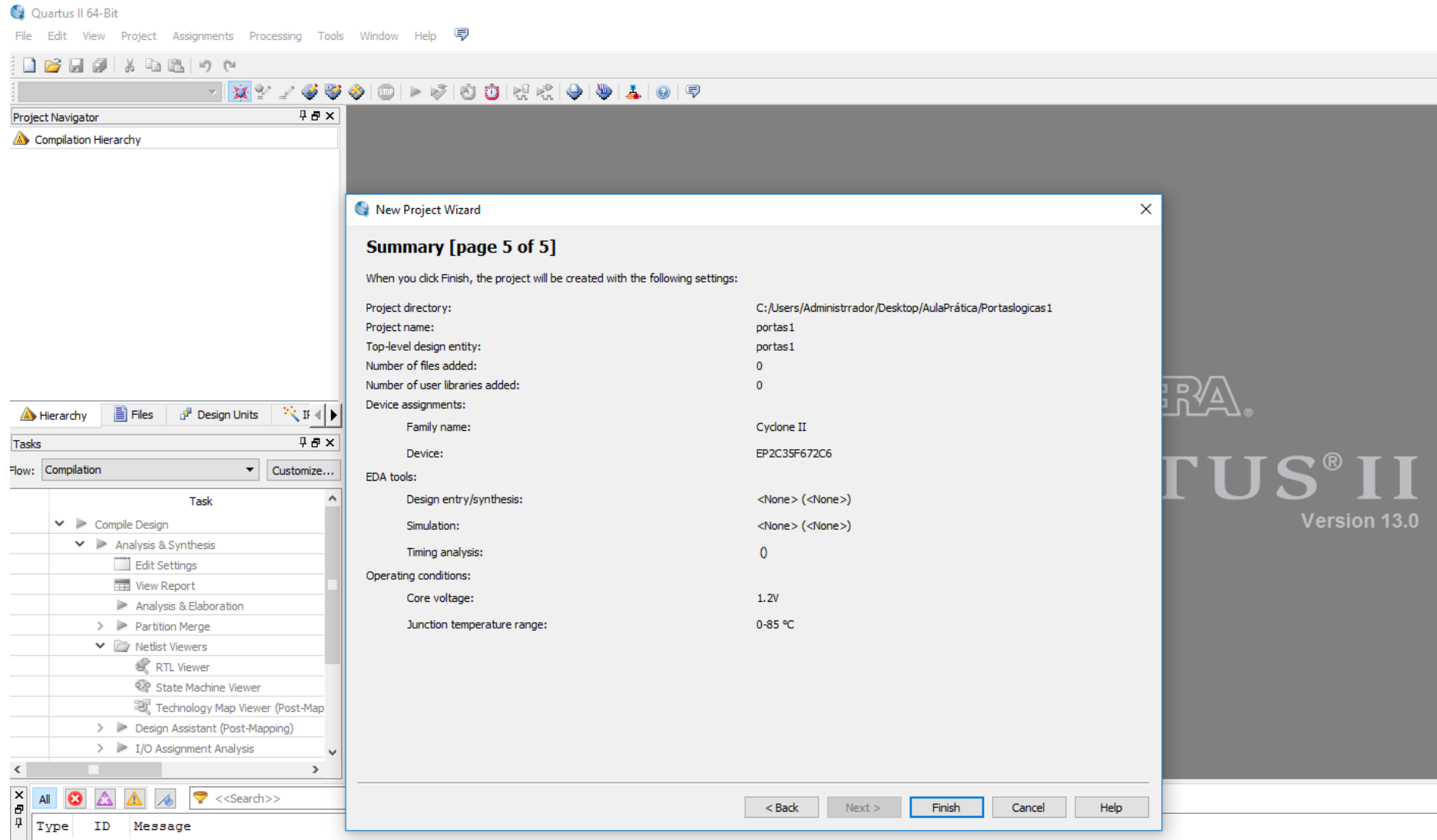
Companion device

HardCopy:

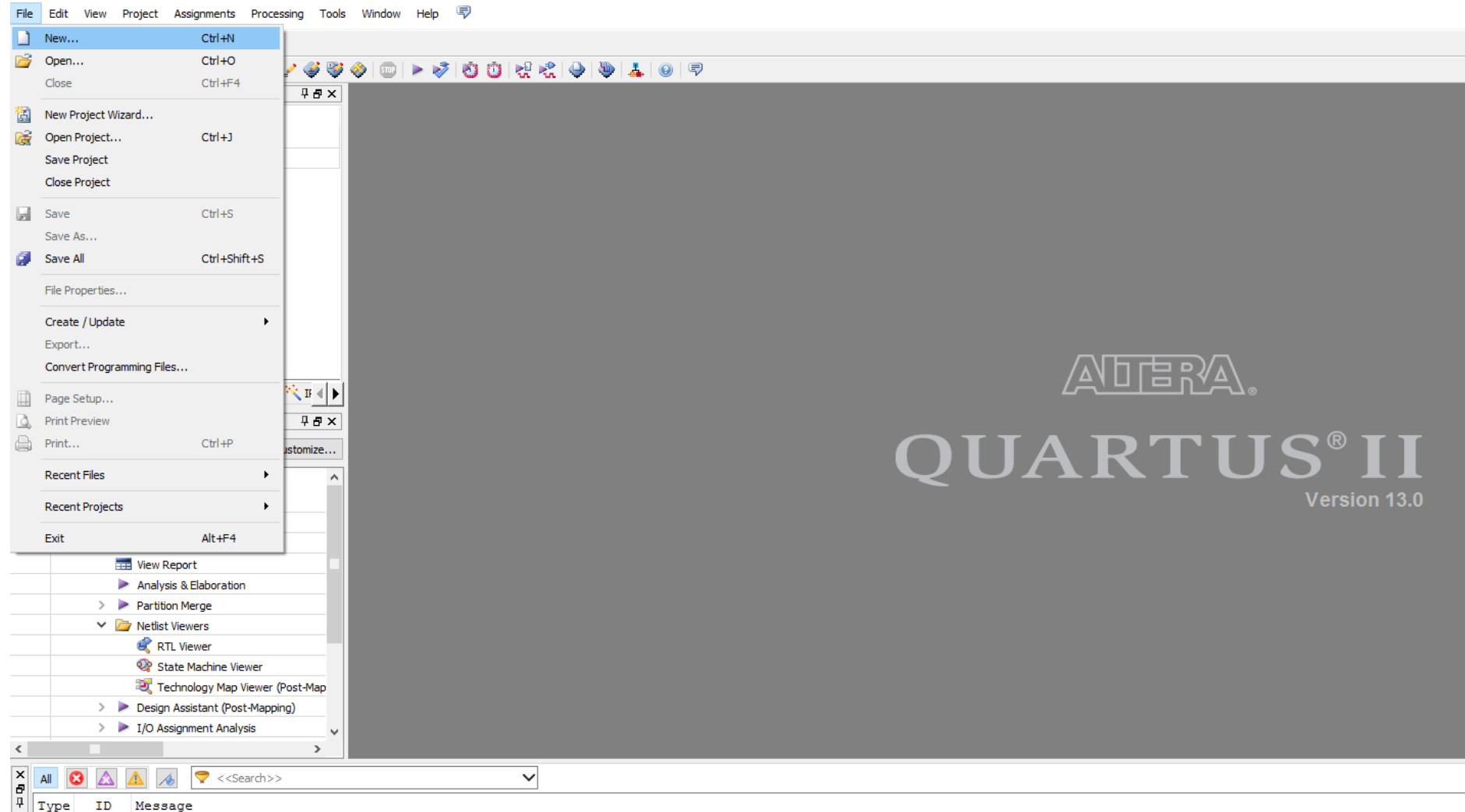
☐ Limit DSP & RAM to HardCopy device resources

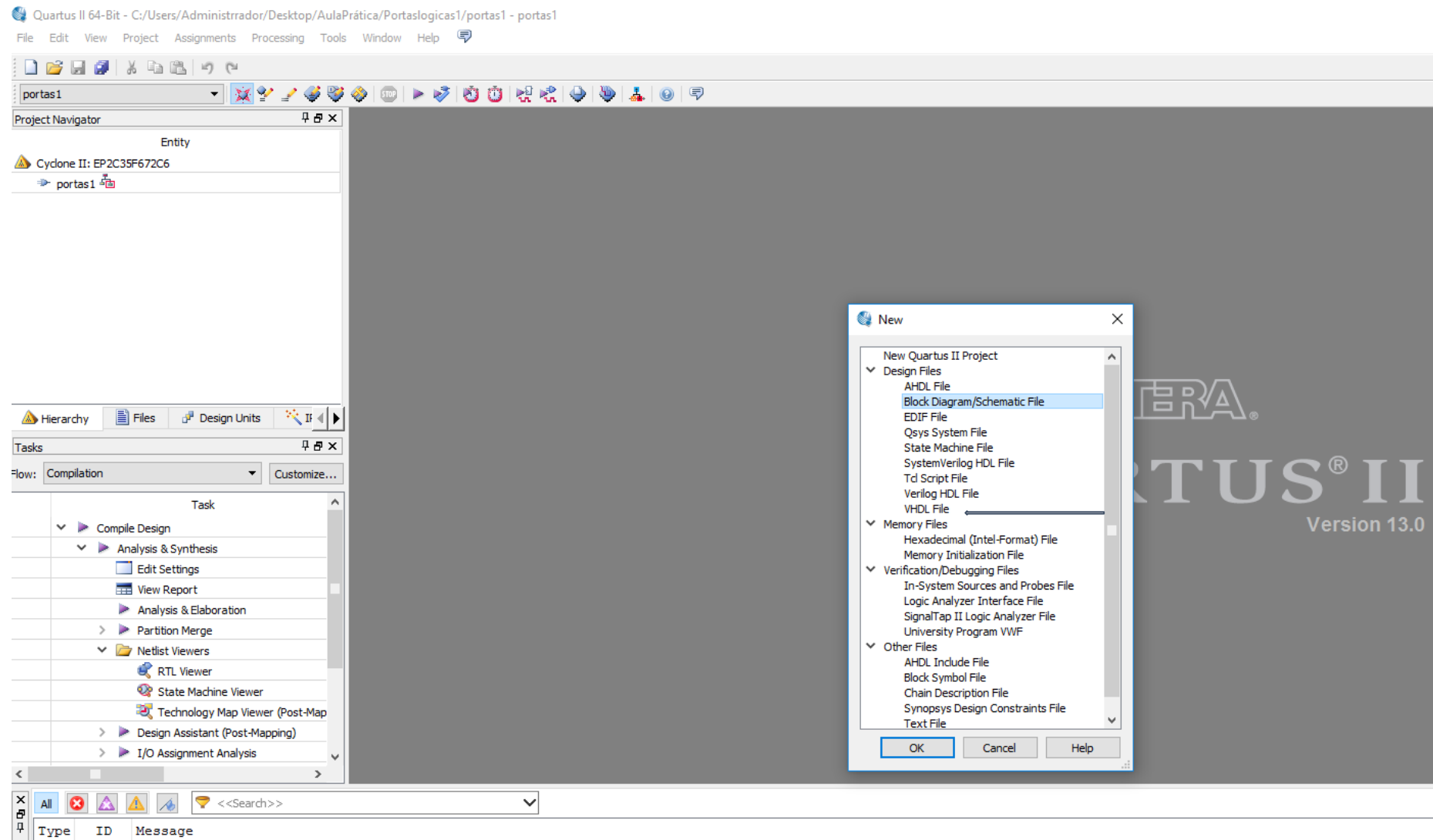
To use this feature you must purchase the Quartus II Subscription Edition software

< Back Next > Finish Cancel Help



Quartus II 64-Bit - C:/Users/Administrador/Desktop/AulaPrática/Portaslogicas1/portas1 - portas1



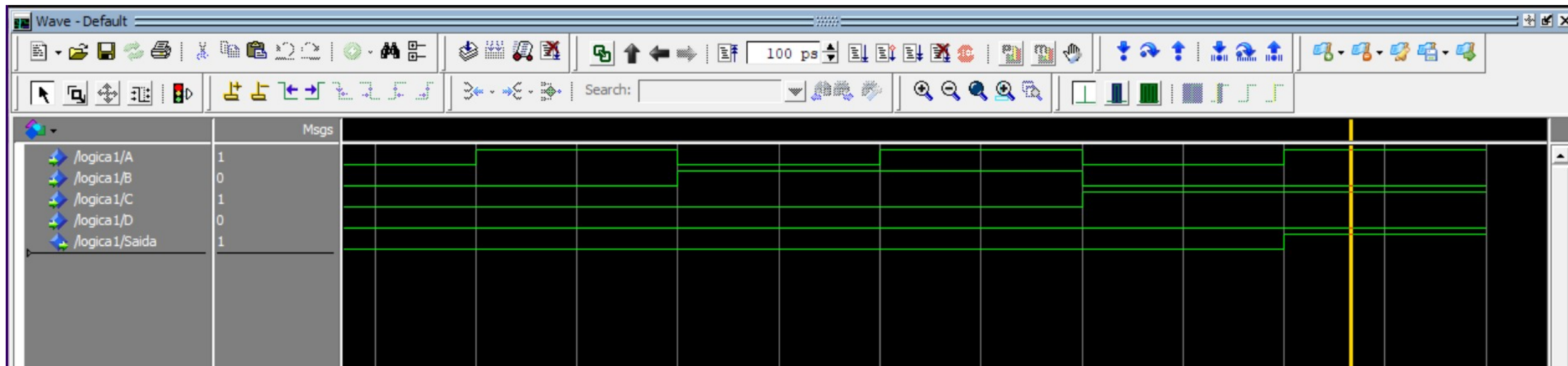


```

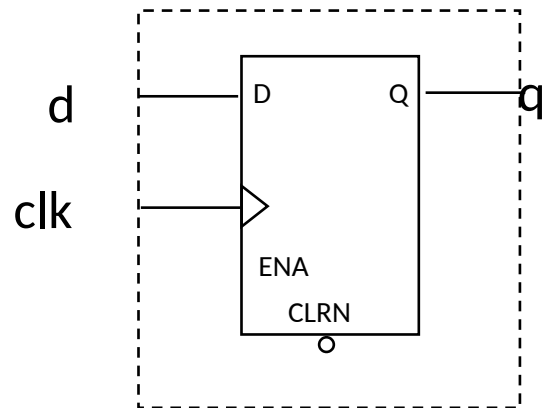
1  library ieee; -- biblioteca geral all the time
2  use ieee.std_logic_1164.all; -- padronizada para lógica
3
4  entity exemplo1 is
5  |  -- Definir as entradas e saidas e os respectivos tipos
6  |  Port (
7  |      A: in bit;
8  |      B : in bit;
9  |      C : in bit;
10 |      D : in bit;
11 |      Saida : out bit);
12 |
13 |  end exemplo1;
14
15  -- Descreve o funcionamento do circuito
16  architecture mfe of exemplo1 is
17  |
18  |  --caso queira utilizar, os sinais servem como cabo de conexão,
19  |  --portanto devem ser declarados antes de iniciar descrever o hardware,
20  |  signal r, s, t :bit;
21  |
22  |  begin
23  |
24  |      r <= A and (B or C); -- and e or são palavras reservadas
25  |
26  |      s <= D xor C; -- <= é atribuição de sinais := é para variável;
27  |
28  |      t <= C nand D;
29  |      Saida <= r and (s or t);
30  |
31  |  end mfe;

```

Simular no ModelSim



Exemplo: Descrever em VHDL o comportamento do Flip Flop D. Simule o funcionamento do hardware no ModelSim.



force CLK 0 0, 1 50 -r 100

```

1  -- Edson Antonio Batista
2
3  -----
4  LIBRARY ieee; -- declaração da biblioteca
5  USE ieee.std_logic_1164.all; -- declaração do pacote
6
7  ENTITY FlipFlopD2 IS
8
9  -- Definição das Entradas e Saídas do Componente
10  PORT ( d : in std_logic;
11         clr: in std_logic;
12         clk : in std_logic;
13         q : out std_logic
14       );
15  END FlipFlopD2;
16
17  ARCHITECTURE funcionamento OF FlipFlopD2 IS
18  BEGIN
19
20  PROCESS(clk, clr) -- Este comando define a lista de sensibilidade,
21                    --ou seja, quais sinais são sensíveis neste processo
22  BEGIN
23
24
25  IF (clr = '0') then --Se clear (clr) = 0 então q receberá zero
26
27  q <= '0'; -- <= atribuição de valores para entradas e sinais
28
29  elsif rising_edge(clk) THEN --rising_edge se o clock estiver na borda de subida
30    q <= d;
31  END IF;
32
33  END PROCESS;
34
35  END funcionamento;

```

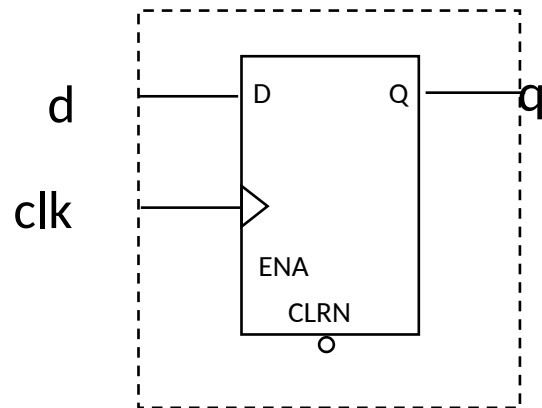
ENTITY *entity_name* IS
 generics
 port declarations
END *entity_name*;

ARCHITECTURE *arch_name* OF *entity_name* IS
 enumerated data types
 internal signal declarations
 component declarations

BEGIN
 signal assignment statements
 process statements
 component instantiations

END *arch_name*;

Exemplo: Descrever em VHDL o comportamento do Flip Flop D. Simule o funcionamento do hardware no ModelSim.



```

17  ARCHITECTURE funcionamento OF FlipFlopD2 IS
18  |
19  BEGIN
20  |
21  PROCESS(clk, clr) -- Este comando define a lista de sensibilidade,
22  |                  --ou seja, quais sinais são sensíveis neste processo
23  |
24  |   BEGIN
25  |
26  |   IF (clr = '0') then --Se clear (clr) = 0 então q receberá zero
27  |
28  |       q <= '0'; -- <= atribuição de valores para entradas e sinais
29  |
30  |   elsif rising_edge(clk) THEN --rising_edge se o clock estiver na borda de subida
31  |       q <= d;
32  |   END IF;
33  |
34  |   END PROCESS;
35  |
36  END funcionamento;

```

Introdução Matrix Math Package

The VHDL matrix math packages can be downloaded at:

http://www.vhdl.org/fphdl/real_matrix_pkg.zip

By David Bishop

- A biblioteca VHDL matrix é baseada na funcionalidade das operações realizadas no Matlab.
- Para habilitar a biblioteca deve-se executar os seguintes procedimentos.

```
vlib ieee_proposed.lib
```

```
vmap ieee_proposed ./ieee_proposed.lib
```

```
vcom -2008 -work ieee_proposed real_matrix_pkg.vhdl
```

```
vcom -2008 -work ieee_proposed real_matrix_pkg_body.vhdl
```

```
vcom -2008 -work ieee_proposed complex_matrix_pkg.vhdl
```

```
vcom -2008 -work ieee_proposed complex_matrix_pkg_body.vhdl
```

```
vcom -2008 -work ieee_proposed fixed_matrix_pkg.vhdl
```

```
vcom -2008 -work ieee_proposed fixed_matrix_pkg_body.vhdl
```

```
vlib work
```

```
vmap work work
```

```
vcom -2008 test_real_matrix.vhdl
```

```
vcom -2008 test_complex_matrix.vhdl
```

```
vcom -2008 test_fixed_matrix.vhdl
```

Descompactar a biblioteca da Altera



real_matrix_pkg (1).zip

Operadores	Função	exemplo
*	Multiplicação entre matrizes	<code>real_matrix * real_matrix</code> retorna <code>real_matrix</code>
+	Soma de matrizes	<code>real_matrix + real_matrix</code> retorna <code>real_matrix</code>
-	Subtração entre matrizes	<code>real_matrix - real_matrix</code> retorna <code>real_matrix</code>
/	Divisão entre matrizes	<code>real_matrix / real_matrix</code> retorna <code>real_matrix</code>
**	Matriz elevada a um inteiro	<code>real_matrix ** integer</code> retorna <code>real_matrix</code>
=	Gera igualdade caso as dimensões sejam respeitadas.	<code>real_vector = real_matrix</code> <code>real_vector /= real_matrix</code>
abs	Módulo de uma matriz e/ou vetor	Retorna o valor absoluto

Função	Descrição
Sqrt	Extraí a raízes quadrada de cada elemento
Exp	Exponencial de elemento por elemento
Log	Realiza o logaritmo de elemento por elemento
Det	Calcula o determinante da matrix
Inv	Realiza a operação inversa de cada matriz
Transpose	Encontra a transposta da matriz
Zeros	Cria uma matriz de zeros. Zeros (rows, columns)
Ones	Gera uma matriz com número 1. Ones (rows, columns)
eye	Monta uma matriz identidade. eye (rows, columns)
submatrix	Gera uma matriz a partir de outra matriz. submatrix (arg, x, y, rows, columns).
buildmatrix	Comando utilizado para construir uma matriz. buildmatrix (arg, result, x, y)
InsertColumn	Inseri um vetor coluna em uma matriz
Size	Retorna o tamanho da matriz

```
6  library ieee;
7  library ieee_proposed;
8
9  use ieee.std_logic_1164.all;
10 use ieee.std_logic_unsigned.all;
11 use ieee.std_logic_arith.all;
12
13 use ieee_proposed.real_matrix_pkg.all;
..
```


Exemplo 1:

Dadas as matrizes a e b desenvolva um hardware em VHDL que realize a multiplicação entre estas matrizes quando o hardware for habilitado ($rst = 0$). Verifique o resultado desta multiplicação através de ferramenta de simulação.

Observação: Quando $rst = 1$, carrega os valores das matrizes.

$$a = \begin{bmatrix} 1.0 & 6.0 \\ 4.0 & 3.0 \\ 7.0 & 1.0 \\ 2.0 & 5.0 \end{bmatrix} \quad b = \begin{bmatrix} 2.0 & 0.5 & 3.0 \\ 7.0 & 1.0 & 0.0 \end{bmatrix}$$

Gerar arquivo de simulação - ModelSim

quit -sim -- sai da simulação

vcom top.vhd -- compila o arquivo top.vhd

vsim work.top -- simula a entidade top

add wave -- adiciona para as formas de ondas

force -- force um valor

run 1000 -- roda 1000ps a simulação

do simula.do

Exemplo 2:

Dada a matriz **M** e **n = 1.5**, desenvolva um hardware em VHDL que realize as operações indicadas em a) e b), quando o hardware estiver habilitado (rst = 1). Verifique os resultados através do ModelSim.

$$M = \begin{bmatrix} 0.5 & 0.0 & 1.0 \\ 2.0 & 1.5 & 1.0 \\ 0.0 & 0.5 & 2.0 \end{bmatrix}$$

a) M^T

b) $n * M^T + M * M^T$

Exemplo 3:

Dadas as matrizes A e C determine a matriz F.

$$A = \begin{bmatrix} 1.0 & 0.5 \\ 0.8 & 0.2 \end{bmatrix} \quad C = \begin{bmatrix} 0.0 & 1.0 \end{bmatrix} \quad F = \begin{bmatrix} CA \\ CA^2 \end{bmatrix}$$

Exemplo 4:

As matrizes A e B descrevem parte do comportamento de um sistema eletromecânico. Verifique se este sistema é controlável (usando VHDL).

Observação: o teste de controlabilidade de um sistema é dado por: $\det[(B \ AB \ A^2B \ \dots \ A^{n-1}B)]$. Sendo n a dimensão da matriz A. Se o determinante da matriz de controlabilidade $(B \ AB \ A^2B \ \dots \ A^{n-1}B)$ for diferente de zero, então o sistema é controlável. Se o sistema for controlável uma saída S deve ser igual 1, caso contrário $S = 0$.

$$A = \begin{bmatrix} 1.0 & 0.5 \\ 0.2 & 0.0 \end{bmatrix} \quad B = \begin{bmatrix} 1.0 \\ 0.0 \end{bmatrix}$$

Exemplo 5:

Dada a matriz A , encontre a inversa desta matriz caso o determinante seja diferente de zero. O hardware deverá enviar a mensagem “Gera” caso exista a inversa de A , caso contrário deve ser enviada a mensagem “Erro”. Em ambos os casos o hardware deverá informar o valor do determinante.

$$A = \begin{bmatrix} 1.0 & -0.2 & 2.0 \\ 0.0 & 0.5 & 0.1 \\ 4.0 & 0.0 & 2.0 \end{bmatrix}$$

Exemplo 6:

Idem ao exercício 5.

$$A = \begin{bmatrix} 1.0 & -0.5 \\ 2.5 & 3.0 \end{bmatrix}$$

Exemplo 7:

Um sistema de controle para satélite é apresentado na equação de estado abaixo. Supondo as condições iniciais $x_1(0) = 0$ e $x_2(0) = 1$, sendo x_1 relacionado ao deslocamento e x_2 relaciona a velocidade. Projete um hardware em VHDL que calcule os vetores de estados (deslocamento e velocidade) e o ganho $u(k)$ correspondente.

$$\begin{bmatrix} x_1(k+1) \\ x_2(k+1) \end{bmatrix} = \begin{bmatrix} 0.95 & 0.0825 \\ -1 & 0.65 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix}$$

$$u(k) = [10.0 \quad 3.5] \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix}$$

Sendo o ganho, $U_k = -Kx(k)$

Passo $k = 0$

O vetor de estado inicial é:

$$x(0) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Aplicando a lei de controle positiva:

$$u(0) = Kx(0) = \begin{bmatrix} 10.0 & 3.5 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$u(0) = 10(0) + 3.5(1) = 3.5$$

No caso de realimentação negativa:

$$u(0) = -Kx(0) = -3.5$$

Passo $k = 1$

$$x(1) = Ax(0) = \begin{bmatrix} 0.95 & 0.0825 \\ -1 & 0.65 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$x(1) = \begin{bmatrix} 0.95(0) + 0.0825(1) \\ -1(0) + 0.65(1) \end{bmatrix} = \begin{bmatrix} 0.0825 \\ 0.65 \end{bmatrix}$$

Agora, o ganho:

$$u(1) = Kx(1) = \begin{bmatrix} 10.0 & 3.5 \end{bmatrix} \begin{bmatrix} 0.0825 \\ 0.65 \end{bmatrix}$$

$$u(1) = 10(0.0825) + 3.5(0.65) = 0.825 + 2.275 = 3.1$$

No caso negativo:

$$u(1) = -3.1$$

Passo $k = 2$

$$x(2) = Ax(1) = \begin{bmatrix} 0.95 & 0.0825 \\ -1 & 0.65 \end{bmatrix} \begin{bmatrix} 0.0825 \\ 0.65 \end{bmatrix}$$

$$x(2) = \begin{bmatrix} 0.95(0.0825) + 0.0825(0.65) \\ -1(0.0825) + 0.65(0.65) \end{bmatrix}$$

$$x(2) = \begin{bmatrix} 0.078375 + 0.053625 \\ -0.0825 + 0.4225 \end{bmatrix} = \begin{bmatrix} 0.132 \\ 0.34 \end{bmatrix}$$

Então,

$$u(2) = Kx(2) = \begin{bmatrix} 10.0 & 3.5 \end{bmatrix} \begin{bmatrix} 0.132 \\ 0.34 \end{bmatrix}$$

$$u(2) = 10(0.132) + 3.5(0.34) = 1.32 + 1.19 = 2.51$$

No caso negativo:

$$u(2) = -2.51$$

Passo $k = 3$

$$x(3) = Ax(2) = \begin{bmatrix} 0.95 & 0.0825 \\ -1 & 0.65 \end{bmatrix} \begin{bmatrix} 0.132 \\ 0.34 \end{bmatrix}$$

$$x(3) = \begin{bmatrix} 0.95(0.132) + 0.0825(0.34) \\ -1(0.132) + 0.65(0.34) \end{bmatrix}$$

$$x(3) = \begin{bmatrix} 0.1254 + 0.02805 \\ -0.132 + 0.221 \end{bmatrix} = \begin{bmatrix} 0.15345 \\ 0.089 \end{bmatrix}$$

Agora,

$$u(3) = Kx(3) = \begin{bmatrix} 10.0 & 3.5 \end{bmatrix} \begin{bmatrix} 0.15345 \\ 0.089 \end{bmatrix}$$

$$u(3) = 10(0.15345) + 3.5(0.089) = 1.5345 + 0.3115 = 1.846$$

No caso negativo:

$$u(3) = -1.846$$

Exemplo 8:

Desenvolva um hardware em VHDL que permita ao usuário entrar com as matrizes de estados (A e B) e o hardware execute as seguintes operações:

- a) $A * B$;
- b) $A^T * B$;
- c) $A + A^n$, sendo n definido pelo usuário. Os resultados desta operação deverão ser apresentados para cada iteração, ou seja, se $n = 3$, $A + A^0, A + A^1, A + A^2$ e $A + A^3$.

Exercício 1:

Deseja se aplicar a fórmula de Ackermann para projetar o ganho do controlador K para um satélite. **Projete** um hardware em VHDL **que encontre o ganho do controlador com base no período (T) estabelecido pelo usuário**. As informações sobre os parâmetros de controle são apresentados nas equações abaixo:

Polinômio característico

$$\alpha_c(\emptyset) = \emptyset^2 + \alpha_1\emptyset + \alpha_2I$$

Sendo: $\emptyset = \begin{bmatrix} 1.0 & T \\ 0.0 & 0.1 \end{bmatrix}$ $\alpha_1 = -1.6$ $\alpha_2 = 0.7$

A matriz de controlabilidade

$$C^{-1} = [\delta \quad \emptyset\delta]^{-1}$$

Sendo: $\delta = \begin{bmatrix} T^2 \\ 2 \\ T \end{bmatrix}$

Ganho do Controlador

$$K = [0 \quad 1]C^{-1}\alpha_c(\emptyset)$$

Model Predictive Control - MPC

- $Y = Fx(k_i) + \varphi \Delta U$

$$x_m(k+1) = A_m x_m(k) + B_m u(k)$$

$$y(k) = C_m x_m(k)$$

$$\overbrace{\begin{bmatrix} x(k+1) \\ \Delta x_m(k+1) \\ y_m(k+1) \end{bmatrix}} = \overbrace{\begin{bmatrix} A_m & 0_m^T \\ C_m A_m & 1 \end{bmatrix}}^A \overbrace{\begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix}}^{x(k)} + \overbrace{\begin{bmatrix} B_m \\ C_m B_m \end{bmatrix}}^B \Delta u(k)$$

$$y(k) = \overbrace{\begin{bmatrix} 0_m & 1 \end{bmatrix}}^C \overbrace{\begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix}}$$

$$\overbrace{\begin{bmatrix} y(k+1) \\ y(k+2) \\ \vdots \\ y(k+n) \end{bmatrix}}^Y = \overbrace{\begin{bmatrix} CA \\ CA^2 \\ CA^3 \\ \vdots \\ CA^{N_p} \end{bmatrix}}^F x(k_i) + \overbrace{\begin{bmatrix} CB & 0 & 0 & \dots & 0 \\ CAB & CB & 0 & \dots & 0 \\ CA^2 B & CAB & CB & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ CA^{N_p-1} B & CA^{N_p-2} B & CA^{N_p-3} B & \dots & CA^{N_p-N_c} B \end{bmatrix}}^{\varphi} \Delta U$$

$$K_{mpc} = [1 \ 0 \ \dots \ 0](\varphi^T \varphi + \bar{R})^{-1}(\varphi^T F)$$

$$K_y = [1 \ 0 \ \dots \ 0](\varphi^T \varphi + \bar{R})^{-1}(\varphi^T \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix})$$

Supondo que uma planta é descrita pela equação de estado:

$$\begin{aligned}x_m(k+1) &= ax_m(k) + bu(k) \\ y(k) &= x_m(k)\end{aligned}$$

Projeto 1 - Entrega Individual:

- 1) Código em VHDL **(3,5)**;
- 2) Código em Matlab **(2,0)**;
- 3) Simulação em ModelSim (com arquivo executável) **(4,0)**;
- 4) Resultado no Matlab **(0,5)**;

Considere:

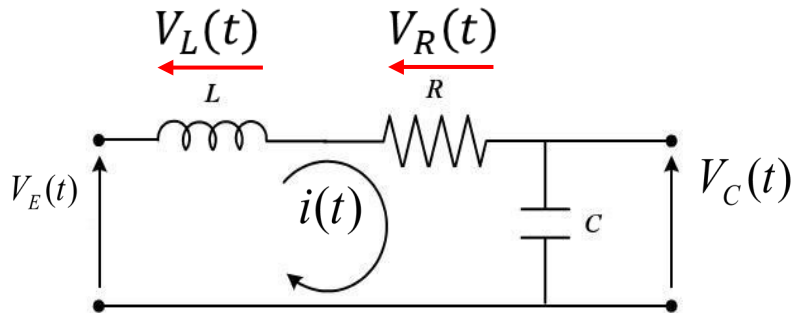
$$N_p = 10 \text{ e } N_c = 4$$

$$\text{Setpoint}(k_i) = 1$$

$$x(10) = [0.1 \quad 0.2]^T$$

- a) Encontre as matrizes aumentadas do modelo no espaço de estado;
- b) Determine as matrizes F e ϕ ;
- c) Calcule os parâmetros que formam a predição das futuras saídas: $\phi^T \phi$, $\phi^T F$ e $\phi^T \bar{R}_s$;
- d) Encontre a solução ótima $\Delta U(10)$ para $r_w = 0$ e $r_w = 10$. $\Delta U = (\phi^T \phi + \bar{R})^{-1} \phi^T (R_s - Fx(k_i))$
- e) Obtenha os valores dos ganhos do controlador (K_{mpc} , K_y e K_x).

Observação: Desenvolva um arquivo de simulação .do para testar as funcionalidades do hardware no ModelSim.



$$V_E(t) = V_R(t) + V_L(t) + V_C(t)$$

$$V_E(t) = RC \frac{dv_C(t)}{dt} + LC \frac{d^2v_C}{dt^2} + V_C(t)$$

$$V_E(s) = RCsV_C(s) + LCs^2V_C(s) + V_C(s)$$

$$V_E(s) = V_C(s)(RCs + LCs^2 + 1)$$

$$V_R(t) = Ri(t)$$

$$V_L(t) = L \frac{di(t)}{dt}$$

$$i(t) = C \frac{dv_C(t)}{dt}$$

FUNÇÃO DE TRANSFERÊNCIA

$$\frac{V_C(s)}{V_E(s)} = \frac{1}{LCs^2 + RCs + 1} = \frac{1/LC}{s^2 + \frac{R}{L}s + \frac{1}{LC}}$$

$$R = 1200;$$

$$L = 0.4;$$

$$C = 100 \cdot 10^{-9};$$

$$\frac{2.5 \cdot 10^{-7}}{s^2 + 3000s + 2.5 \cdot 10^{-7}}$$

$$\frac{V_C(s)}{V_E(s)} = \frac{1}{LCs^2 + RCs + 1} = \frac{1/LC}{s^2 + \frac{R}{L}s + \frac{1}{LC}}$$

$$\frac{2.5 * 10^{-7}}{s^2 + 3000s + 2.5 * 10^{-7}}$$

$$\begin{aligned} R &= 1200; \\ L &= 0.4; \\ C &= 100 * 10^{-9}; \end{aligned}$$

ESPAÇO DE ESTADO

$$\dot{x} = Ax + Bv$$

$$y = Cx + Dv$$

$$x_1 = V_C(t)$$

$$x_2 = \frac{dV_C(t)}{dt}$$

$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = \frac{V_E(t)}{LC} - \frac{R}{L}x_2 - \frac{x_1}{LC}$$

$$V_E(t) = RC \frac{dv_C(t)}{dt} + LC \frac{d^2v_C}{dt^2} + V_C(t)$$

$$\frac{d^2v_C}{dt^2} = \frac{V_E(t)}{LC} - \frac{R}{L} \frac{dv_C(t)}{dt} - \frac{V_C(t)}{LC}$$

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1/LC & R/L \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1/LC \end{bmatrix} V_E(t)$$

$$y(t) = [1 \quad 0] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$


```
R = 1200;
L = 0.4;
C = 100*10^-9;
Ts = 0.01;
%Calculo dos coeficientes
RL = R/L;
LC = 1/L*C;

%Montagem da Função de Transferência
num = LC;
den = [1 RL LC];
FT = tf(num,den);

%Cria um espaço de estado
[As,Bs,Cs,Ds] = tf2ss(num,den);
Modeloss = ss(As,Bs,Cs,Ds);

%Converte FT continua para digital
[numD,denD] = c2dm(num,den,Ts);

%Converte espaço contínuo para discreto
[Ad,Bd,Cd,Dd] = c2dm(As,Bs,Cs,Ds,Ts,'zoh');
```

FT =

$$\frac{2.5e-07}{s^2 + 3000 s + 2.5e-07}$$

Continuous-time transfer function.

Modeloss =

```
a =
      x1      x2
x1    -3000   -2.5e-07
x2         1         0
```

```
b =
      u1
x1     1
x2     0
```

```
c =
      x1      x2
y1         0   2.5e-07
```

```
d =
      u1
y1     0
```

Continuous-time state-space model.

Ad =

```
0.0000   -0.0000
0.0003    1.0000
```

Bd =

```
1.0e-03 *
```

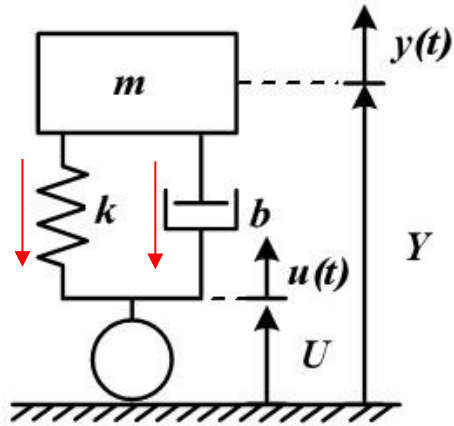
```
0.3333
0.0032
```

Cd =

```
1.0e-06 *
0      0.2500
```

Dd =

```
0
```

$$s^2 Y(s) = -\frac{k}{m} Y(s) - \frac{b}{m} s Y(s) + \frac{1}{m} U(s)$$

$$Y(s) \left[s^2 + \frac{b}{m} s + \frac{k}{m} \right] = \frac{1}{m} U(s)$$

$$\frac{Y(s)}{U(s)} = \frac{1/m}{s^2 + \frac{b}{m} s + \frac{k}{m}}$$

$$m\ddot{y}(t) = u(t) - ky(t) - b\dot{y}(t)$$

$$\ddot{y}(t) = -\frac{k}{m}y(t) - \frac{b}{m}\dot{y}(t) + \frac{1}{m}u(t)$$

$$x_1 = y(t)$$

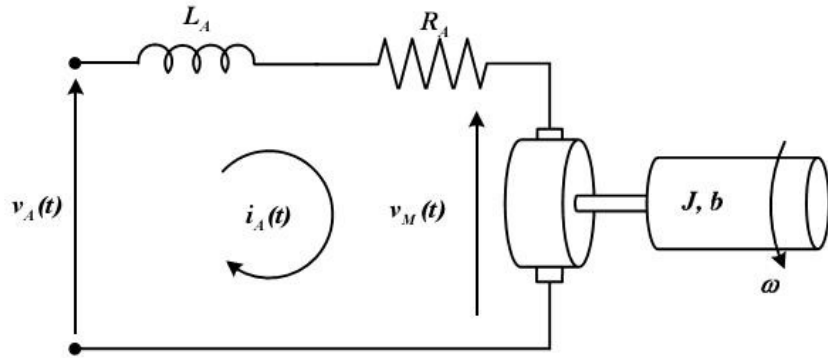
$$x_2 = \dot{y}(t)$$

$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = -\frac{k}{m}x_1 - \frac{b}{m}x_2 + \frac{1}{m}u(t)$$

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k/m & b/m \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1/m \end{bmatrix} u(t)$$

$$y(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$



$$v_A(t) = L_A \frac{di_A}{dt} + R_A i_A(t) + v_M(t)$$

$$T_M(t) = k_M i_A(t)$$

$$v_M(t) = k_b \omega(t)$$

$$J\dot{\omega}(t) = T_M(t) - T_b(t) - T_c(t)$$

$$T_c(t) = 0$$

$$J\dot{\omega}(t) = k_M i_A(t) - b\omega(t)$$

$$V_A(s) = L_A s I_A(s) + R_A I_A(s) + V_M(s)$$

$$T_M(s) = k_M I_A(s)$$

$$I_A(s) = \frac{T_M(s)}{k_M}$$

$$Js\omega(s) = T_M(s) - b\omega(s)$$

$$T_M(s) = \omega(s)(Js + b) \therefore I_A(s) = \frac{\omega(s)(Js + b)}{k_M}$$

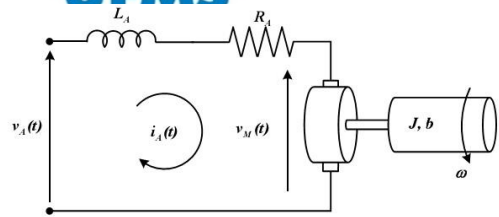
$$V_M(s) = k_b \omega(s)$$

$$V_A(s) = (L_A s + R_A) I_A(s) + V_M(s)$$

$$V_A(s) = \frac{(L_A s + R_A)(Js + b)}{k_M} \omega(s) + k_b \omega(s)$$

$$V_A(s) = \left(\frac{(L_A s + R_A)(Js + b) + k_M k_b}{k_M} \right) \omega(s)$$

$$\frac{\omega(s)}{V_A(s)} = \frac{k_M}{(L_A s + R_A)(Js + b) + k_M k_b}$$



$$x_1(t) = \omega(t)$$

$$x_2(t) = i_A(t)$$

$$\dot{x}_1(t) = \dot{\omega}(t)$$

$$\dot{x}_2(t) = \frac{di_A(t)}{dt}$$

$$\dot{\omega}(t) = \frac{k_M}{J} i_A(t) - \frac{b}{J} \omega(t)$$

$$\dot{x}_1(t) = \frac{k_M}{J} x_2(t) - \frac{b}{J} x_1(t)$$

$$\frac{di_A}{dt} = -\frac{R_A i_A(t)}{L} - \frac{k_b \omega(t)}{L} + v_A(t)$$

$$\dot{x}_2(t) = -\frac{k_b}{L} x_1(t) - \frac{R_A}{L} x_2(t) + \frac{1}{L} v_A(t)$$

$$\begin{bmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \end{bmatrix} = \begin{bmatrix} -b/J & k_M/J \\ -k_b/L & -R_A/L \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} + \begin{bmatrix} 0 \\ 1/L \end{bmatrix} v_A(t)$$

$$y(t) = [1 \quad 0] \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}$$

Model Predictive Control - MPC

- $Y = Fx(k_i) + \varphi \Delta U$

$$x_m(k+1) = A_m x_m(k) + B_m u(k)$$

$$y(k) = C_m x_m(k)$$

$$\overbrace{\begin{bmatrix} \Delta x_m(k+1) \\ y_m(k+1) \end{bmatrix}}^{x(k+1)} = \overbrace{\begin{bmatrix} A_m & 0_m^T \\ C_m A_m & 1 \end{bmatrix}}^A \overbrace{\begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix}}^{x(k)} + \overbrace{\begin{bmatrix} B_m \\ C_m B_m \end{bmatrix}}^B \Delta u(k)$$

$$y(k) = \overbrace{\begin{bmatrix} 0_m & 1 \end{bmatrix}}^C \begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix}$$

Crie um arquivo executável no Matlab e Construa as matrizes aumentadas dos circuitos modelados no trabalho

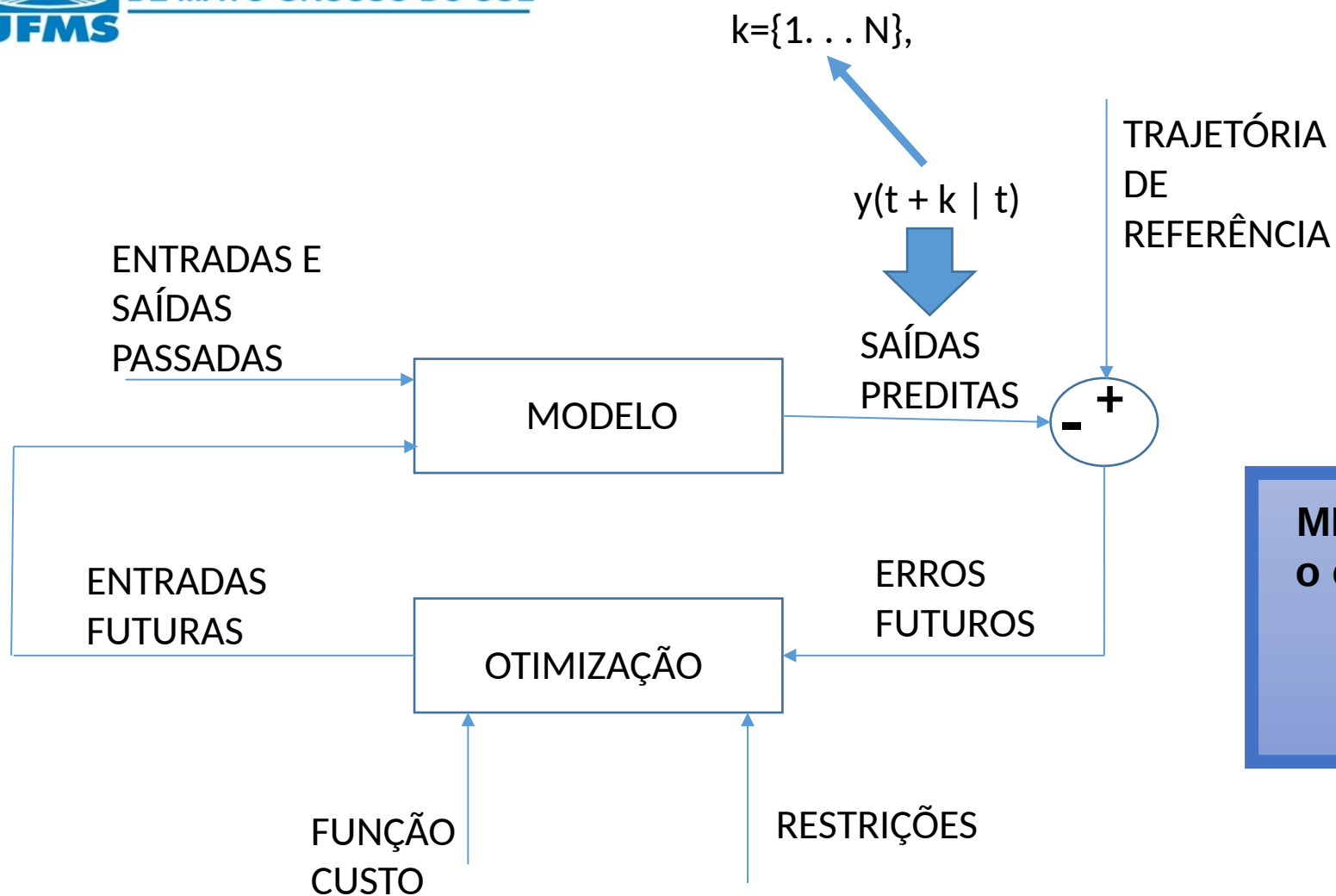
$$\overbrace{\begin{bmatrix} y(k+1) \\ y(k+2) \\ \vdots \\ y(k+n) \end{bmatrix}}^Y = \overbrace{\begin{bmatrix} CA \\ CA^2 \\ CA^3 \\ \vdots \\ CA^{N_p} \end{bmatrix}}^F x(k_i) + \overbrace{\begin{bmatrix} CB & 0 & 0 & \dots & 0 \\ CAB & CB & 0 & \dots & 0 \\ CA^2 B & CAB & CB & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ CA^{N_p-1} B & CA^{N_p-2} B & CA^{N_p-3} B & \dots & CA^{N_p-N_c} B \end{bmatrix}}^{\varphi} \Delta U$$

$$K_{mpc} = [1 \ 0 \ \dots \ 0](\varphi^T \varphi + \bar{R})^{-1}(\varphi^T F)$$

$$K_y = [1 \ 0 \ \dots \ 0](\varphi^T \varphi + \bar{R})^{-1}(\varphi^T \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix})$$

Controle Preditivo baseado em Modelos

- A técnica de Controlador Preditivo é utilizada em diversas áreas devido a robustez em trabalhar com sistemas não lineares, com SISO e MIMO.
- O Controle Preditivo baseado em Modelos (MPC) desenvolve a obtenção do sinal de controle por meio da minimização de uma função objetivo (função custo) considerando o erro futuro, além disso, possibilita a utilização de restrições nas variáveis de processo. **Princípio do horizonte móvel.**
- As técnicas convencionais do MPC baseiam-se no horizonte finito e usam um horizonte de controle N_c , um horizonte de predição N_p , além da parte de desempenho que correspondem a minimização da função custo.



MPC foi desenvolvido considerando o conceito de predição e a obtenção do sinal de controle através da minimização de uma função objetivo.

O modelo dinâmico da planta é primordial para projetar o Controlador Preditivo.

- O horizonte de controle (N_c) é o horizonte sobre o qual encontra-se a ação de verificação.
- O horizonte de previsão (N_p) é o horizonte sobre o qual prediz-se o comportamento autônomo do sistema.
- **O horizonte de controle é escolhido para ser menor ou igual ao horizonte de predição.**
- **Função Custo ou função objetivo** – Trata-se de uma função erro com base na diferença entre as respostas desejadas e reais. Esta função é calculada dentro da janela de predição através de algoritmos de otimização.

Considerando o instante da amostragem $k_i, k_i > 0$

- O vetor variável de estado é obtido através de medição $x(k_i)$. Este vetor de estado $x(k_i)$ fornece informações atuais da planta.

Observação: Em muitos casos o vetor de estado $x(k_i)$ não pode medido.

- A predição do estado e da saída são baseadas em dois parâmetros para projetar o Controlador Preditivo.
 - **Nc** = o número de parâmetros necessário para alcançar a trajetória de controle futura.
 - **Np** = o tamanho da janela de otimização da junção custo.

- Espaço de estados discreto: predição **1-passo** a frente

$$\begin{aligned}x_m(k+1) &= A_m x_m(k) + B_m u(k) \\ y(k) &= C_m x_m(k)\end{aligned}$$

- Como prever **1-passo** a frente? $y(k+1)$?

$$y(k+1) = C_m x_m(k+1)$$

$$y(k+1) = C_m (A_m x_m(k) + B_m u(k))$$

- Como prever **2-passos** a frente?

$$y(k + 2) = C_m x_m(k + 2)$$

$$x_m(k + 2) = A_m x_m(k + 1) + B_m u(k + 1)$$

$$x_m(k + 2) = A_m (A_m x_m(k) + B_m u(k)) + B_m u(k + 1)$$

$$x_m(k + 2) = A_m^2 x_m(k) + A_m B_m u(k) + B_m u(k + 1)$$

$$y(k + 2) = C_m (A_m^2 x_m(k) + A_m B_m u(k) + B_m u(k + 1))$$

$$y(k + 2) = C_m A_m^2 x_m(k) + C_m A_m B_m u(k) + C_m B_m u(k + 1)$$

$$y(k + 1) = C_m A_m x_m(k) + C_m B_m u(k)$$

$$y(k + 2) = C_m A_m^2 x_m(k) + C_m A_m B_m u(k) + C_m B_m u(k + 1)$$

$$y(k + n) = C_m A_m^n x_m(k) + C_m (A_m^{n-1} B_m u(k) + A_m^{n-2} B_m u(k + 1) + \dots + B_m u(k + n - 1))$$

$$y(k + n) = C_m A_m^n x_m(k) + \sum_{i=0}^{n-1} C_m A_m^{n-i-1} B_m u(k + i)$$

$$\Delta u(k_i), \Delta u(k_i + 1), \dots, \Delta u(k_i + N_c - 1),$$

$$x(k_i + N_p | k_i) = A^{N_p} x(k_i) + A^{N_p-1} B \Delta u(k_i) + A^{N_p-2} B \Delta u(k_i + 1) \\ + \dots + A^{N_p-N_c} B \Delta u(k_i + N_c - 1).$$

$$y(k_i + N_p | k_i) = C A^{N_p} x(k_i) + C A^{N_p-1} B \Delta u(k_i) + C A^{N_p-2} B \Delta u(k_i + 1) \\ + \dots + C A^{N_p-N_c} B \Delta u(k_i + N_c - 1).$$

$$Y = [y(k_i + 1 | k_i) \ y(k_i + 2 | k_i) \ y(k_i + 3 | k_i) \ \dots \ y(k_i + N_p | k_i)]^T$$

$$\Delta U = [\Delta u(k_i) \ \Delta u(k_i + 1) \ \Delta u(k_i + 2) \ \dots \ \Delta u(k_i + N_c - 1)]^T$$

- As matrizes F e ϕ são definidas conforme o **horizonte de predição** N_p e **horizonte de controle** N_c :

$$Y = Fx(k_i) + \phi\Delta U$$

$$\overbrace{\begin{bmatrix} y(k+1) \\ y(k+2) \\ \vdots \\ y(k+n) \end{bmatrix}}^Y = \overbrace{\begin{bmatrix} CA \\ CA^2 \\ CA^3 \\ \vdots \\ CA^{N_p} \end{bmatrix}}^F x(k_i) + \overbrace{\begin{bmatrix} CB & 0 & 0 & \dots & 0 \\ CAB & CB & 0 & \dots & 0 \\ CA^2B & CAB & CB & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ CA^{N_p-1}B & CA^{N_p-2}B & CA^{N_p-3}B & \dots & CA^{N_p-N_c}B \end{bmatrix}}^{\phi} \Delta U$$

- A , B e C são as matrizes aumentadas do sistema.

- Ricker (1991) : adição de um **integrador**
- SISO



SUGESTÃO: ITEM 1.2 DO
LIVRO TEXTO
“State-space Models
with Embedded
Integrator”

$$\overbrace{\begin{bmatrix} \Delta x_m(k+1) \\ y_m(k+1) \end{bmatrix}}^{x(k+1)} = \overbrace{\begin{bmatrix} A_m & o_m^T \\ C_m A_m & 1 \end{bmatrix}}^A \overbrace{\begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix}}^{x(k)} + \overbrace{\begin{bmatrix} B_m \\ C_m B_m \end{bmatrix}}^B \Delta u(k)$$

$$y(k) = \overbrace{\begin{bmatrix} o_m & 1 \end{bmatrix}}^C \begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix}$$

$$o_m = \overbrace{\begin{bmatrix} 0 & \dots & 0 \end{bmatrix}}^{n_1}$$

n_1 tem a dimensão do vetor de estado (não aumentado).

- MIMO

$$\overbrace{\begin{bmatrix} \Delta x_m(k+1) \\ y_m(k+1) \end{bmatrix}}^{x(k+1)} = \overbrace{\begin{bmatrix} A_m & o_m^T \\ C_m A_m & I_{qxq} \end{bmatrix}}^A \overbrace{\begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix}}^{x(k)} + \overbrace{\begin{bmatrix} B_m \\ C_m B_m \end{bmatrix}}^B \Delta u(k)$$

$$y(k) = \overbrace{\begin{bmatrix} o_m & I_{qxq} \end{bmatrix}}^C \begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix}$$

$$o_m = \overbrace{\begin{bmatrix} 0 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 0 \end{bmatrix}}^{n_1} \Bigg\} q$$

- Função de **custo**:
- Como MPC baseia-se no modelo/planta, o desempenho dessa técnica fica vulnerável aos erros de modelagem e/ou os distúrbios não são computados.

$$J = \underbrace{(R_s - Y)^T (R_s - Y)}_{\text{Penaliza o erro da saída para a referência}} + \underbrace{\Delta U^T \bar{R} \Delta U}_{\text{Penaliza uma variação brusca do controle}}$$

Penaliza o erro da saída para a referência

Penaliza uma variação brusca do controle

$$J = \sum_{k=1}^n (e_{k+1}^2 + \lambda(u_k - u_{ss})^2 + \mu(\Delta u_k)^2)$$

Otimização

Para um determinado **setpoint** $r(k_i)$ e o tempo de amostragem k_i dentro da janela de predição, o objetivo do controlador preditivo é trazer a saída prevista o mais próximo possível para o sinal de setpoint, considerando que na janela de otimização o setpoint é constante.

O vetor que contém o setpoint é dado por:

$$R_s^T = \overbrace{[1 \ 1 \ \dots \ 1]}^{N_p} r(k_i)$$

A função custo J reflete o objetivo do controlador.

$$J = (R_s - Y)^T (R_s - Y) + \Delta U^T \bar{R} \Delta U$$

Sendo:

$$\bar{R} = r_w I_{N_c \times N_c}$$

Sendo

$r_w \geq 0$, peso para sintonia do controlador preditivo.

$I_{N_c \times N_c}$, matriz identidade de dimensão N_c

Derivada da função custo $J = (R_s - Y)^T (R_s - Y) + \Delta U^T \bar{R} \Delta U$

$$\frac{\partial J}{\partial \Delta U} = -2\phi^T (R_s - Fx(k_i)) + 2(\phi^T \phi + \bar{R})\Delta U \qquad \frac{\partial J}{\partial \Delta U} = 0$$

A solução ótima para o sinal de controle é dada por:

$$\Delta U = (\phi^T \phi + \bar{R})^{-1} \phi^T (R_s - Fx(k_i)) \text{ Corrigir a equação}$$

Desde que a **matriz Hessian** existe.

$$(\phi^T \phi + \bar{R})^{-1}$$

Sistema de controle em malha fechada – Ganhos do Controlador preditivo

$$\Delta u(k_i) = \overbrace{[1 \ 0 \cdots 0]}^{N_c} (\Phi^T \Phi + \bar{R})^{-1} (\Phi^T R_s r(k_i) - \Phi^T F x(k_i))$$

$$= K_y r(k_i) - K_{mpc} x(k_i)$$

K_y é o primeiro elemento de $(\Phi^T \Phi + \bar{R})^{-1} \Phi^T R_s$

K_{mpc} é a primeira linha de $(\Phi^T \Phi + \bar{R})^{-1} \Phi^T F$

$$K_{MPC} = \overbrace{[1 \ 0 \ 0 \ 0]}^{N_c} (\Phi^T \Phi + \bar{R})^{-1} (\Phi^T F) \qquad K_{mpc} = [K_x \ K_y]$$

Ou seja:

- a parcela $(\Phi^T \Phi + \bar{R})^{-1} \Phi^T R_s r(k_i)$ está relacionada com a busca pelo setpoint.
- A parcela $(\Phi^T \Phi + \bar{R})^{-1} \Phi^T F x(k_i)$ está relacionada com a realimentação.

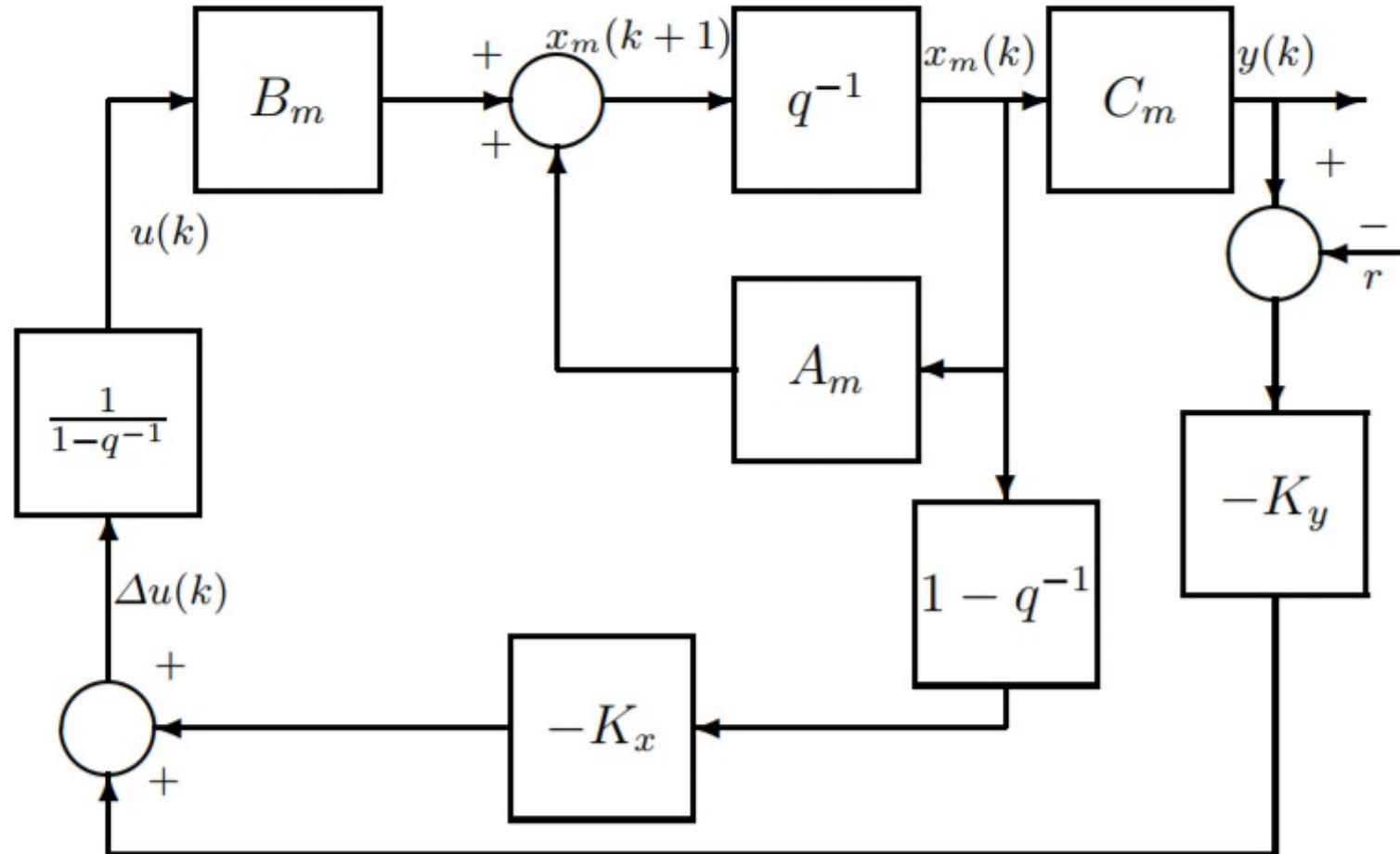
- Cálculo do ΔU : Horizonte de controle N_c

$$\Delta U = (\varphi^T \varphi + \bar{R})^{-1} \varphi^T (\bar{R}_s r(k_i) - Fx(k_i))$$

- Cálculo dos ganhos:

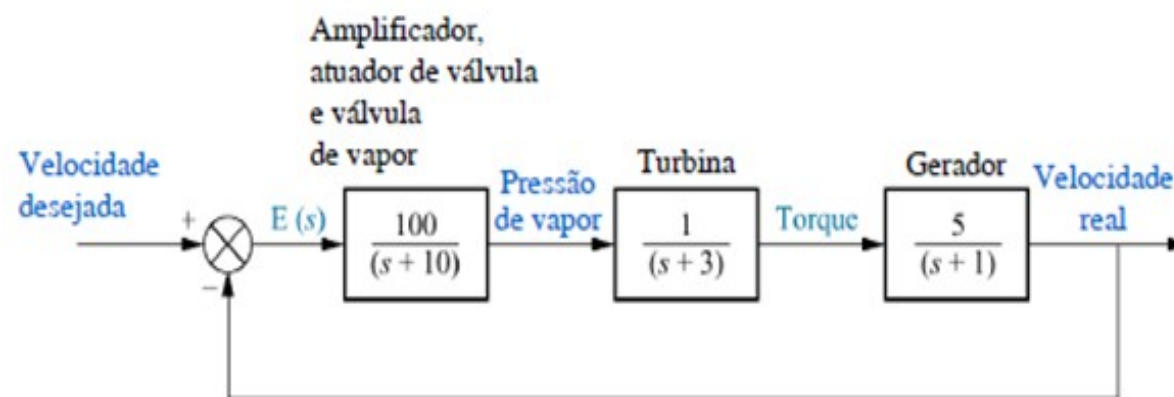
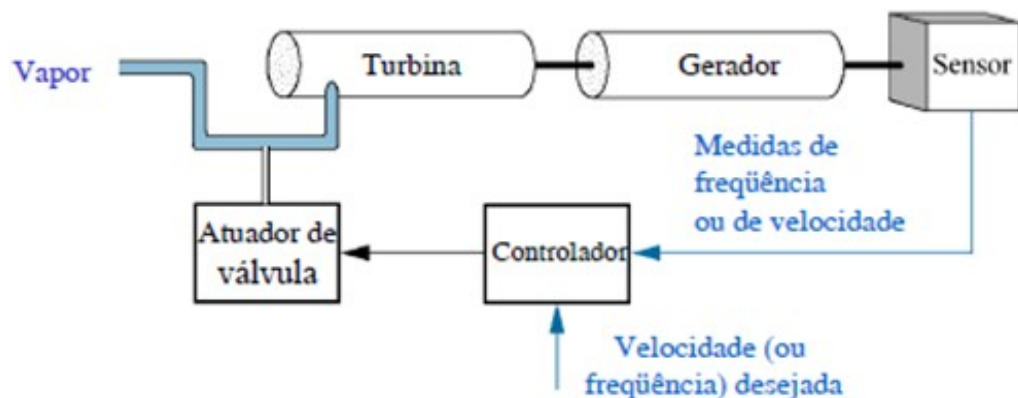
$$K_{mpc} = [1 \quad 0 \quad \dots \quad 0] (\varphi^T \varphi + \bar{R})^{-1} (\varphi^T F)$$

$$K_y = [1 \quad 0 \quad \dots \quad 0] (\varphi^T \varphi + \bar{R})^{-1} (\varphi^T \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix})$$



Exemplo (adaptado do Nise): Os controladores de velocidade encontram uma ampla aplicação nos setores industrial e doméstico. Os diagramas das figuras abaixo, mostra aplicação dessa correspondente ao par turbina-gerador. Regulando a velocidade, o sistema de controle assegura que a frequência gerada permaneça dentro da tolerância.

Projete um Controle Preditivo baseado em Modelo para o sistema Turbina-Gerador com uma velocidade desejada de 100 m/s.



Passo 1 – Representação no espaço de estado (digital) em malha aberta da planta:

$$G(s) = \frac{500}{s^3 + 14s^2 + 43s + 30} = \frac{500}{(s + 10)(s + 3)(s + 1)}$$

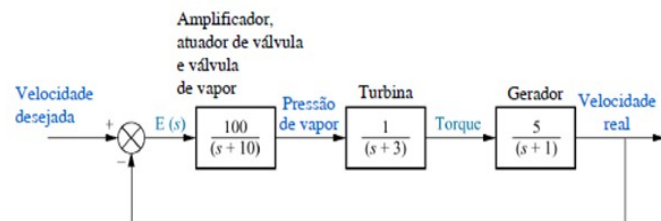
Observador: Convertendo função de transferência para espaço de estado → Nise pag. 102

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} -14 & -43 & -30 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} u$$

$$y = [0 \quad 0 \quad 500] \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$\begin{bmatrix} x_1(k+1) \\ x_2(k+1) \\ x_3(k+1) \end{bmatrix} = \begin{bmatrix} 0.1580 & -2.2494 & -1.5025 \\ 0.0501 & 0.8591 & -0.0958 \\ 0.0032 & 0.0948 & 0.9964 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \\ x_3(k) \end{bmatrix} + \begin{bmatrix} 0.0501 \\ 0.0032 \\ 0.0001 \end{bmatrix} u(k)$$

$$y(k) = [0 \quad 0 \quad 500] \begin{bmatrix} x_1(k) \\ x_2(k) \\ x_3(k) \end{bmatrix}$$



Passo 2 – Encontrar as matrizes aumentadas:

$$x(k+1) = \overbrace{\begin{bmatrix} 0.1580 & -2.2494 & -1.5025 & 0.0 \\ 0.0501 & 0.8591 & -0.0958 & 0.0 \\ 0.0032 & 0.0948 & 0.9964 & 0.0 \\ 1.5963 & 47.3898 & 498.2123 & 1.0 \end{bmatrix}}^A x(k) + \overbrace{\begin{bmatrix} 0.0501 \\ 0.0032 \\ 0.0001 \\ 0.0596 \end{bmatrix}}^B \Delta(k)$$

$$y(k) = \overbrace{[0 \ 0 \ 0 \ 1]}^C x(k)$$

$$o_m = [0 \ 0 \ 0]$$

$$A_m = \begin{bmatrix} 0.1580 & -2.2494 & -1.5025 \\ 0.0501 & 0.8591 & -0.0958 \\ 0.0032 & 0.0948 & 0.9964 \end{bmatrix}$$

$$B_m = \begin{bmatrix} 0.0501 \\ 0.0032 \\ 0.0001 \end{bmatrix}$$

$$C_m = [0 \ 0 \ 500]$$

$$\overbrace{\begin{bmatrix} \Delta x_m(k+1) \\ y_m(k+1) \end{bmatrix}}^{x(k+1)} = \overbrace{\begin{bmatrix} A_m & 0_m^T \\ C_m A_m & 1 \end{bmatrix}}^A \overbrace{\begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix}}^{x(k)} + \overbrace{\begin{bmatrix} B_m \\ C_m B_m \end{bmatrix}}^B \Delta u(k)$$

$$y(k) = \overbrace{[0_m \ 1]}^C \begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix}$$

Passo 3 – Encontrar as matrizes F e Φ :

Fazendo:

$N_c = 4$;

$N_p = 9$;

$$F = 1.0 * 10^3 \begin{bmatrix} 0.0016 & 0.0474 & 0.4982 & 0.0010 \\ 0.0058 & 0.1317 & 0.9877 & 0.0010 \\ 0.0123 & 0.2411 & 1.4610 & 0.0010 \\ 0.0203 & 0.3654 & 1.9125 & 0.0010 \\ 0.0292 & 0.4970 & 2.3384 & 0.0010 \\ 0.0386 & 0.6303 & 2.7368 & 0.0010 \\ 0.0480 & 0.7616 & 3.1069 & 0.0010 \\ 0.0572 & 0.8882 & 3.4489 & 0.0010 \\ 0.0661 & 1.0086 & 3.7638 & 0.0010 \end{bmatrix}$$

Lembrete:

$$C_{1 \times 4} A_{4 \times 4}$$

$$\begin{bmatrix} y(k+1) \\ y(k+2) \\ \vdots \\ y(k+n) \end{bmatrix} = \begin{bmatrix} CA \\ CA^2 \\ CA^3 \\ \vdots \\ CA^{N_p} \end{bmatrix} x(k_i) + \begin{bmatrix} CB & 0 & 0 & \dots & 0 \\ CAB & CB & 0 & \dots & 0 \\ CA^2B & CAB & CB & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ CA^{N_p-1}B & CA^{N_p-2}B & CA^{N_p-3}B & \dots & CA^{N_p-N_c}B \end{bmatrix} \Delta U$$

$$\Phi = \begin{bmatrix} 0.0596 & 0.0 & 0.000 & 0.000 \\ 0.3502 & 0.0596 & 0.000 & 0.000 \\ 0.8890 & 0.3502 & 0.0596 & 0.0000 \\ 1.6178 & 0.8890 & 0.3502 & 0.0596 \\ 2.4696 & 1.6178 & 0.8890 & 0.3502 \\ 3.3878 & 2.4696 & 1.6178 & 0.8890 \\ 4.3297 & 3.3878 & 2.4696 & 1.6178 \\ 5.2650 & 4.3297 & 3.3878 & 2.4696 \\ 6.1730 & 5.2650 & 4.3297 & 3.3878 \end{bmatrix}$$

Passo 4 – Encontrar o sinal de controle Δu .

$$\Delta U = (\Phi^T \Phi + \bar{R})^{-1} (\Phi^T R_s - \Phi^T F x(k_i))$$

$$\Phi^T F_{4 \times 4} \quad x(k_i)_{4 \times 1}$$

$$R_s^T = [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] * 100$$

$$r(k_i) = 100$$

$$\Phi^T_{4 \times 9} R_s = \begin{bmatrix} 100 \\ 100 \\ 100 \\ 100 \\ 100 \\ 100 \\ 100 \\ 100 \\ 100 \end{bmatrix}$$

$$\bar{R} = r_w I_{N_c \times N_c} \quad r_w = 0.2$$

$$\bar{R}=0,2 \begin{bmatrix} 1.0 & 0 & 0 & 0 \\ 0 & 1.0 & 0 & 0 \\ 0 & 0 & 1.0 & 0 \\ 0 & 0 & 0 & 1.0 \end{bmatrix}$$

$$\Delta U = 10^{-12} \begin{bmatrix} 0.2050 \\ -0.6415 \\ 0.7029 \\ -0.0439 \end{bmatrix}$$

Passo 5 – Encontrar o valor dos ganhos.

$$K_{MPC} = \overbrace{[1 \ 0 \ 0 \ 0]}^{Nc} (\phi^T \phi + \bar{R})^{-1} (\phi^T F)$$

$$K_y = [1 \ 0 \ \dots \ 0] (\varphi^T \varphi + \bar{R})^{-1} (\varphi^T \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix})$$

$$R_s = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

Kmpc =

1.0e+03 *

(0.0199 0.5032 4.3529 0.0058)

$k_x = 4352.9$

$k_y = 5.8$

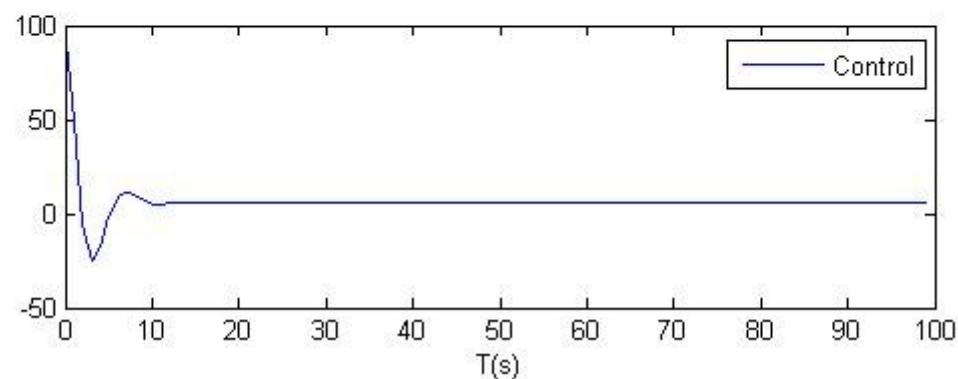
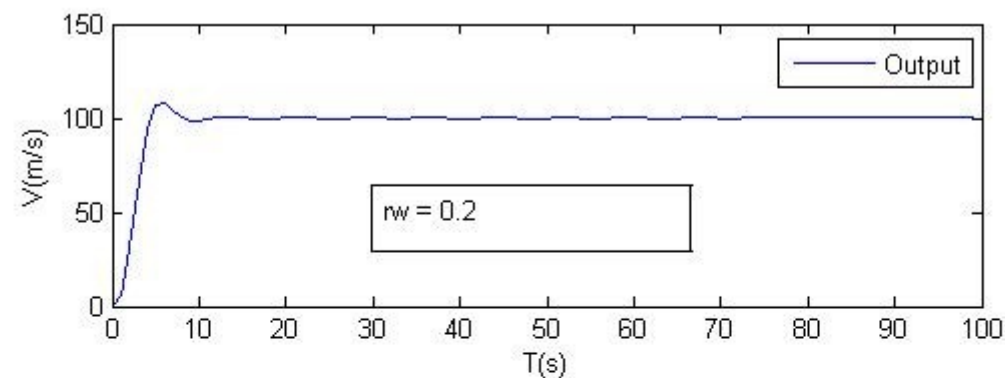
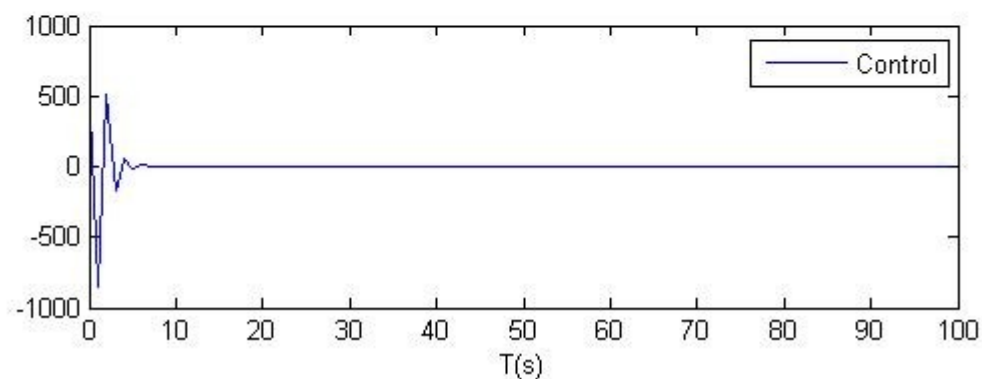
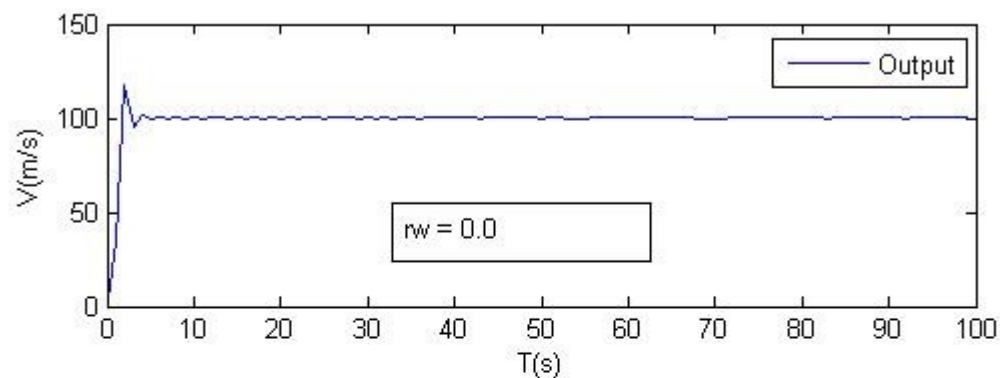
1.3.3 MATLAB Tutorial: Computation of MPC Gains

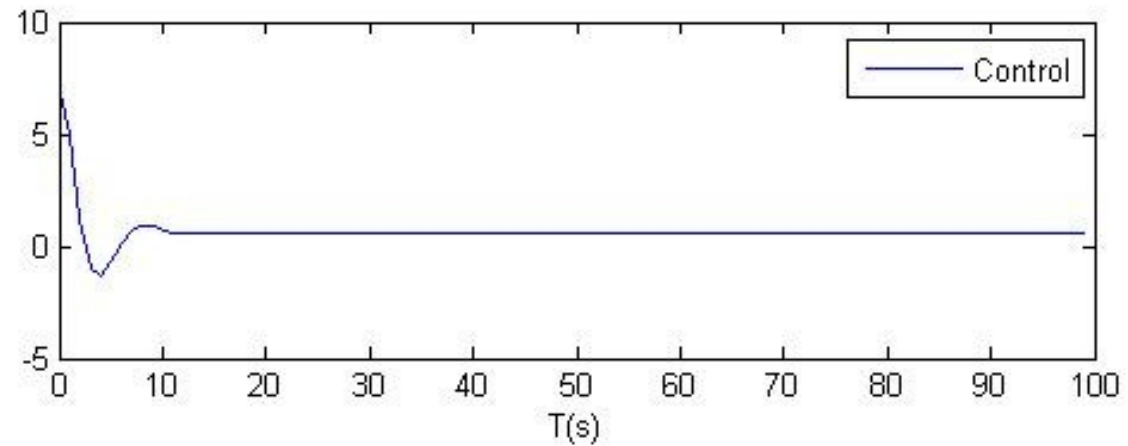
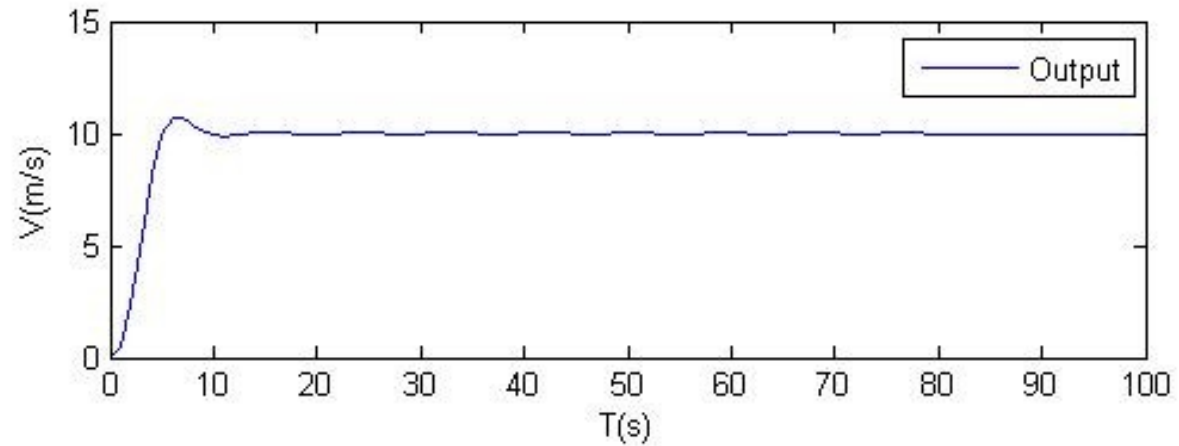
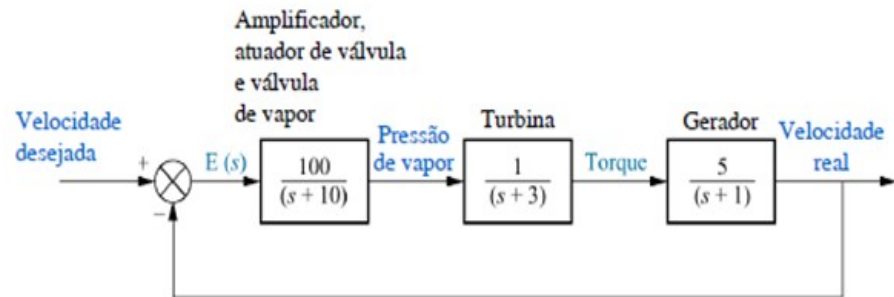
```
num = [500];
den = [1 14 43 30];
Plant = tf(num,den)
[Am, Bm, Cm, Dm] = tf2ss(num,den);
PlantSS = ss(Plant);
[Ap,Bp,Cp,Dp] =
c2dm(Am,Bm,Cm,Dm,0.1);
Nc = 4;
Np = 9;
[m1,n1]=size(Cp);
[n1,n_in]=size(Bp);
Ae=eye(n1+m1,n1+m1);
Ae(1:n1,1:n1)=Ap;
Ae(n1+1:n1+m1,1:n1)=Cp*Ap;
Be=zeros(n1+m1,n_in);
Be(1:n1,:)=Bp;
Be(n1+1:n1+m1,:)=Cp*Bp;
Ce=zeros(m1,n1+m1);
Ce(:,n1+1:n1+m1)=eye(m1,m1);
```

```
BarRs=ones(Np,1);
PhiPhi= Phi'*Phi;
PhiF= Phi'*F;
PhiR=Phi'*BarRs;
```

```
N_sim=100;
setpoint = 100;
r=setpoint*ones(N_sim,1);
```

```
DeltaU=inv(PhiPhi+0.2*eye(Nc,Nc))*(PhiR*r(kk)-PhiF*Xf);
Kmpc = [1 0 0 0]*inv(PhiPhi+0.2*eye(Nc,Nc))*(PhiF);
deltau=DeltaU(1,1);
u=u+deltau;
```



```

setpoint = 10;
r=setpoint*ones(N_sim,1);
u=0; % u(k-1) =0
y=0;
for kk=1:N_sim;
DeltaU=inv(PhiPhi+0.5*eye(Nc,Nc))*(PhiR*r(kk)-PhiF*Xf);

```

Lista de Exercícios:
Problemas 1.1 a 1.6
Utilize o Matlab para
auxiliar na demonstração
dos resultados.

Model Predictive Control System Design and Implementation Using MATLAB[®]

%-----

%Projeto MPC para Simulink

Ts = 0.1;

MPC_Turbina = mpc(Plant,Ts,9,4);

%-----

,

>> Plant

Plant =

500

$s^3 + 14s^2 + 43s + 30$

Continuous-time transfer function.

Command Window

New to MATLAB? See resources for [Getting Started](#).

>> help mpc

Model Predictive Control Toolbox
Version 6.0 (R2017b) 24-Jul-2017

General.

`mpcprops` - Help on model predictive controller properties.
`mpcverbosity` - Change the level of verbosity of `mpc` Toolbox.

Construction.

`mpc` - Create a model predictive controller.
`setmpcsignals` - Set input and output signal types in the predictive model.
`gpc2mpc` - Generate a SISO model predictive controller equivalent to a generalized predictive controller

Command Window

New to MATLAB? See resources for [Getting Started](#).

mpc Create an `mpc` controller.

`MPCOBJ = mpc(PLANT)` creates the `mpc` object based on the linear plant model `PLANT`, which must be a SS, TF, ZPK or a linear model from System Identification Toolbox such as `IDSS`, `IDTF`, `IDPROC`, `IDPOLY` and `IDGREY`.

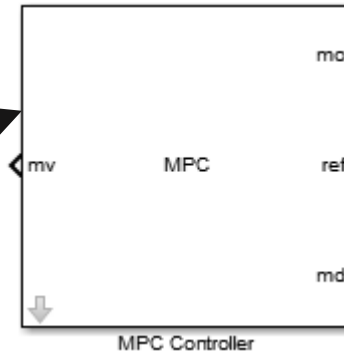
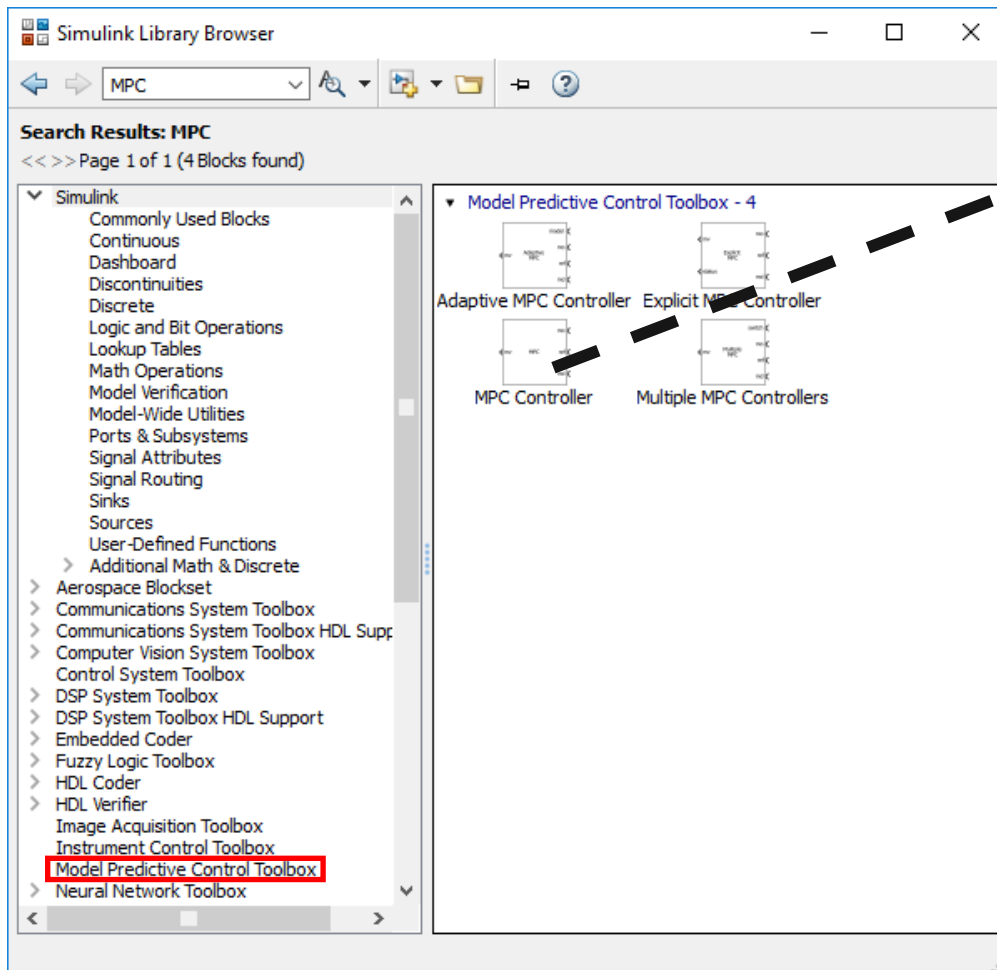
`MPCOBJ = mpc(PLANT,TS)` specifies a sampling time for the `mpc` controller. A continuous-time `PLANT` model is discretized with sampling time `TS`. A discrete-time `PLANT` model is resampled to sampling time `TS`.

`MPCOBJ = mpc(PLANT,TS,P)` specifies the prediction horizon `P`. `P` is a positive, finite integer.

`MPCOBJ = mpc(PLANT,TS,P,M)` specifies the control horizon, `M`. `M` is either an integer ($\leq P$) or a vector of blocked moves such that $\text{sum}(M) \leq P$.

`MPCOBJ = mpc(PLANT,TS,P,M,Weights,MV,OV,DV)` also allows you to specify `Weights`, constraints on `MV`, `MV` rates and plant outputs, scale factors, as well as other properties with structures `Weights`, `MV`, `OV` and `DV` in that order. Those properties can also be specified using either

MPC – Matlab/Simulink

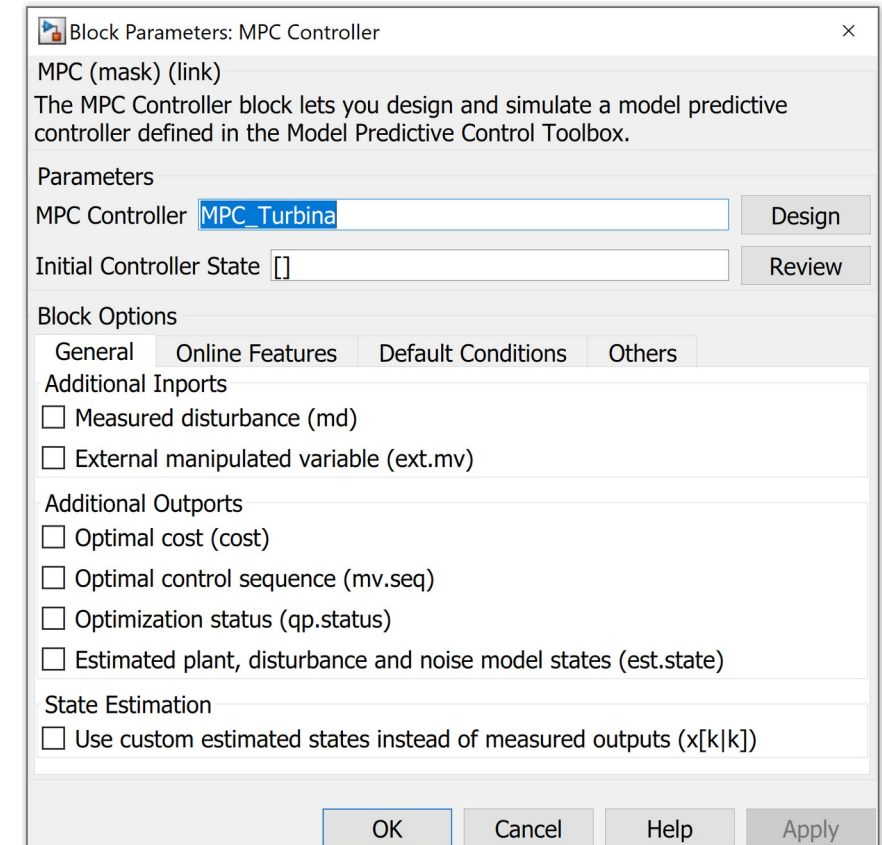
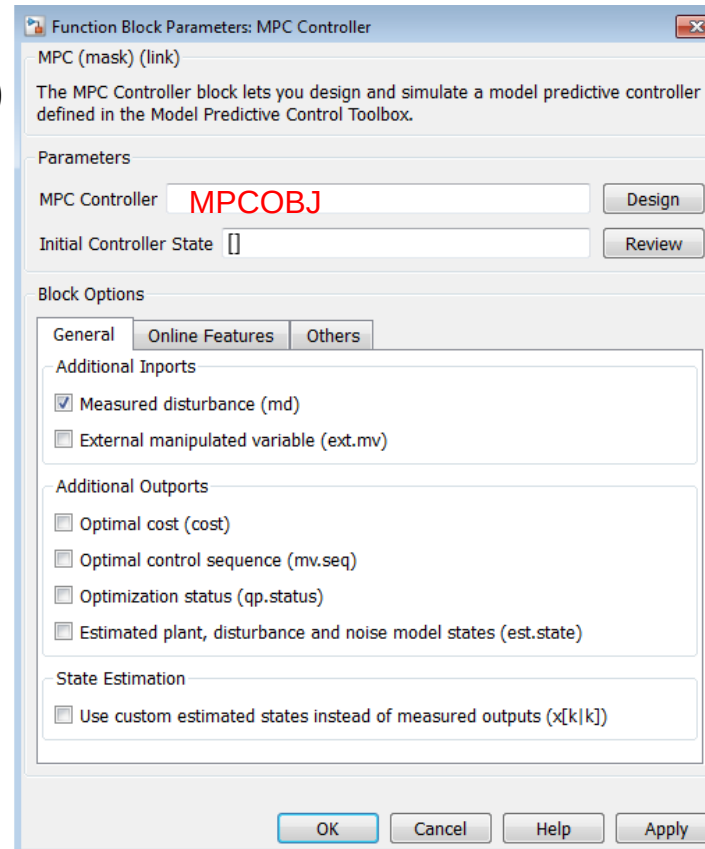


- measured output signal (mo);
- reference signal (ref);
- measured disturbance signal (md);
- manipulated variables (mv).

- Para utilizar o bloco MPC no Simulink é necessário configurar e compilar o MPC object no Matlab;
- A planta faz parte da configuração e compilação no MPC object

MPCOBJ = mpc (MODELS,TS,P,M, W)

- MODELS: Planta;
- TS: Tempo de Amostragem;
- P : Np
- M: Nc
- W: peso



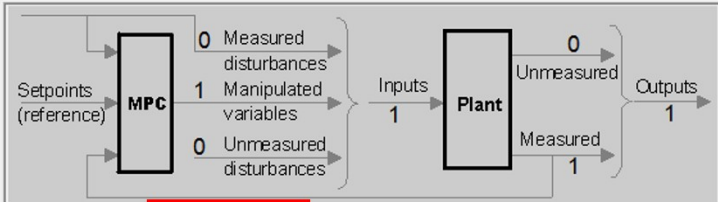
Control and Estimation Tools Manager

File MPC Help

Workspace

- Project - MPCSimulink1
 - Operating Points
 - Default Operating F
 - MPC Design Task - MP**
 - Plant models
 - Controllers
 - Scenarios

MPC structure overview



Import Plant ... Import Controller ... Help

Input signal properties

Name	Type	Description	Units	Nominal	ScaleFactor
MV1	Manipulated			0	1

Output signal properties

Name	Type	Description	Units	Nominal	ScaleFactor
MO1	Measured			0	1

Adding model "MPC1_Plant" to plant models list.
Controller "MPC1" added to task "MPC Design Task - MPC Controller".

View MPC Structure

MPC Structure



Plant Inputs

Signal Type	Size	Channel Indices
Manipulated Variables (MV)	1	1

Plant Outputs

Signal Type	Size	Channel Indices
Measured Outputs (MO)	1	1

OK Help

Import Plant Model

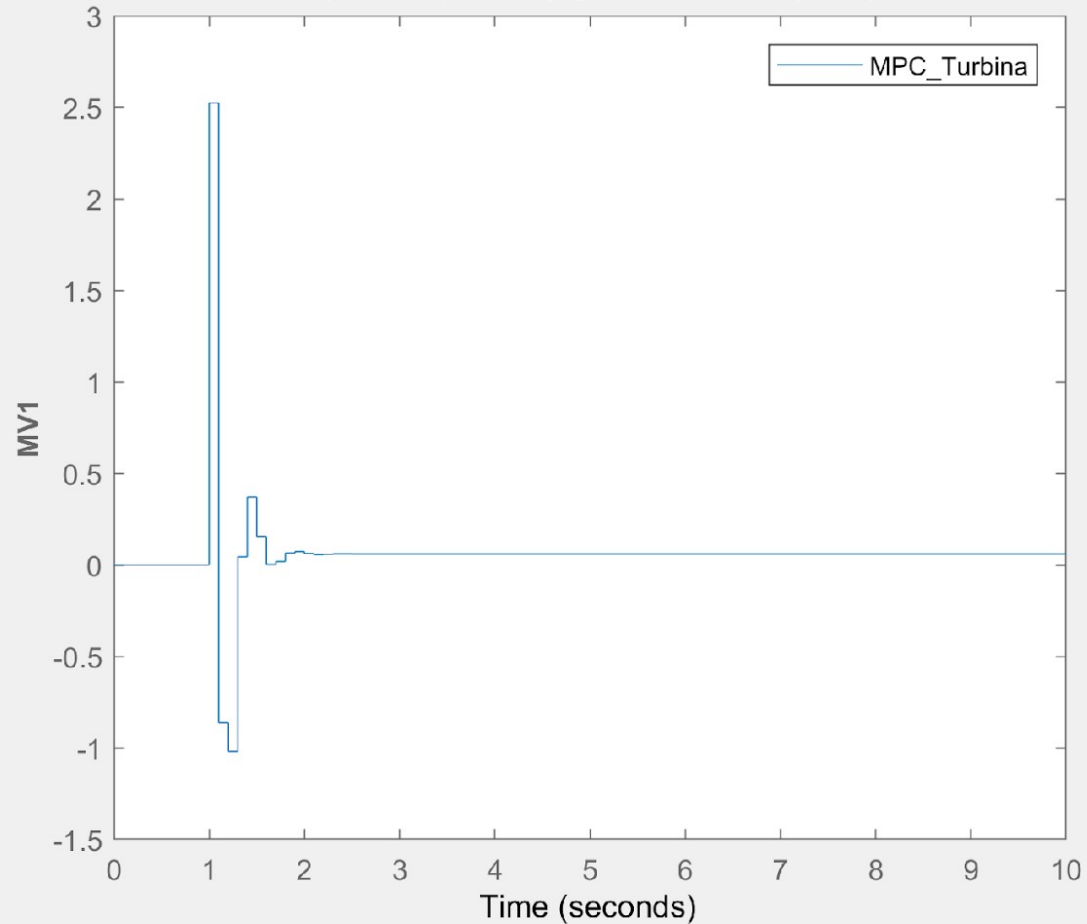
Import a plant model from MATLAB Workspace:

Select	Name	Type	Order	Inputs	Outputs
☐	Plant	tf	3	1	1
☐	PlantSS	ss	3	1	1

Refresh workspace Import Cancel Help

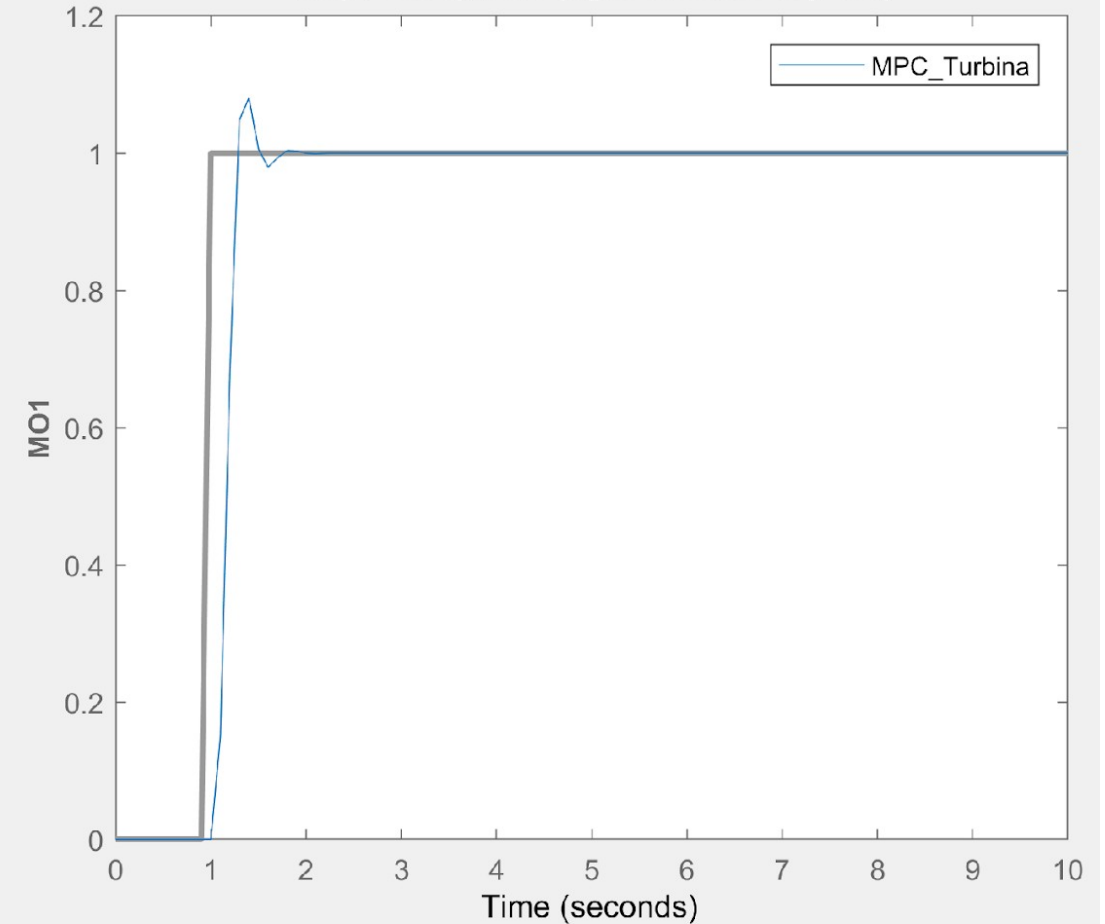
scenario1: Input

Input Response (against internal plant)



scenario1: Output

Output Response (against internal plant)



Simulink Library Browser

space

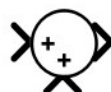
Search Results: space

<<>> Page 1 of 1 (90 Blocks found)

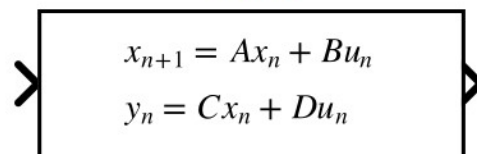
Simulink

- Commonly Used Blocks
- Continuous
- Dashboard
- Discontinuities
- Discrete
- Logic and Bit Operations
- Lookup Tables
- Math Operations
- Model Verification
- Model-Wide Utilities
- Ports & Subsystems
- Signal Attributes
- Signal Routing

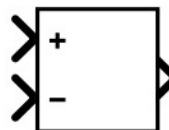
Simulink - 12 (simulink/Continuous/State-Space)



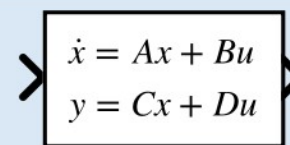
Sum



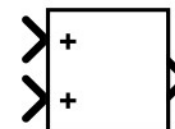
Discrete State-Space



Subtract



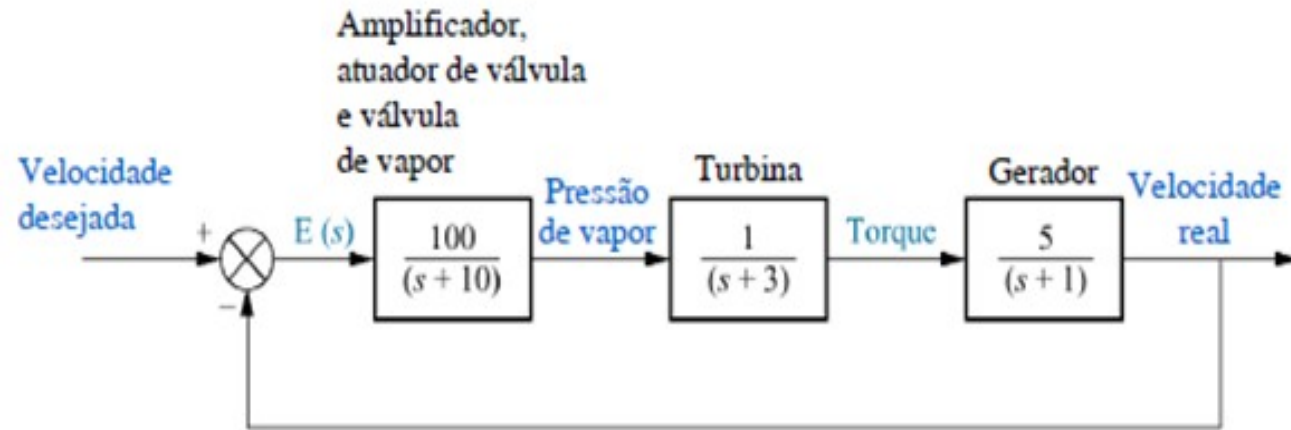
State-Space



Add



Sum



>> Am

Am =

-14	-43	-30
1	0	0
0	1	0

>> Bm

Bm =

1
0
0

>> Cm

Cm =

0	0	500
---	---	-----

Dm =

0

Block Parameters: State-Space

State Space

State-space model:
 $\frac{dx}{dt} = Ax + Bu$
 $y = Cx + Du$

Parameters

A:
[-14 -43 -30;1 0 0;0 1 0]

B:
[1;0;0]

C:
[0 0 500]

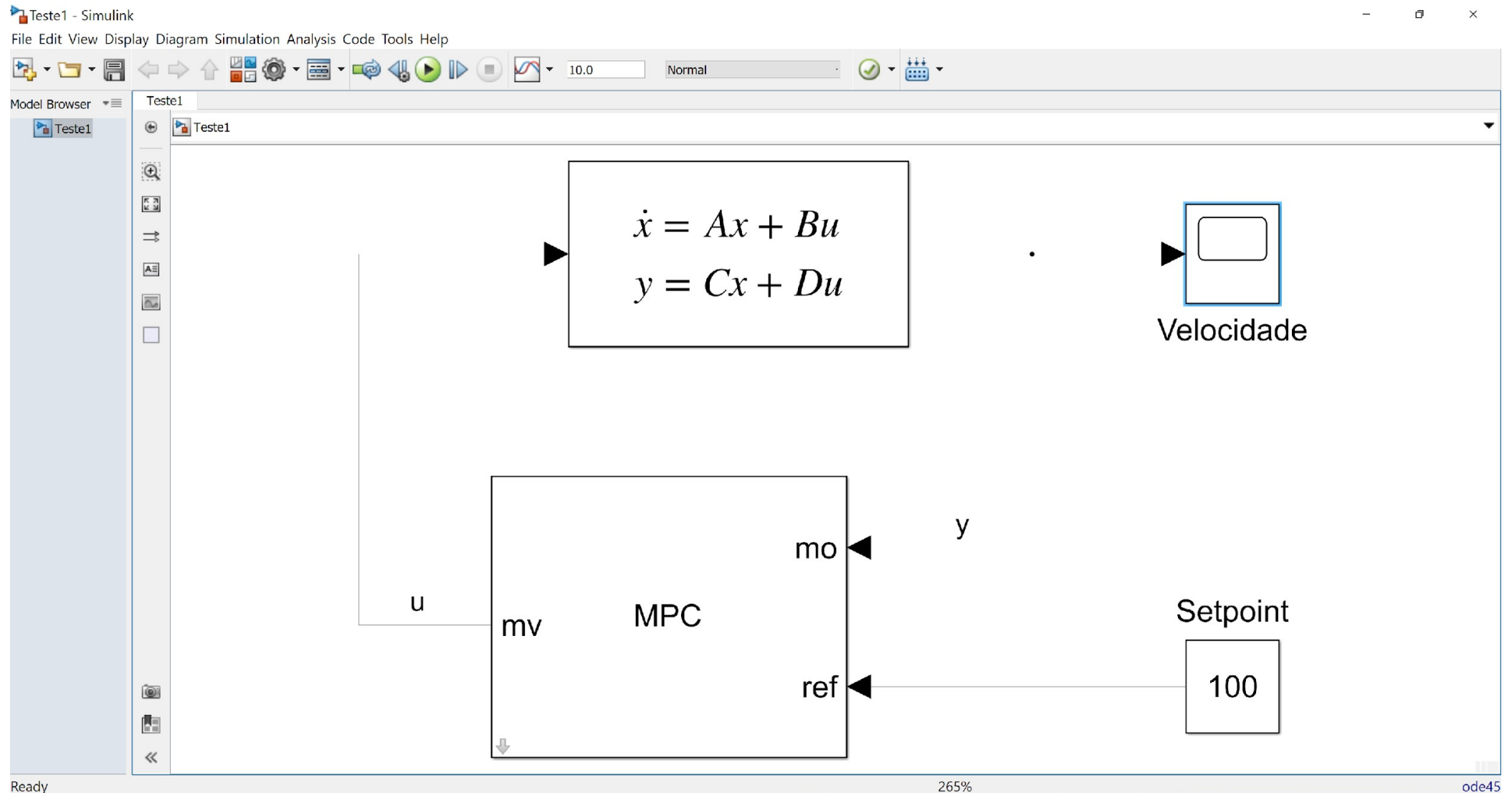
D:
[0]

Initial conditions:
0

Absolute tolerance:
auto

State Name: (e.g., 'position')
"

? OK Cancel Help Apply



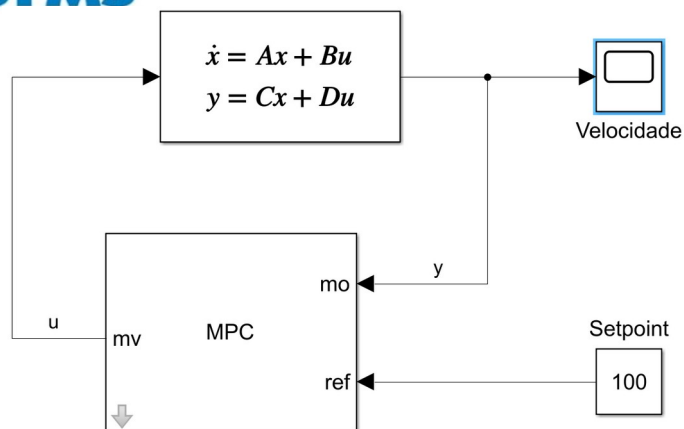
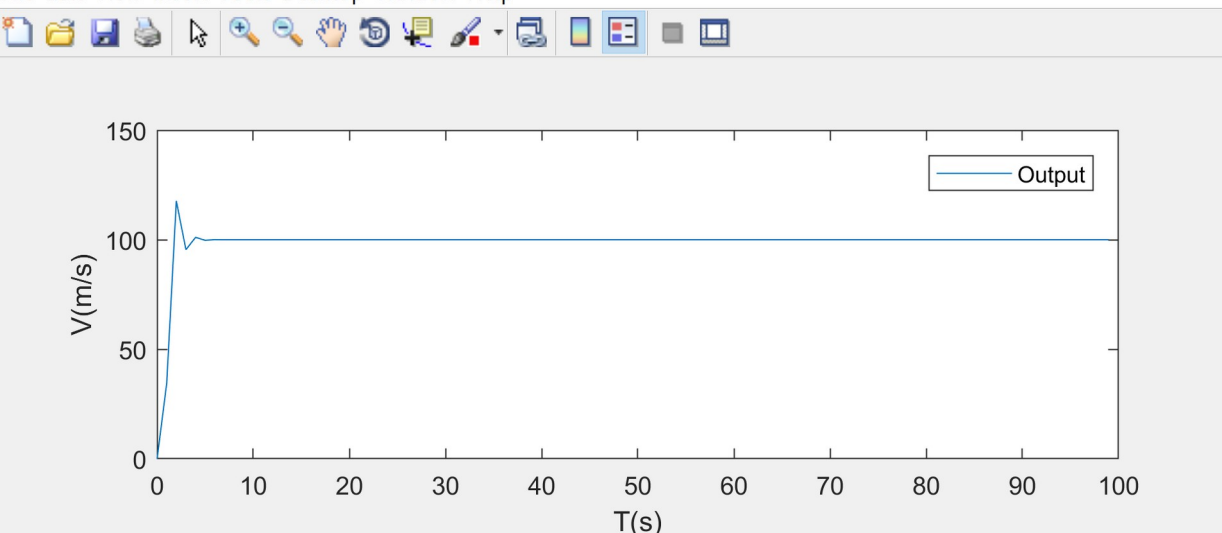


Figure 2

File Edit View Insert Tools Desktop Window Help



```

55 %* (Pn1k);
56 vetorHum = [1;1;1;1;1;1;1;1;1];
57 Ky = [1 0 0 0]*inv(PhiPhi+0.0*eye(Nc,Nc))*PhiR;
58 %*(Phi'*(vetorHum))
59 deltau=DeltaU(1,1);
60 u=u+deltau;
61 u1(kk)=u;
62 y1(kk)=y;
63 xm_old=xm;
64 xm=Ap*xm+Bp*u;
65 y=Cp*xm;
66 Xf=[xm-xm_old;y];
67 end
68 k=0:(N_sim-1);
69 figure
70 subplot(211)
71 plot(k,y1)
72 xlabel('T(s)')
73 ylabel('V(m/s)')
74 legend('Output')
75 subplot(212)
76 plot(k,u1)
77 xlabel('T(s)')
78 legend('Control')
79 %-----

```

Repostas

$$\Phi^T \Phi = \begin{bmatrix} 1.1541 & 1.0407 & 0.9116 & 0.7726 \\ 1.0407 & 0.9549 & 0.8475 & 0.7259 \\ 0.9116 & 0.8475 & 0.7675 & 0.6674 \\ 0.7726 & 0.7259 & 0.6674 & 0.5943 \end{bmatrix}$$

$$\Phi^T F = \begin{bmatrix} 9.2325 & 3.2147 \\ 8.3259 & 2.7684 \\ 7.2927 & 2.3355 \\ 6.1811 & 1.9194 \end{bmatrix}; \Phi^T \bar{R}_s = \begin{bmatrix} 3.2147 \\ 2.7684 \\ 2.3355 \\ 1.9194 \end{bmatrix}.$$

$$\Delta U = (\Phi^T \Phi)^{-1} (\Phi^T \bar{R}_s - \Phi^T F x(k_i)) = [7.2 \ -6.4 \ 0 \ 0]^T.$$

rst	0	Signal In
Np	-214...	Signal In
A	{0.8 ...	Signal Inter...
B	{0.1}...	Signal Inter...
C	{0 1}	Signal Inter...
F	{0.8 ...	Signal Inter...
Phi	{0.1 ...	Signal Inter...
Phi_T	{0.1 ...	Signal Inter...
Phi_Phi	{1.15...	Signal Inter...

Processes (Active)

Name	Type (filtered)	State

```

306 u10 := inFi_Fi*FiRs_Fxi;
307
308
309 yend := F0*xi + Fi*u10;
310
311 end if;
312 A <= Ae1;
313 B <= Be1;
314 C <= Ce1;
315 F <= F0;
316 Phi <= Fi;
317 R <= Rbarra;
318 Rs <= rsbarra;
319 Phi_T <= Fi_T;
320 Phi_Phi <= Fi_Fi;
321 Phi_F <= Fi_F;
322 Phi_Rs <= Fi_Rs;
323 Phi_Fxi <= Fi_Fxi;
324 PhiRs_Fxi <= FiRs_Fxi;
325 PhiFiR <= PHR;
326 inPhi <= inFi_Fi;
327
328 DeltaU <= u10;
329
330 Yout <= yend;
331 --X1 <= xi;
332

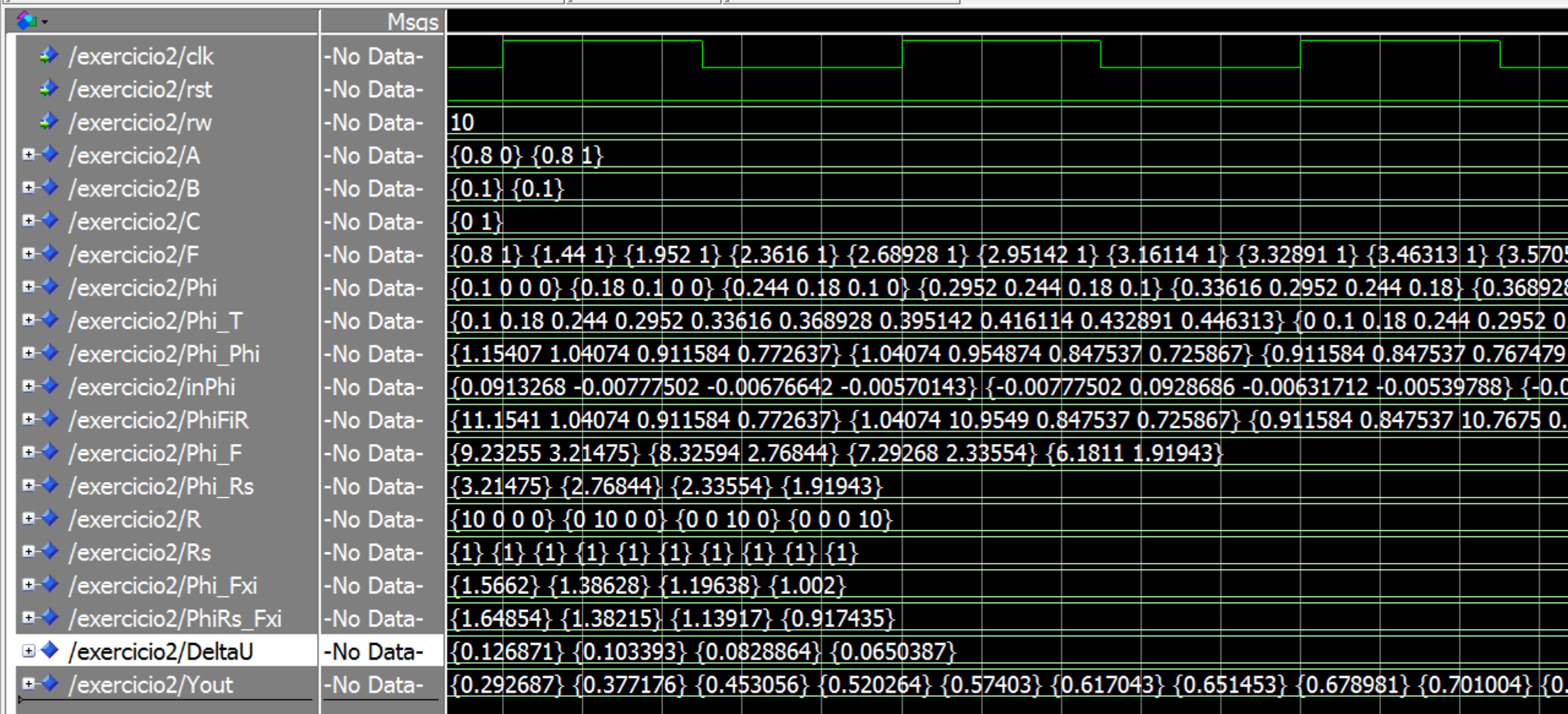
```

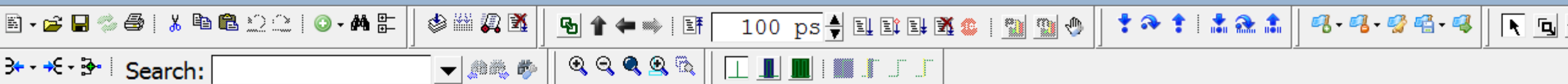
/exercicio2/line_73/u10

{0.126871} {0.103393} {0.0828864} {0.0650387}

100 ps

Search:





	Msas	
/exercicio2/clock	1	
/exercicio2/rst	0	
/exercicio2/rw	0	0
/exercicio2/A	{0.8 0} ...	{0.8 0} {0.8 1}
/exercicio2/B	{0.1} {0.1}	{0.1} {0.1}
/exercicio2/C	{0 1}	{0 1}
/exercicio2/F	{0.8 1} ...	{0.8 1} {1.44 1} {1.952 1} {2.3616 1} {2.68928 1} {2.95142 1} {3.16114 1} {3.32891 1} {3.46313 1} {3.5705}
/exercicio2/Phi	{0.1 0 0...}	{0.1 0 0 0} {0.18 0.1 0 0} {0.244 0.18 0.1 0} {0.2952 0.244 0.18 0.1} {0.33616 0.2952 0.244 0.18} {0.368928}
/exercicio2/Phi_T	{0.1 0.1...}	{0.1 0.18 0.244 0.2952 0.33616 0.368928 0.395142 0.416114 0.432891 0.446313} {0 0.1 0.18 0.244 0.2952 0.3}
/exercicio2/Phi_Phi	{1.1540...}	{1.15407 1.04074 0.911584 0.772637} {1.04074 0.954874 0.847537 0.725867} {0.911584 0.847537 0.767479 0}
/exercicio2/inPhi	{100 -1...}	{100 -180 80 3.40329e-011} {-180 424 -324 80} {80 -324 443.523 -206.35} {8.1841e-011 80 -206.35 135.699}
/exercicio2/PhiFiR	{1.1540...}	{1.15407 1.04074 0.911584 0.772637} {1.04074 0.954874 0.847537 0.725867} {0.911584 0.847537 0.767479 0}
/exercicio2/Phi_F	{9.2325...}	{9.23255 3.21475} {8.32594 2.76844} {7.29268 2.33554} {6.1811 1.91943}
/exercicio2/Phi_Rs	{3.2147...}	{3.21475} {2.76844} {2.33554} {1.91943}
/exercicio2/R	{0 0 0 0...}	{0 0 0 0} {0 0 0 0} {0 0 0 0} {0 0 0 0}
/exercicio2/Rs	{1} {1} ...	{1} {1} {1} {1} {1} {1} {1} {1} {1} {1}
/exercicio2/Phi_Fxi	{1.5662...}	{1.5662} {1.38628} {1.19638} {1.002}
/exercicio2/PhiRs_Fxi	{1.6485...}	{1.64854} {1.38215} {1.13917} {0.917435}
/exercicio2/DeltaU	{7.2} {-...}	{7.2} {-6.4} {1.48162e-010} {1.47793e-011}
/exercicio2/Yout	{1} {1} ...	{1} {1} {1} {1} {1} {1} {1} {1} {1} {1}

Projeto de um Controlador Peditivo (MPC) - Cosimulação

Dada a planta (equação abaixo), projete um controlador preditivo, para um setpoint = 10, $N_c = 3$, $N_p = 5$.

Planta

$$\frac{(s+1)}{s(s+2)}$$

- Projeto MPC – Matlab/Simulink;

Simulink Library Browser

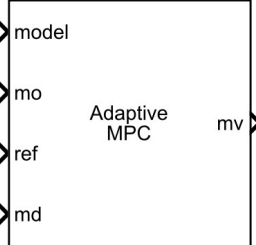
Search Results: MPC
<<>> Page 1 of 1 (5 Blocks found)

Simulink

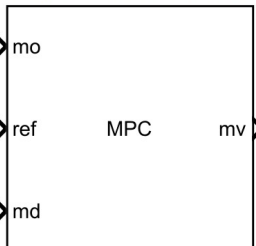
- Commonly Used Blocks
- Continuous
- Dashboard
- Discontinuities
- Discrete
- Logic and Bit Operations
- Lookup Tables
- Math Operations
- Model Verification
- Model-Wide Utilities
- Ports & Subsystems
- Signal Attributes
- Signal Routing
- Sinks
- Sources
- User-Defined Functions
- Additional Math & Discrete
- Aerospace Blockset
- Audio System Toolbox
- Automated Driving System Toolbox
- Communications System Toolbox
- Communications System Toolbox HDL Support
- Computer Vision System Toolbox
- Control System Toolbox
- Data Acquisition Toolbox
- DSP System Toolbox
- DSP System Toolbox HDL Support
- Embedded Coder
- Fuzzy Logic Toolbox
- HDL Coder
- HDL Verifier
- Image Acquisition Toolbox

Model Predictive Control Toolbox - 5

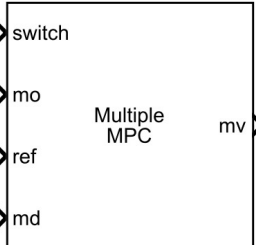
Adaptive MPC Controller



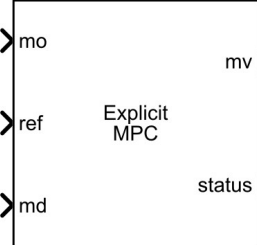
MPC Controller



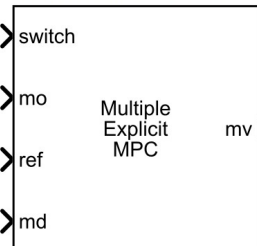
Multiple MPC Controllers



Explicit MPC Controller



Multiple Explicit MPC Controllers

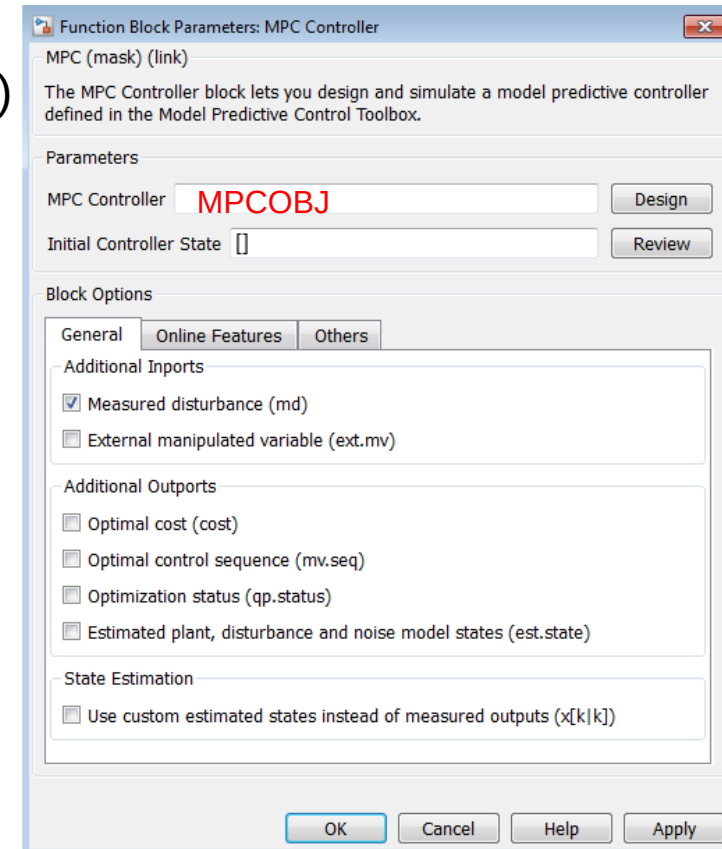


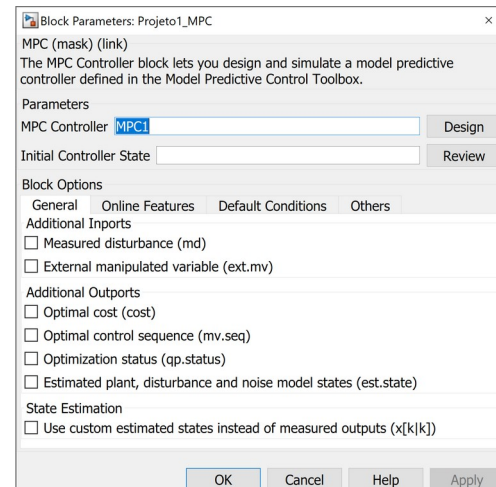
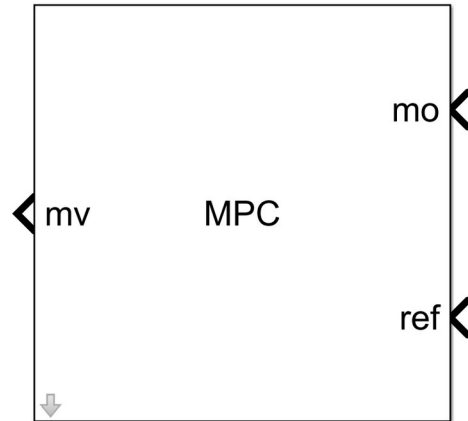
Configuração do MPC object

- $MPCobj = mpc(Plant)$
- $MPCobj = mpc(Plant, Ts)$
- $MPCobj = mpc(Plant, Ts, p, m, W, MV, OV, DV)$
- $MPCobj = mpc(Models, Ts, p, m, W, MV, OV, DV)$

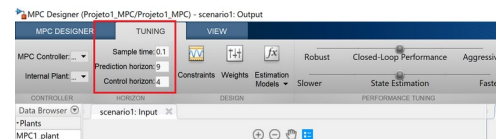
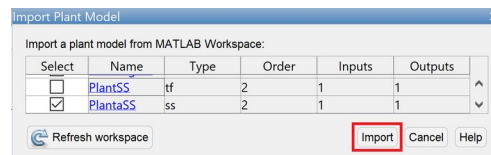
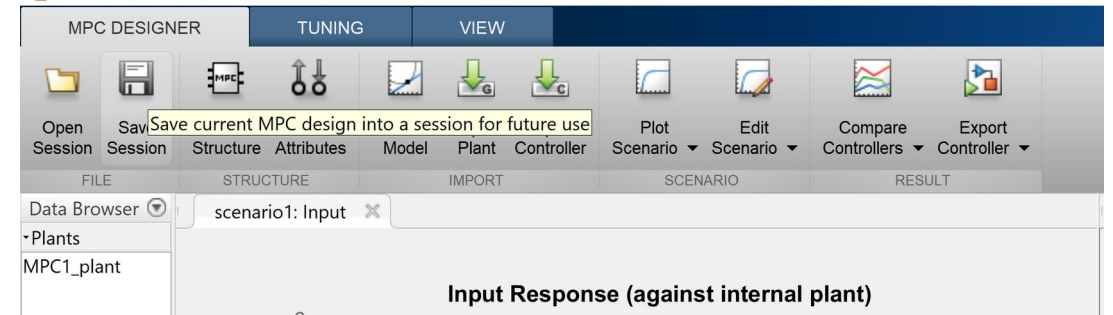
$MPCOBJ = mpc (MODELS, TS, P, M, W)$

- MODELS: Planta;
- TS: Tempo de Amostragem;
- P : N_p
- M: N_c
- W: peso

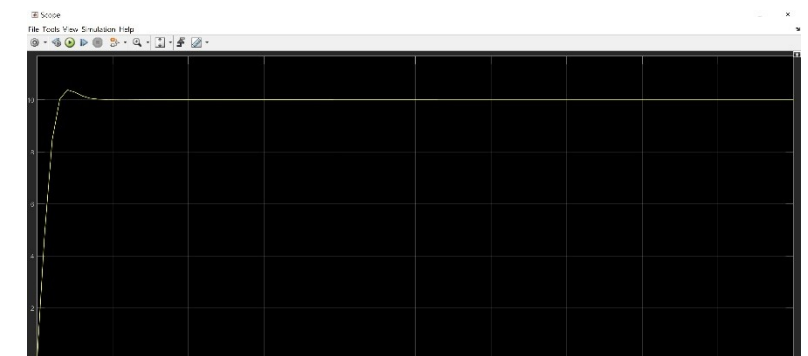
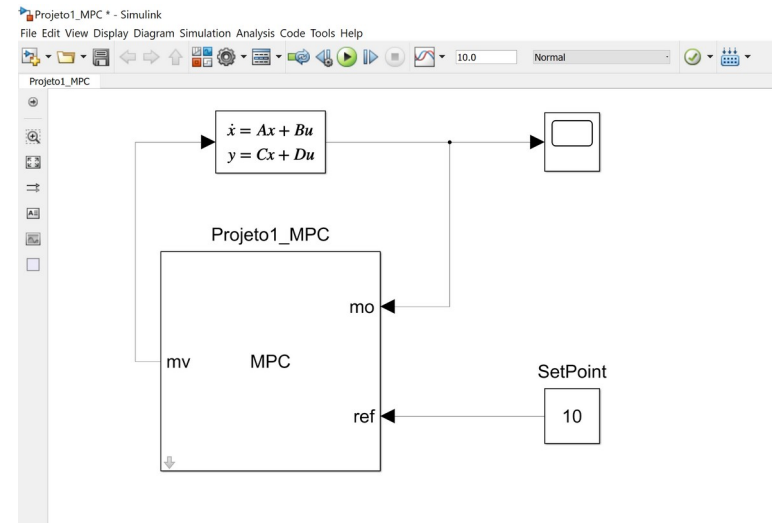
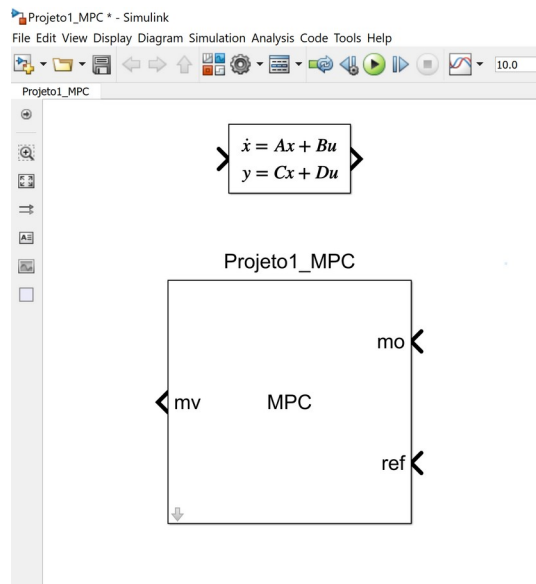
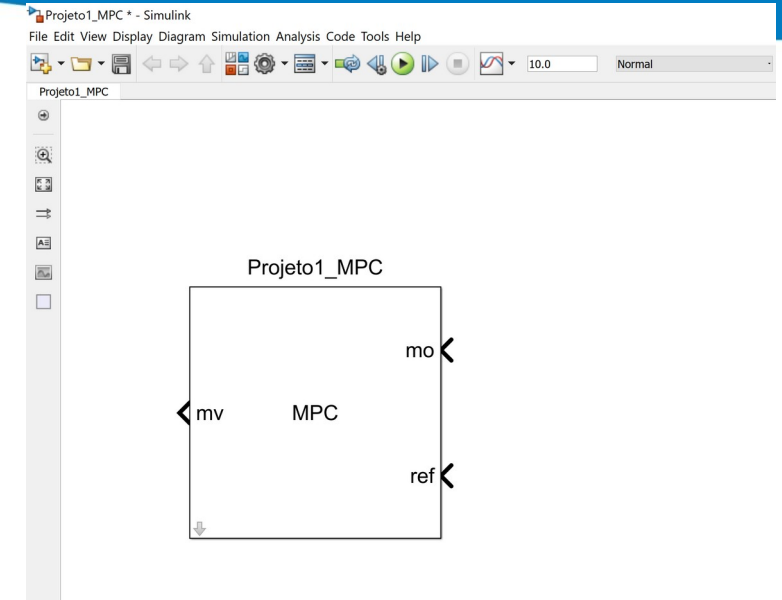
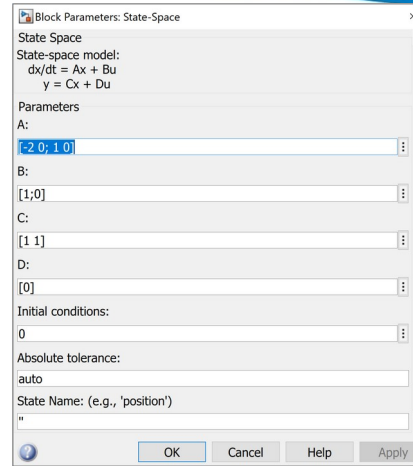


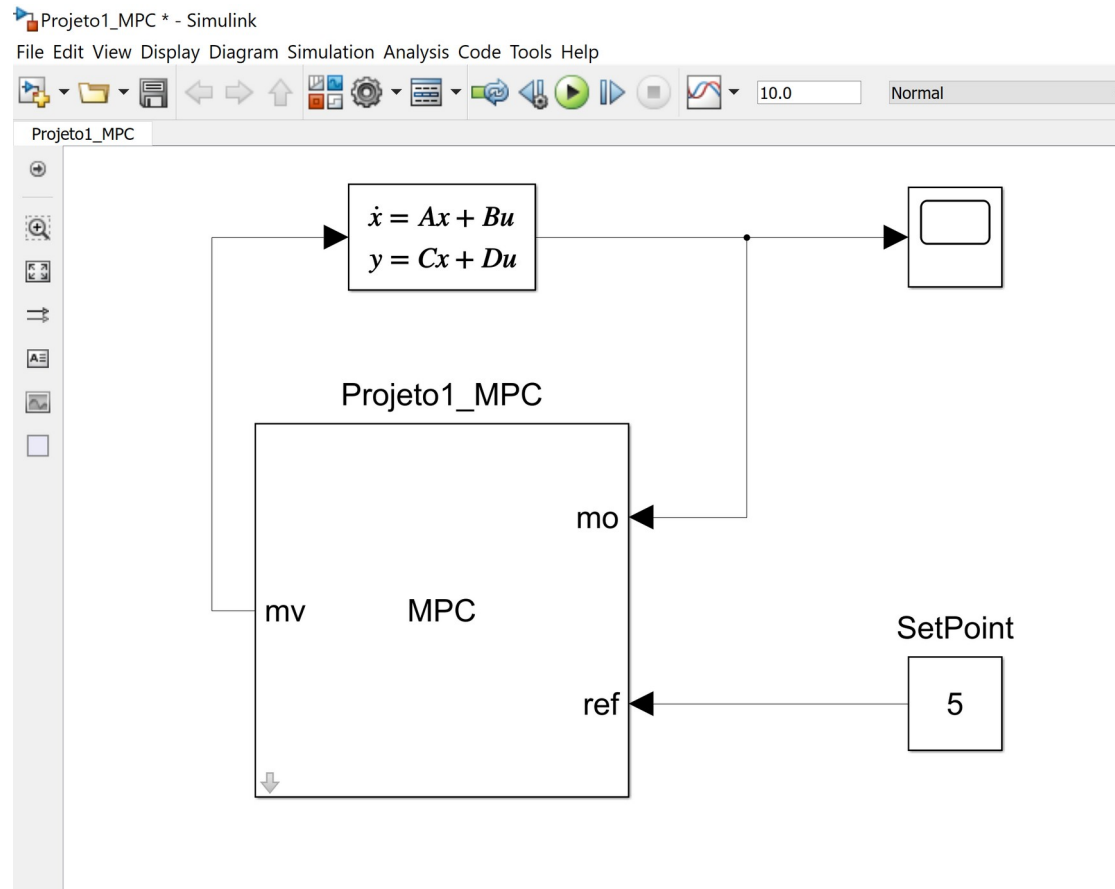


MPC Designer (Projeto1_MPC/Projeto1_MPC) - scenario1: Output



$$\frac{s + 1}{s^2 + 2s}$$





where,

$$x = \begin{bmatrix} C'_A \\ T' \end{bmatrix}, u = \begin{bmatrix} T'_c \\ C'_{Ai} \end{bmatrix}, y = \begin{bmatrix} T' \\ C'_A \end{bmatrix}$$

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}, C = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, D = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

TRABALHO 02 – PROJETO UM CONTROLADOR MPC PARA CSRT – MATLAB/SIMULINK

CSTR Model

The following differential equations represent the linearized model of a continuous stirred-tank reactor (CSTR) involving an exothermic reaction:

$$\frac{dC'_A}{dt} = a_{11}C'_A + a_{12}T' + b_{11}T'_c + b_{12}C'_{Ai}$$

$$\frac{dT'}{dt} = a_{21}C'_A + a_{22}T' + b_{21}T'_c + b_{22}C'_{Ai}$$

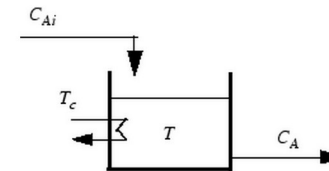
where the inputs are:

- C_{Ai} — Concentration of reagent A in the feed stream (kgmol/m³)
- T_c — Reactor coolant temperature (degrees C)

and the outputs are:

- T — Reactor temperature (degrees C)
- C_A — Residual concentration of reagent A in the product stream (kgmol/m³)

The prime terms, such as C'_A , denote a deviation from the nominal steady-state condition at which the model has been linearized.



ENVIAR A PASTA DO PROJETO VIA EMAIL: título do email projetoCRST
Data de entrega: 21/10/2020.

Create a state-space model of a CSTR system.

```
A = [-0.0285 -0.0014; -0.0371 -0.1476];
B = [-0.0850 0.0238; 0.0802 0.4462];
C = [0 1; 1 0];
D = zeros(2,2);
CSTR = ss(A,B,C,D);
```

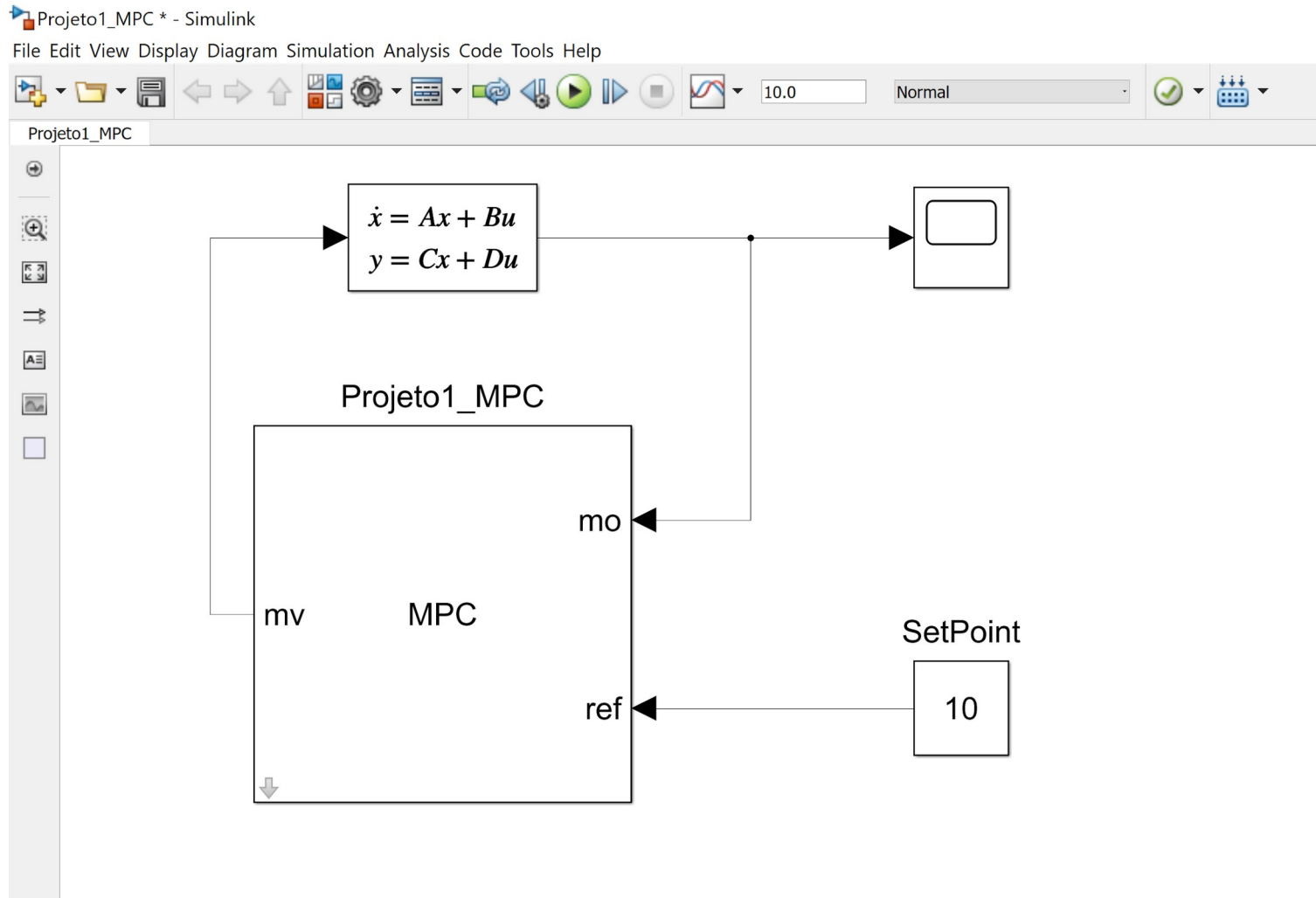
Measurement of reagent concentrations is often difficult. For this example, assume that:

- T_c is a manipulated variable.
- C_{Ai} is an unmeasured disturbance.
- T is a measured output.
- C_A is an unmeasured output.

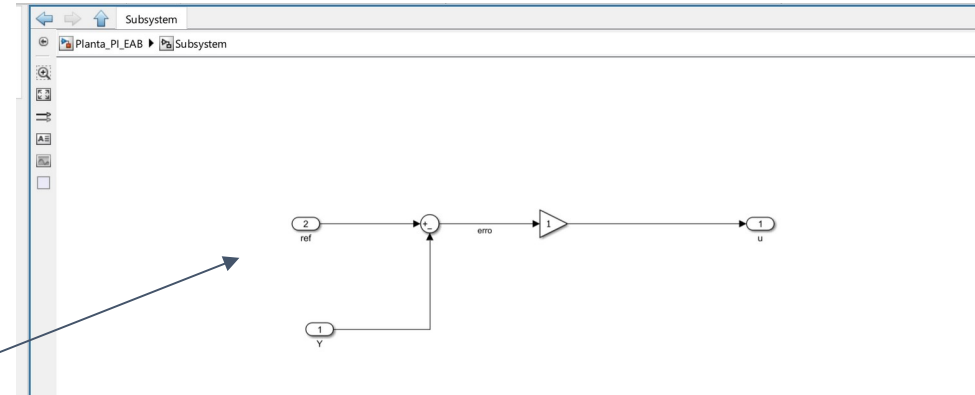
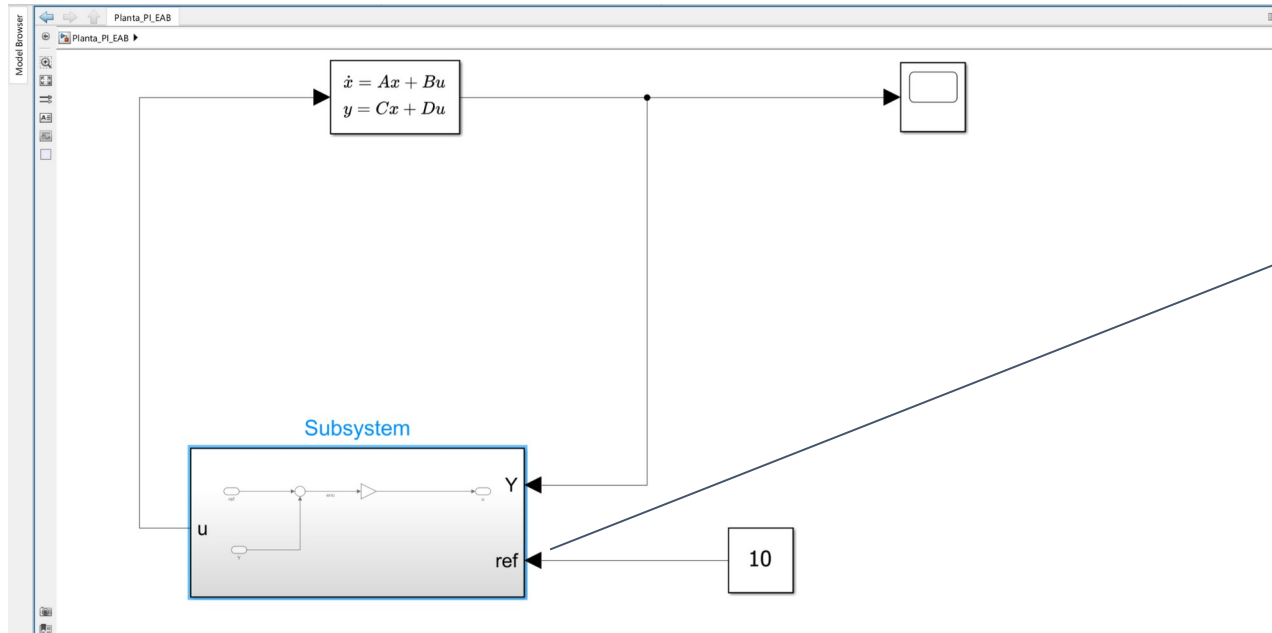
The model can be described in state-space format:

$$\frac{dx}{dt} = Ax + Bu$$

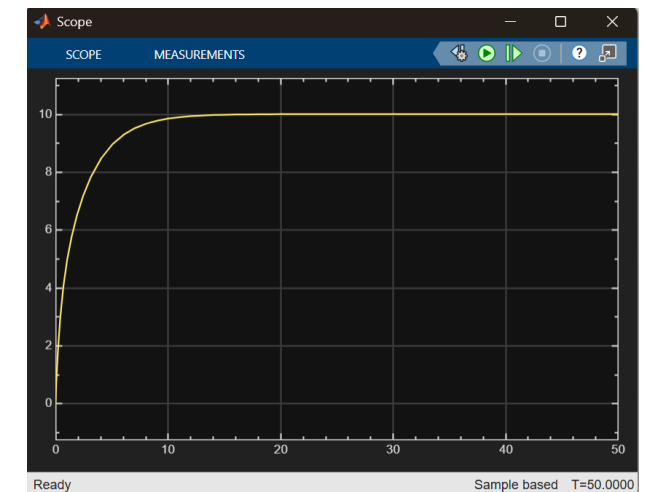
$$y = Cx + Du$$

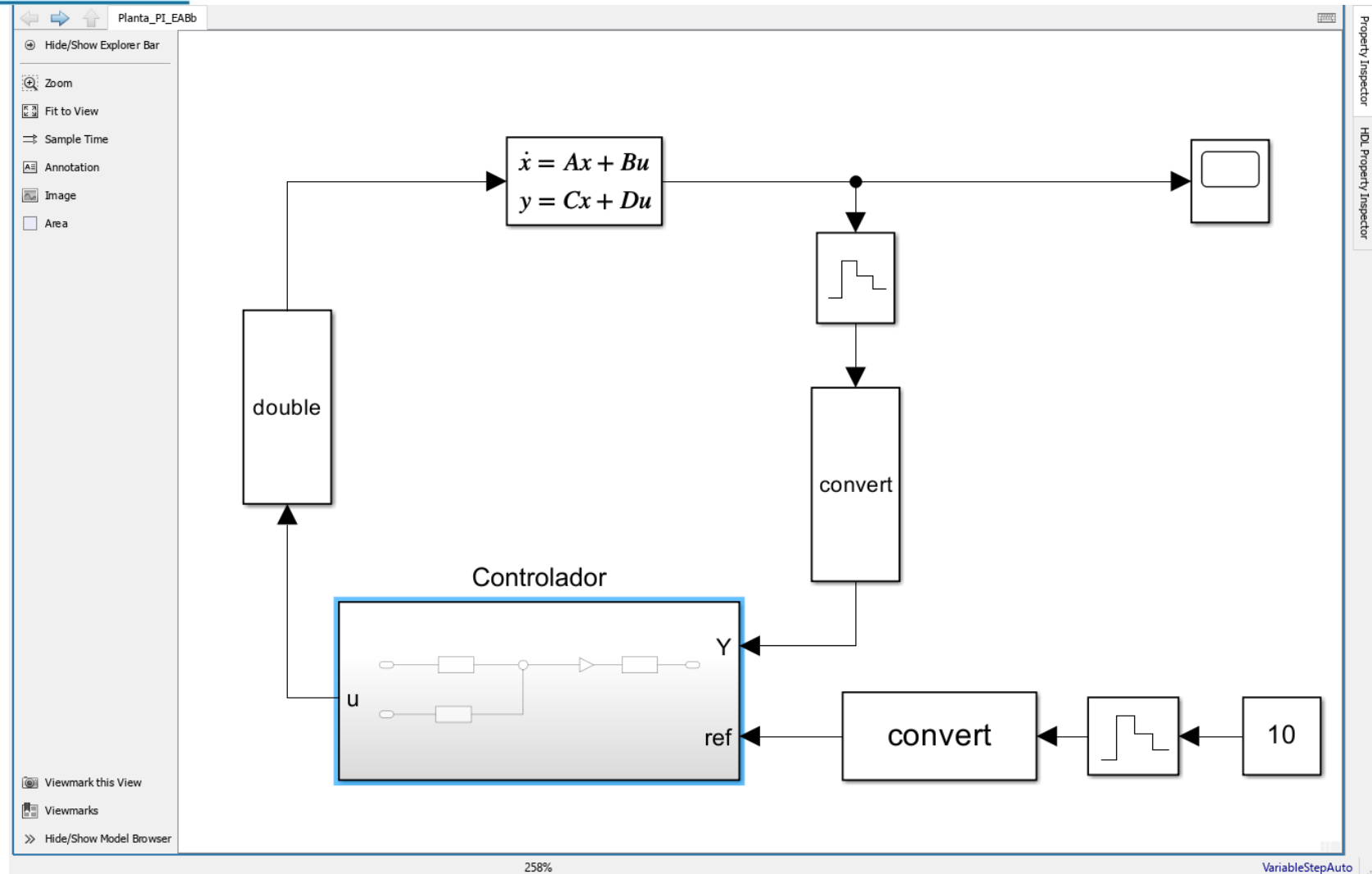


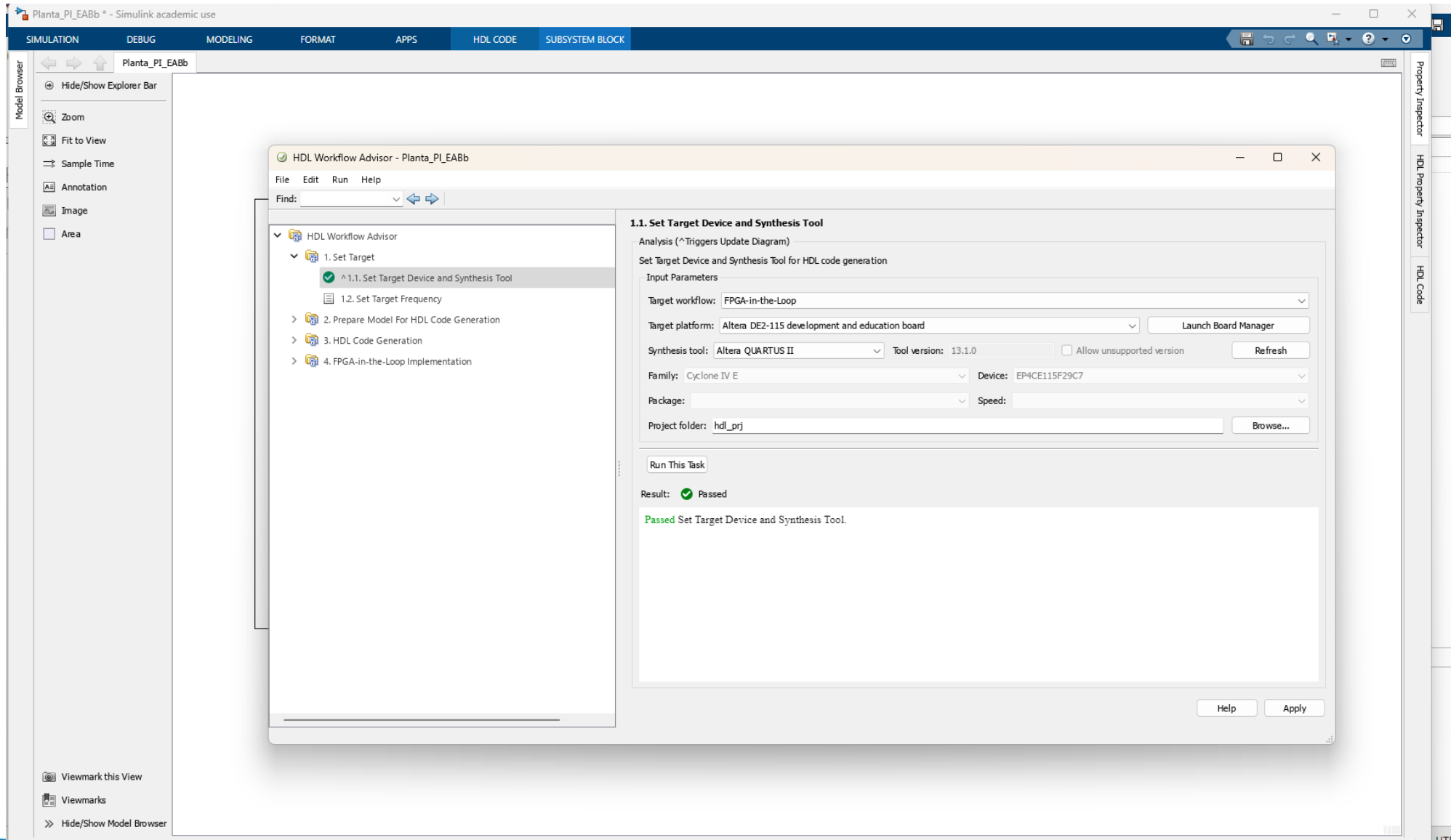
Substituir o bloco MPC por um Subsystem



$$u(k) = K_p [r(k) - y(k)]$$







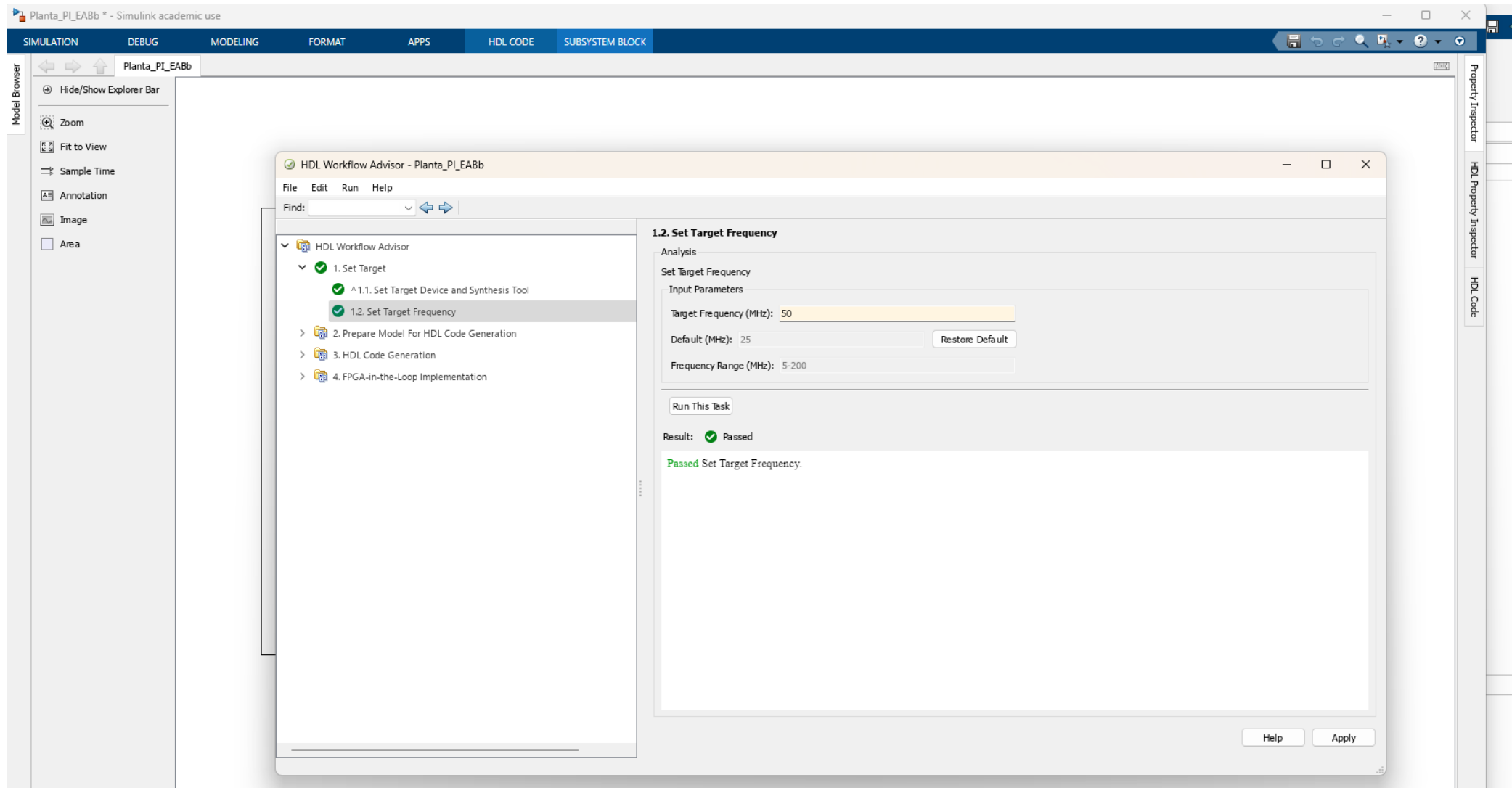
The screenshot shows the HDL Workflow Advisor window for the project 'Planta_PI_EABb'. The window is titled 'HDL Workflow Advisor - Planta_PI_EABb' and has a menu bar with 'File', 'Edit', 'Run', and 'Help'. The left pane shows a tree view of the workflow steps:

- HDL Workflow Advisor
 - 1. Set Target
 - 1.1. Set Target Device and Synthesis Tool (selected)
 - 1.2. Set Target Frequency
 - 2. Prepare Model For HDL Code Generation
 - 3. HDL Code Generation
 - 4. FPGA-in-the-Loop Implementation

The right pane displays the configuration for '1.1. Set Target Device and Synthesis Tool'. It includes a section for 'Input Parameters' with the following settings:

- Target workflow: FPGA-in-the-Loop
- Target platform: Altera DE2-115 development and education board
- Synthesis tool: Altera QUARTUS II
- Tool version: 13.1.0
- Family: Cyclone IV E
- Device: EP4CE115F29C7
- Package: (empty)
- Speed: (empty)
- Project folder: hdl_prj

Below the input parameters, there is a 'Run This Task' button and a 'Result' section showing 'Passed'. A message box at the bottom states 'Passed Set Target Device and Synthesis Tool.'.



Planta_PI_EABb * - Simulink academic use

SIMULATION DEBUG MODELING FORMAT APPS HDL CODE SUBSYSTEM BLOCK

Model Browser

- Hide/Show Explorer Bar
- Zoom
- Fit to View
- Sample Time
- Annotation
- Image
- Area

HDL Workflow Advisor - Planta_PI_EABb

File Edit Run Help

Find: [] [] []

HDL Workflow Advisor

- 1. Set Target
 - 1.1. Set Target Device and Synthesis Tool
 - 1.2. Set Target Frequency
- 2. Prepare Model For HDL Code Generation
- 3. HDL Code Generation
- 4. FPGA-in-the-Loop Implementation

1.2. Set Target Frequency

Analysis

Set Target Frequency

Input Parameters

Target Frequency (MHz): 50

Default (MHz): 25 [Restore Default]

Frequency Range (MHz): 5-200

[Run This Task]

Result: ✔ Passed

Passed Set Target Frequency.

[Help] [Apply]

Planta_PI_EABb * - Simulink academic use

SIMULATION DEBUG MODELING FORMAT APPS HDL CODE SUBSYSTEM BLOCK

Model Browser

- Hide/Show Explorer Bar
- Zoom
- Fit to View
- Sample Time
- Annotation
- Image
- Area

HDL Workflow Advisor - Planta_PI_EABb

File Edit Run Help

Find:

- ▼ HDL Workflow Advisor
 - ✓ 1. Set Target
 - ✓ ^ 1.1. Set Target Device and Synthesis Tool
 - ✓ 1.2. Set Target Frequency
 - ▼ 2. Prepare Model For HDL Code Generation
 - ⚠ 2.1. Check Model Settings
 - ✓ ^ 2.2. Check FPGA-in-the-Loop Compatibility
 - > 3. HDL Code Generation
 - > 4. FPGA-in-the-Loop Implementation

2.1. Check Model Settings

Analysis

Check basic model-level settings for HDL code generation. To run additional HDL code generation compatibility checks, click the button below to launch the HDL Code Advisor (opens in a separate window).

Input Parameters

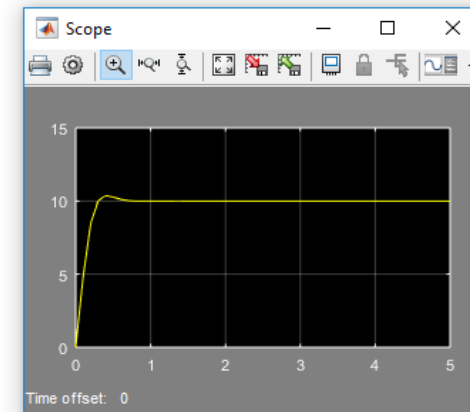
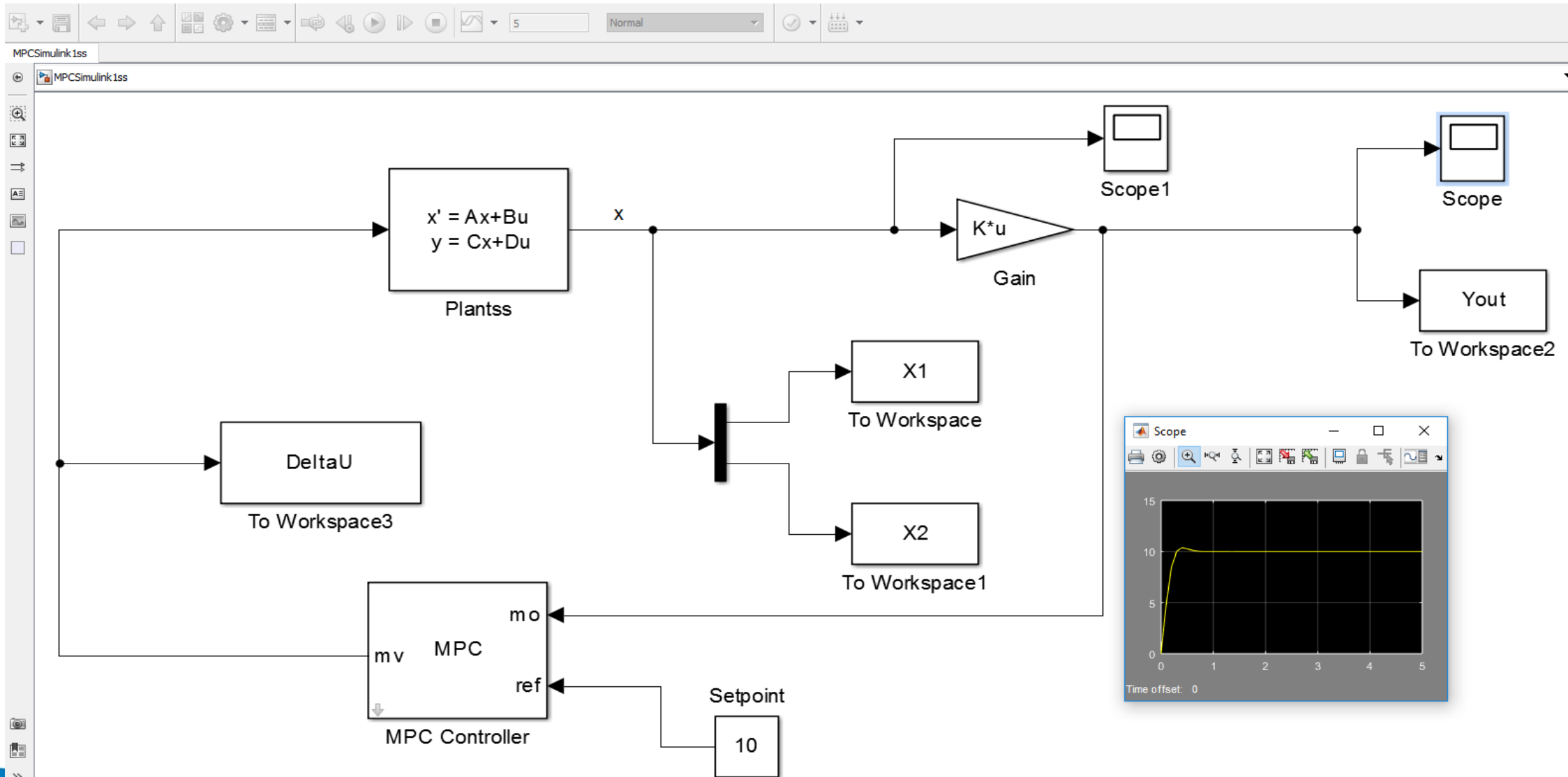
Result: ⚠ Warning

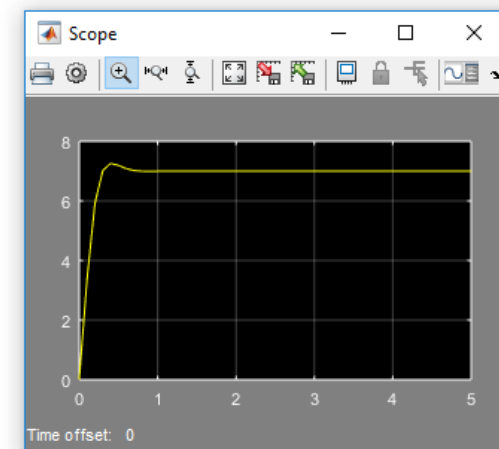
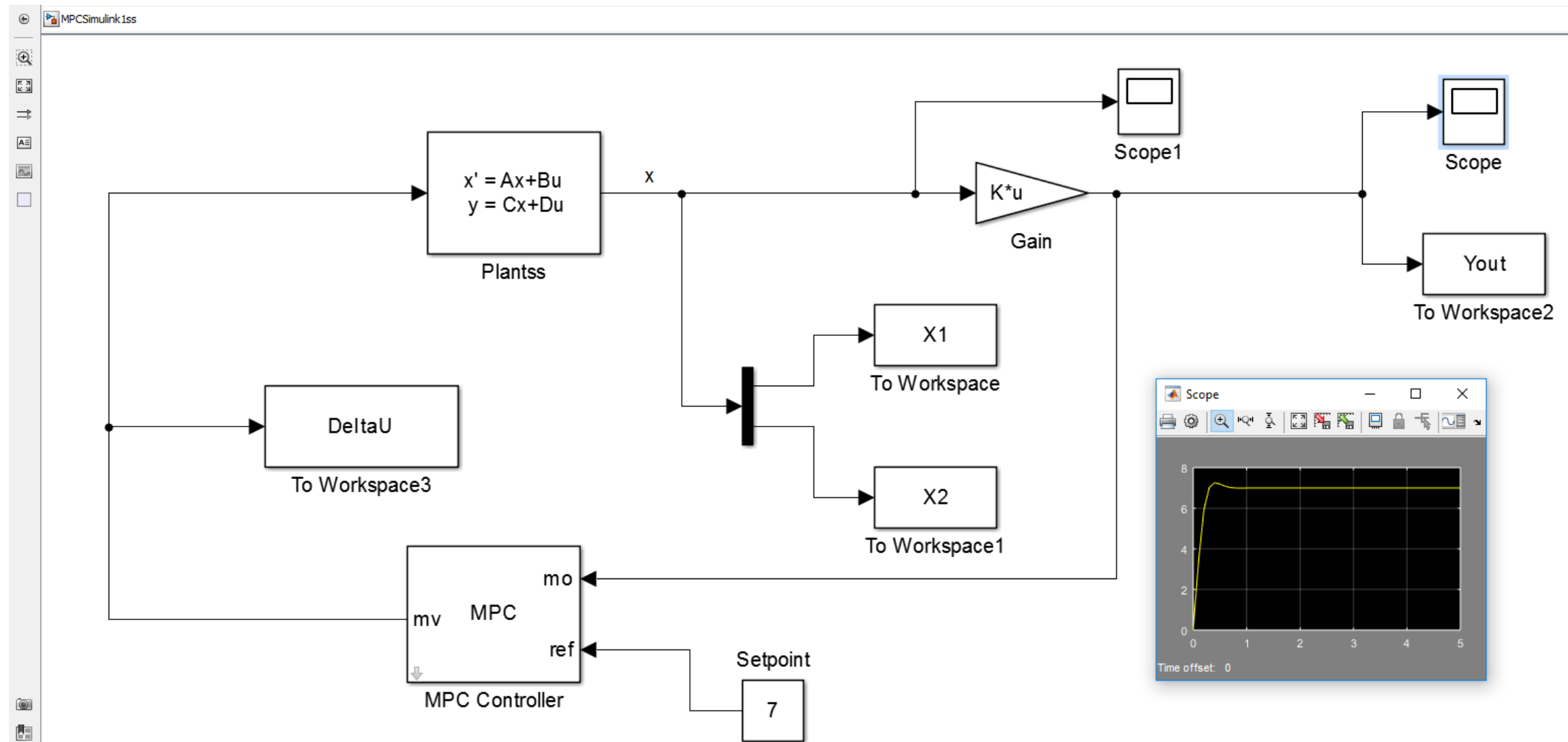
Passed with warning(s) Incorrect model configuration settings for HDL code generation. To fix the problem, click the Modify All button, or click the hyperlink to launch the Configuration Parameter(s) dialog and manually apply the recommended setting.

Block	Settings	Current	Recommended	Severity
Planta_PI_EABb	Block reduction	on	off	warning
Planta_PI_EABb	Conditional input branch execution	on	off	warning
Planta_PI_EABb	Device type	Intel->x86-64 (Windows64)	ASIC/FPGA->ASIC/FPGA	warning
Planta_PI_EABb	Default parameter behavior	Tunable	Inlined	warning

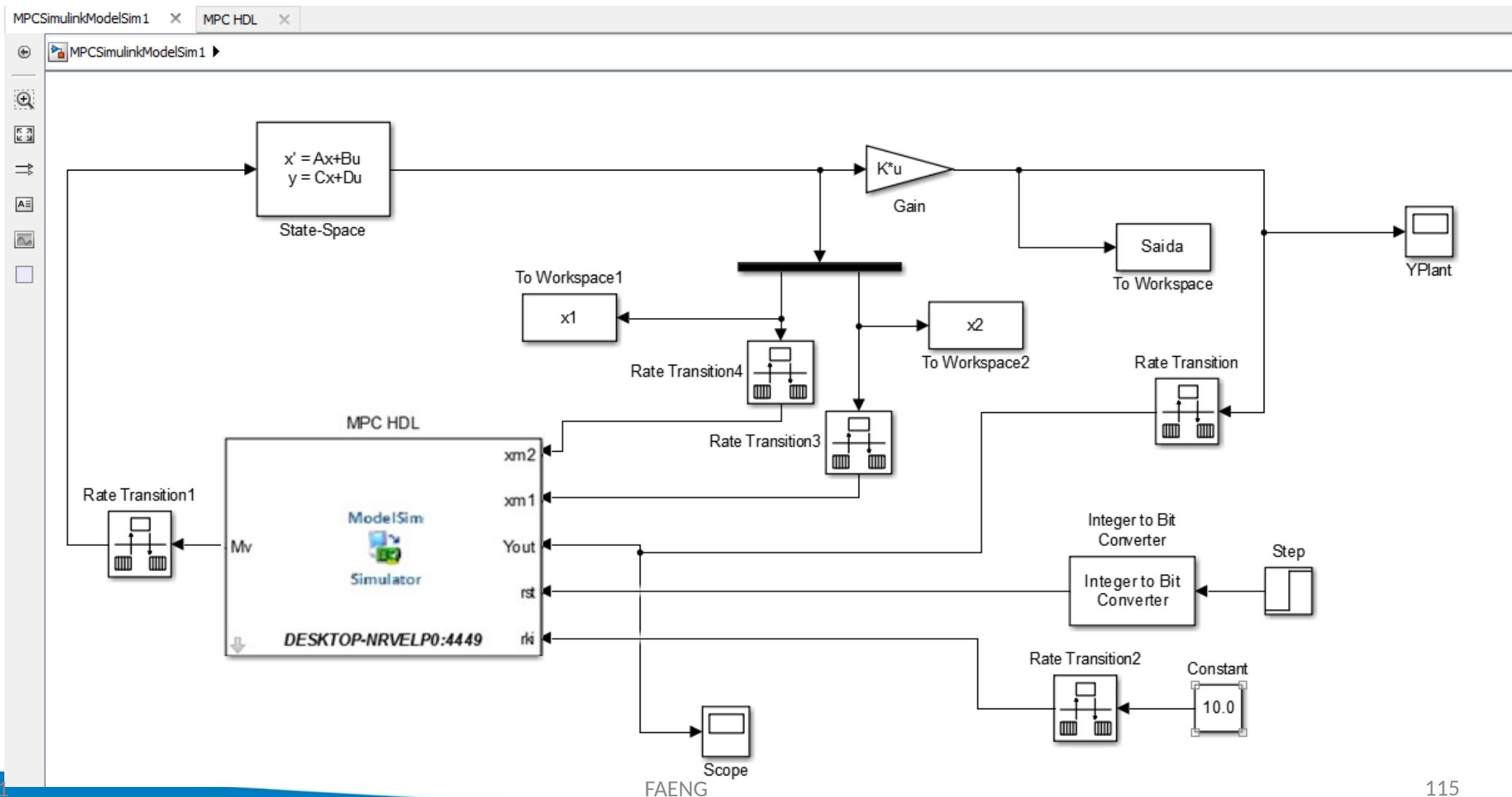
Action

Configure model for recommended settings





Configuração da Co-simulação



- Depois de abrir o Matlab, digite o comando

Código 3.1. `addpath C:/Modeltech_pe_edu_10.4a/win32pe_edu/;`

- Após a execução do comando anterior, digite no matlab

Código 3.2. `vsim('socketsimulink', 4449);`

Projeto MPC – ModelSim

```

C:/Users/Administrador/Desktop/MPCMatlab1Cosimulation1/Exemplo1/Exemplo1.vhd - Default
Ln#
5
6  library ieee;
7  library ieee_proposed;
8
9  use ieee.std_logic_1164.all;
10 use ieee.std_logic_unsigned.all;
11 use ieee.std_logic_arith.all;
12
13 use ieee_proposed.real_matrix_pkg.all;
14
15 entity Exemplo1 is
16     port( clk, gera : in std_logic;
17           rst : in std_logic;
18           Yout : in real; -- Recebe a saída da planta Y
19           rki : in real; -- Porta para o sinal de setpoint
20           xm1 : in real_matrix (0 to 0, 0 to 0); -- realimentação de estado X1
21           xm2 : in real_matrix (0 to 0, 0 to 0); -- recebe X2
22           Mv : out real_matrix (0 to 0, 0 to 0)); -- Lei de controle - saída do MPC

```

```

C:/Users/Administrador/Desktop/MPCMatlab1Cosimulation1/Exemplo1/Exemplo1.vhd - Default
Ln#
25
26 architecture horizon of Exemplo1 is
27
28     signal A : real_matrix (0 to 2, 0 to 2); -- Matrix augmented A
29     signal B : real_matrix (0 to 2, 0 to 0); -- Matrix B
30     signal C : real_matrix (0 to 0, 0 to 2); -- Matric C
31     signal F : real_matrix (0 to 4, 0 to 2); -- Matrix F
32     signal Phi : real_matrix (0 to 4, 0 to 2); -- Matrix Phi  $\Phi$ 
33     signal PhiT : real_matrix (0 to 2, 0 to 4); -- Transpose of Phi
34     =====
35     signal Rs_setpoint : real_matrix (0 to 0, 0 to 4); --Rs*rki vetor setpoint
36     signal Rbarra : real_matrix (0 to 2, 0 to 2); -- matrix Rbarrra  $rw^*(Inc \times Inc)$ 
37     signal PhiTPhi : real_matrix (0 to 2, 0 to 2); --  $\Phi^T\Phi$ 
38     signal PhiTF : real_matrix (0 to 2, 0 to 2); --  $\Phi^TF$ 
39     signal PhiTPhiRbarra : real_matrix (0 to 2, 0 to 2); --  $(\Phi^T\Phi + \bar{R})$ 
40     signal InvPhiTPhiRbarra : real_matrix (0 to 2, 0 to 2);
41     signal InvFinal : real_matrix (0 to 2, 0 to 2); -- inversa corrigida
42     signal Gain_Kmpc : real_matrix (0 to 0, 0 to 2); -- Gain Kmpc
43     signal Gain_Ky : real_matrix (0 to 0, 0 to 0); -- Gain Ky
44     =====
45     signal Saida : real_matrix (0 to 0, 0 to 0); -- Saida do MPC
46     =====
47
48 begin

```

```

C:/Users/Administrador/Desktop/MPCMatlab1Cosimulation1/Exemplo1/Exemplo1.vhd - Default
Ln#
70  -- To build matrix F
71  variable Fnp5 : real_matrix (0 to 4, 0 to 2) := zeros(5,3);
72  variable CA : real_matrix (0 to 0, 0 to 2):= zeros(1,3);
73  variable CA2 : real_matrix (0 to 0, 0 to 2):= zeros(1,3);
74  variable CA3 : real_matrix (0 to 0, 0 to 2):= zeros(1,3);
75  variable CA4 : real_matrix (0 to 0, 0 to 2):= zeros(1,3);
76  variable CA5 : real_matrix (0 to 0, 0 to 2):= zeros(1,3);
77  =====
78  -- to build matrix Phi
79  variable Phi5 : real_matrix(0 to 4, 0 to 2) := zeros(5,3);
80  variable CB : real_matrix(0 to 0, 0 to 0) := zeros(1,1);
81  variable CAB : real_matrix(0 to 0, 0 to 0) := zeros(1,1);
82  variable CA2B : real_matrix(0 to 0, 0 to 0) := zeros(1,1);
83  variable CA3B : real_matrix(0 to 0, 0 to 0) := zeros(1,1);
84  variable CA4B : real_matrix(0 to 0, 0 to 0) := zeros(1,1);
85  -----
86  -- Phi transpose
87  variable PhiT5 : real_matrix(0 to 2, 0 to 4) := zeros(3,5) ;
88  -----
89  --PhiTranspose*Phi, Inversa
90  variable PhiT_Phi : real_matrix(0 to 2, 0 to 2) := zeros(3,3);
91  variable PhiTPhi_Rbarra : real_matrix(0 to 2, 0 to 2):= zeros(3,3) ;
92  variable InvPhiTPhi_Rbarra : real_matrix(0 to 2, 0 to 2):= zeros(3,3);
93  variable InversaCorrigida : real_matrix(0 to 2, 0 to 2) := zeros(3,3) ;

```

```

C:/Users/Administrador/Desktop/MPCMatlab1Cosimulation1/Exemplo1/Exemplo1.vhd - Default
Ln#
146  if (rst = '0') then
147  -- State space am, bm and cm
148  -- matriz exemplo
149  am := ((0.8187, 0.0), (0.09063, 1.0));
150  bm(0,0) := 0.1813;
151  bm(1,0) := 0.009365;
152
153  cm(0,0) := 0.5;
154  cm(0,1) := 0.5;
155
156  Om1(0,0) := 0.0;
157  Om1(0,1) := 0.0;
158
159  peso := 5.0; -- same rw
160  RbNc := eye(3,3);
161  vectorXK(0,0) := 0.1;
162  vectorXK(1,0) := 0.2;
163  vectorXK(2,0) := 0.5;
164
165  Uk(0,0) := 1.0;
166  vectorHumZero(0,0) := 1.0;
167  vectorHumZero(0,1) := 0.0;
168  vectorHumZero(0,1) := 0.0;

```

```

C:/Users/Administrador/Desktop/MPCMatlab1Cosimulation1/Exemplo1/Exemplo1.vhd - Default
Ln#
270 -- Gain Ky
271 Ky := vectorHumZero*InversaCorrigida*PhiT5*vectorHum;
272 =====
273 -- Gain Kk
274 Kx(0,0) := Kmpc(0,0);
275 kx(0,1) :=Kmpc(0,1);
276 =====
277 -- Clock para construção do parâmetro do controlador
278 if(gera'event and gera = '1') then
279 =====
280 --Build X(k+1) augmented
281 vectorXKhum(0,0) := Xkm(0,0);
282 vectorXKhum(1,0) := Xkm(1,0);
283 vectorXKhum(2,0) := Yout1(0,0);
284
285 -- Build X(k) delay of X(k+1)
286 atraso1 := vectorXKHum;
287 atraso2 := atraso1;
288 vectorXK := atraso2;
289 --Define feedback os space state Figure of book - 1 -q^-1 pag18
290 DeltaXk := Xkm - XkMenosHum;
291
292 KxDeltaXk := -(Kx)*DeltaXk; -- -Kx*[X(k) - X(k-1)]
293 -- update X(k-1)
294 XkMenosHum(0,0) := Xkm(0,0);
295 XkMenosHum(1,0) := Xkm(1,0);

```

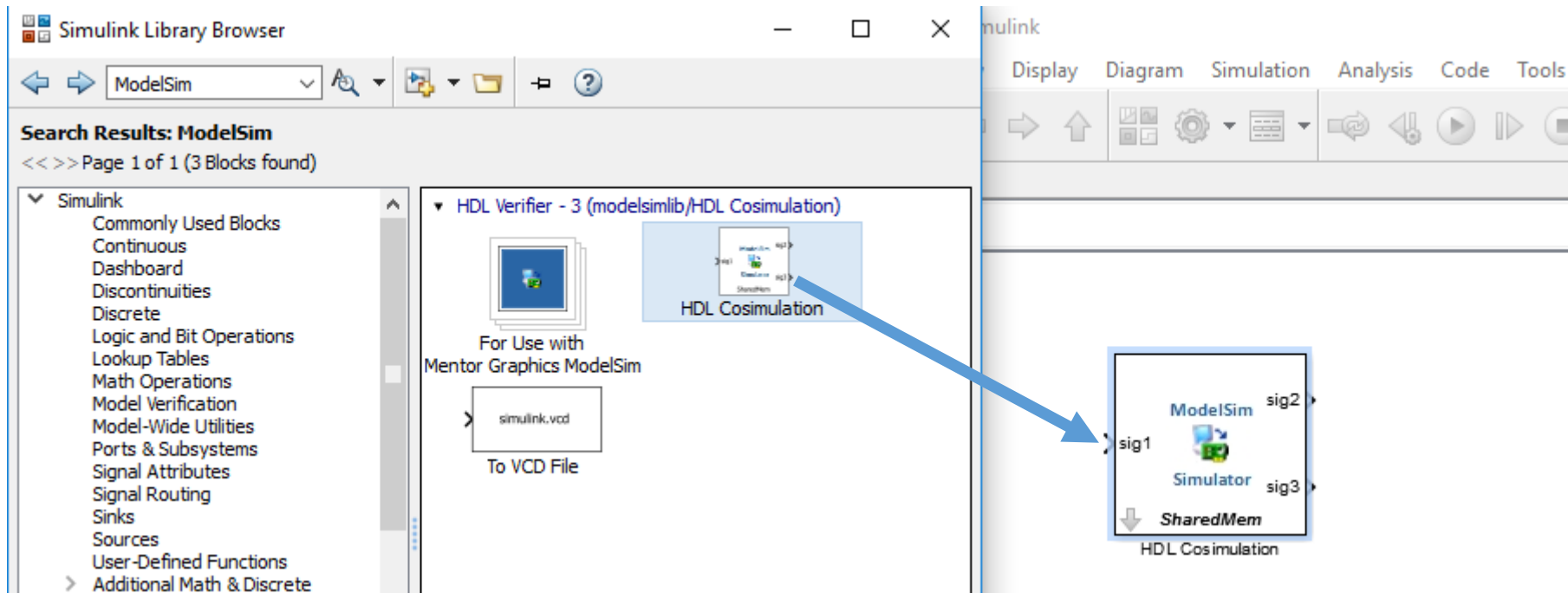

C:/Users/Administrador/Desktop/MPCMatlab1Cosimulation1/Exemplo1/Exemplo1.vhd - Default

```

Ln#
294 XkMenosHum(0,0) := Xkm(0,0);
295 XkMenosHum(1,0) := Xkm(1,0);
296 -----
297 Error := -(Ky)*(Yout - rki);
298 vectorDeltaU := Error + KxDeltaXk;
299 -- integrate
300 Uk := Uk + vectorDeltaU;
301 -----
302 end if;
303 end if;
304 A <= Ael;
305 B <= Bel;
306 C <= Cel;
307 F <= Fnp5;
308 Phi <= Phi5;
309 PhiT <= PhiT5;
310 Rs_setpoint <= Rs_rki;
311 Rbarra <= RbarraW;
312 PhiTPhi <= PhiT_Phi;
313 PhiTPhiRbarra <= PhiTPhi_Rbarra;
314 InvPhiTPhiRbarra <= InvPhiTPhi_Rbarra;
315 InvFinal <= InversaCorrigida;
316 PhiTF <= PhiT_F;
317 Gain_Kmpc <= Kmpc;
318 Gain_Ky <= Ky;
319 Mv <= Uk;

```

Configuração da Co-simulação



Configuração da Co-simulação

Function Block Parameters: HDL Cosimulation

Simulink and ModelSim Cosimulation

Cosimulate hardware components with ModelSim(R) simulators. Inputs from Simulink(R) are applied to HDL signals. Outputs from this block are driven by HDL signals.

Ports | Clocks | Timescales | Connection | Simulation

☒ Enable direct feedthrough
If this block is in a feedback loop and generates algebraic loop warning/error, uncheck this box

Auto Fill Use the 'Auto Fill' button to automatically create the signal interface from a specified HDL component instance.

Full HDL Name	I/O Mode	Sample Time	Data Type	Sign	Fraction Length
/top/sig1	Input	Inher...	Inherit	Signed	Inherit
/top/sig2	Output	10	Inherit	Signed	Inherit
/top/sig3	Output	10	Inherit	Signed	Inherit

New

Delete

Up

Down

OK Cancel Help Apply

Configuração da Co-simulação

Function Block Parameters: MPC HDL
×

Simulink and ModelSim Cosimulation

Cosimulate hardware components with ModelSim(R) simulators. Inputs from Simulink(R) are applied to HDL signals. Outputs from this block are driven by HDL signals.

Ports
Clocks
Timescales
Connection
Simulation

☒ Enable direct feedthrough
If this block is in a feedback loop and generates algebraic loop warning/error, uncheck this box

Auto Fill

Use the 'Auto Fill' button to automatically create the signal interface from a specified HDL component instance.

	Full HDL Name	I/O Mode	Sample Time	Data Type	Sign	Fraction Length
New	/Exemplo1/xm2	Input ▾	Inher...	Inherit ▾	Signed ▾	Inherit
Delete	/Exemplo1/xm1	Input ▾	Inher...	Inherit ▾	Signed ▾	Inherit
Up	/Exemplo1/Yout	Input ▾	Inher...	Inherit ▾	Signed ▾	Inherit
Down	/Exemplo1/rst	Input ▾	Inher...	Inherit ▾	Signed ▾	Inherit
	/Exemplo1/rki	Input ▾	Inher...	Inherit ▾	Signed ▾	Inherit
	/Exemplo1/Mv	Output ▾	0.01	Inherit ▾	Signed ▾	Inherit

OK
Cancel
Help
Apply

Configuração da Co-simulação

Function Block Parameters: MPC HDL

Simulink and ModelSim Cosimulation

Cosimulate hardware components with ModelSim(R) simulators. Inputs from Simulink(R) are applied to HDL signals. Outputs from this block are driven by HDL signals.

Ports Clocks Timescales Connection Simulation

You can generate your HDL clocks in this tab. The edge specifies the active edge in your HDL design. In order to avoid race conditions between the generated clock and the input and output signals, the first active edge will be placed at time Period/2. Other options to generate clocks, resets, and enables include:

- Use Simulink blocks and add the signals to the Ports tab.
- Create waveforms using HDL simulator Tcl commands in the Simulation tab.
- Code them in HDL.

Full HDL Name	Active Clock Edge	Period
/Exemplo1/clk	Rising	1/5000
/Exemplo1/gera	Rising	1/200

New Delete Up Down

OK Cancel Help Apply

Configuração da Co-simulação

Function Block Parameters: MPC HDL

Simulink and ModelSim Cosimulation

Cosimulate hardware components with ModelSim(R) simulators. Inputs from Simulink(R) are applied to HDL signals. Outputs from this block are driven by HDL signals.

Ports Clocks Timescales Connection Simulation

Relate Simulink sample times to the HDL simulation time by specifying a scalefactor. A 'tick' is the HDL simulator time resolution. The Simulink sample time multiplied by the scale factor must be a whole number of HDL ticks.

☐ Automatically determine timescale at start of simulation

Determine Timescale Now Automatically calculates a timescale. Click on the help button for more information.

1 second in Simulink corresponds to in the HDL simulator

OK Cancel Help Apply

Configuração da Co-simulação

Function Block Parameters: MPC HDL

Simulink and ModelSim Cosimulation

Cosimulate hardware components with ModelSim(R) simulators. Inputs from Simulink(R) are applied to HDL signals. Outputs from this block are driven by HDL signals.

Ports | Clocks | Timescales | Connection | Simulation

Connection Mode

☒ Full Simulation

☐ Confirm Interface Only

☐ No Connection

☒ The HDL simulator is running on this computer.

Connection method: Socket

Host name: DESKTOP-NRVELP0

OK Cancel Help Apply

Configuração da Co-simulação

Function Block Parameters: MPC HDL

Simulink and ModelSim Cosimulation

Cosimulate hardware components with ModelSim(R) simulators. Inputs from Simulink(R) are applied to HDL signals. Outputs from this block are driven by HDL signals.

Ports Clocks Timescales Connection Simulation

Time to run HDL simulator before cosimulation starts: 0 ns

Pre-simulation Tcl commands:

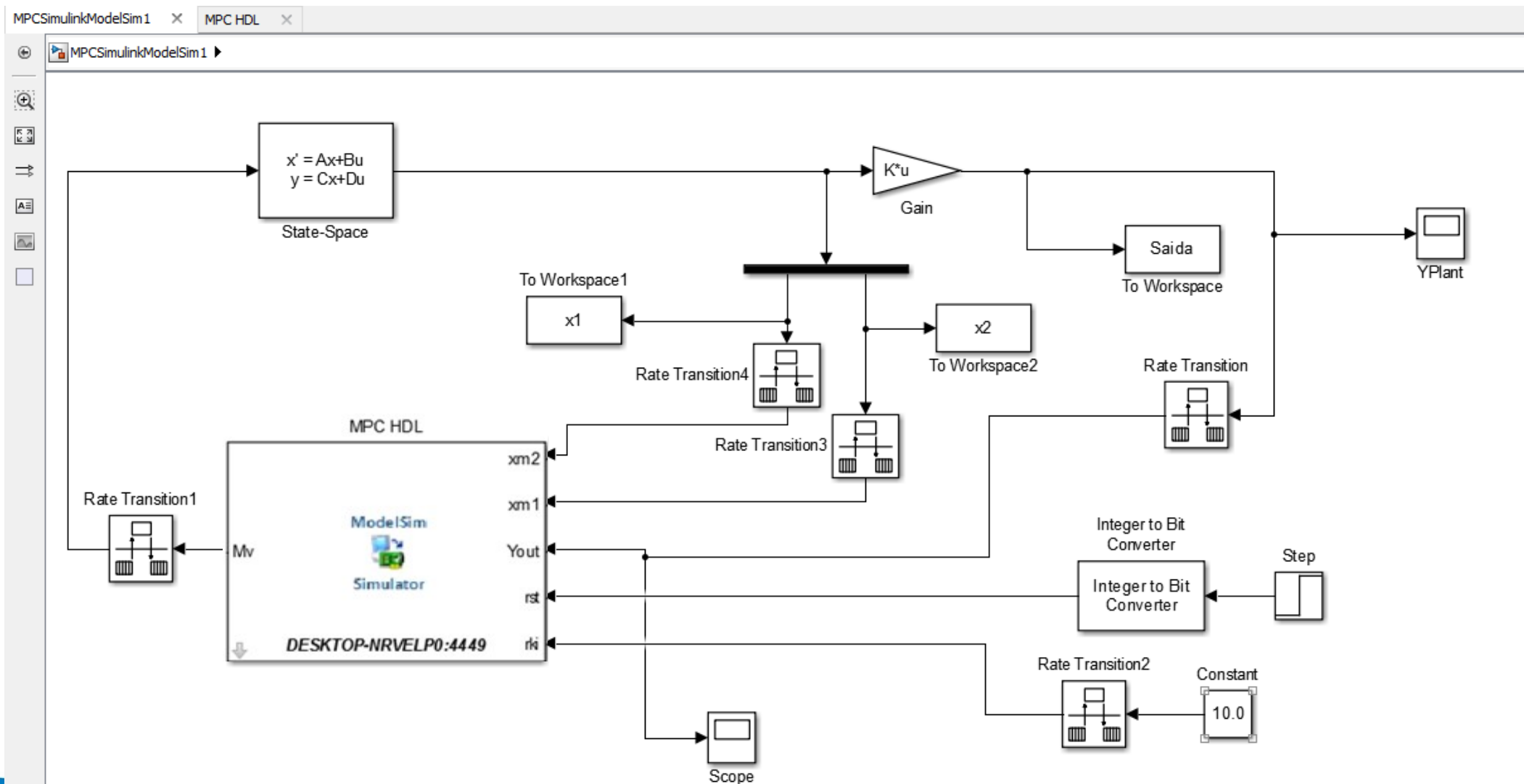
```
puts "Running Simulink Cosimulation block."
```

Post-simulation Tcl commands:

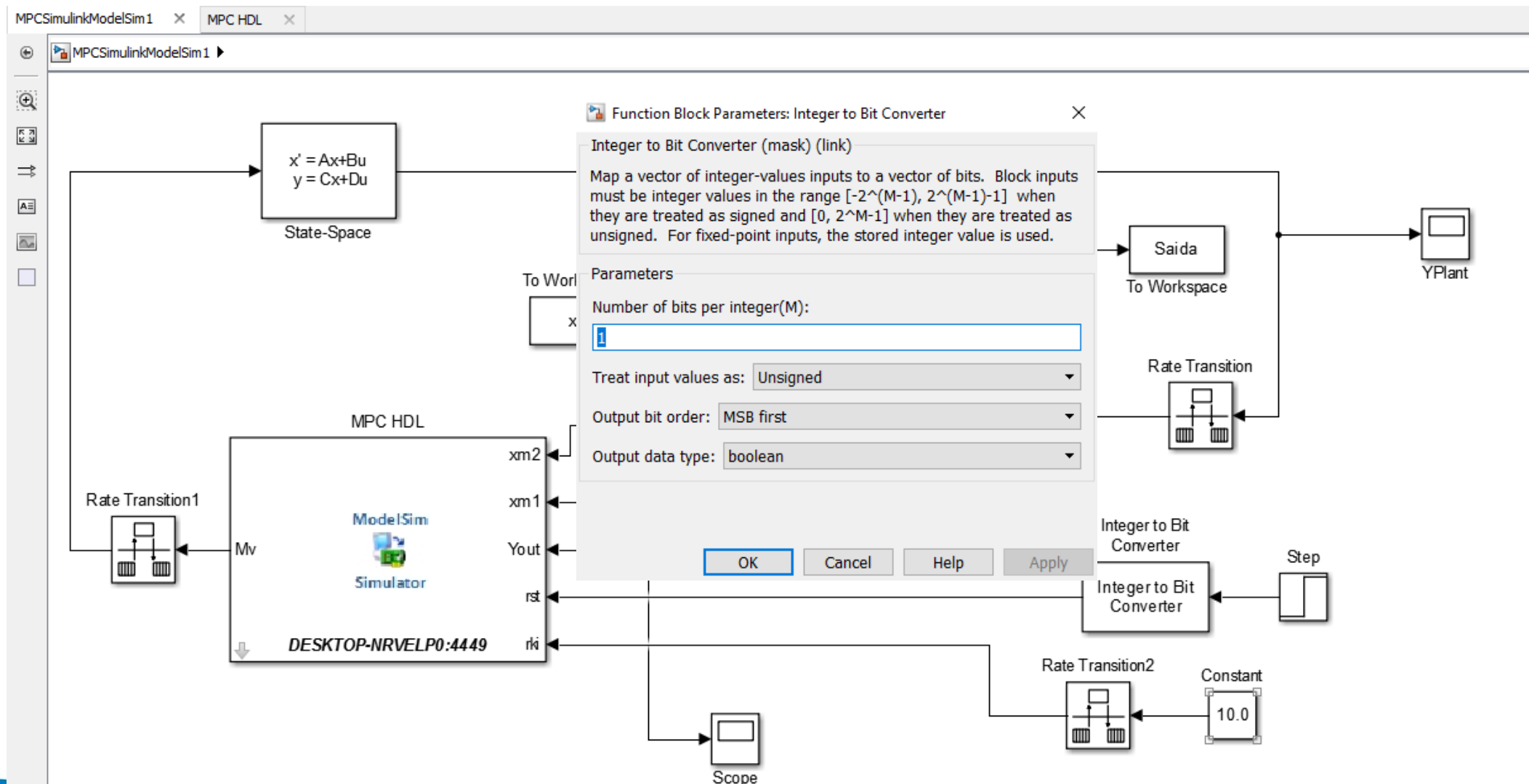
```
puts "done"
```

OK Cancel Help Apply

Configuração da Co-simulação



Configuração da Co-simulação





MATLAB R2015a

HOME PLOTS APPS EDITOR PUBLISH VIEW

New Open Save Find Files Compare Print Go To Find Insert Comment Indent Breakpoints Run Run and Advance Run Section Advance Run and Time

FILE NAVIGATE EDIT BREAKPOINTS RUN

Current Folder

Name

- Img1.3
- itcl3.4
- itk3.4
- Memchan2.3
- mtiRPC
- reg1.2
- rmdb
- ScintillaTk
- slprj
- Tclxpat2.6
- Tclxml2.6
- tk8.5
- tkdiff
- tkdnd
- Tktable2.10
- treectrl2.2.9
- trofs0.4.6
- vfs1.4
- aid.exe

Editor - C:\Users\Administrador\Desktop\MPCMatlab1Cosimulation1\mpc1.m

empcdoubleint.m mpcgain2.m mpc1.m plant2.m

```

1 %Exemplo de um controlador preditivo
2 %Sistemas Embarcados - 2017
3
4 Plant = tf([1 1],[1 2 0]);
5 num = [1 1];
6 den = [1 2 0];
7 %-----
8 [A, B, C, D] = tf2ss(num,den);
9 %Planta utilizada na simulação
10 PlantaSS = ss(Plant);
11 %-----
12 %Planta utilizada para o projeto
13 PlantDigital = c2d(PlantaSS,0.1);

```

Command Window

New to MATLAB? See resources for [Getting Started](#).

>> vsim('socketsimulink',4449)

Library
Name
T
work
Li
ieee_proposed
Li
ieee_proposed.lib
Li
floatfixlib
Li
ieee_env (empty)
Li
infact
Li
mc2_lib (empty)
Li
mgc_ams (empty)
Li
mtiAvm
Li
mtiOvm
Li
mtiDA
Li

C:/Users/Administrador/Desktop/MPCMatlab1Cosimulation1/Exemplo1/Exemplo1.vhd - Default
Ln#
1
2
3
4
5
6
7
8
9
10
11
12
13
14

```

-- Author: Edson Antonio Batista
-- Exemplo: Projeto de Controlador Preditivo com teste via Cosimulação
-- Disciplina: Sistemas Embarcados
-- Outubro/2017

library ieee;
library ieee_proposed;

use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;

use ieee_proposed.real_matrix_pkg.all;

```

Transcript

```

# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Loading package std_logic_arith
# -- Loading package STD_LOGIC_UNSIGNED
# -- Loading package real_matrix_pkg
# -- Compiling entity Exemplo1
# -- Compiling architecture horizon of Exemplo1
# End time: 20:05:20 on Oct 31,2017, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0

ModelSim>

```


Library

Name	T
work	Li
ieee_proposed	Li
ieee_proposed.lib	Li
floatfixlib	Li
ieee_env (empty)	Li
infact	Li
mc2_lib (empty)	Li
mgc_ams (empty)	Li
mtiAvm	Li
mtiOvm	Li
mtiDA	Li

C:/Users/Administrador/Desktop/MPCMatlab1Cosimulation1/Exemplo1/Exemplo1.vhd - Default

Ln#	
1	-- Author: Edson Antonio Batista
2	-- Exemplo: Projeto de Controlador Preditivo com teste via Cosimulação
3	-- Disciplina: Sistemas Embarcados
4	-- Outubro/2017
5	
6	library ieee;
7	library ieee_proposed;
8	
9	use ieee.std_logic_1164.all;
10	use ieee.std_logic_unsigned.all;
11	use ieee.std_logic_arith.all;
12	
13	use ieee_proposed.real_matrix_pkg.all;
14	

Transcript

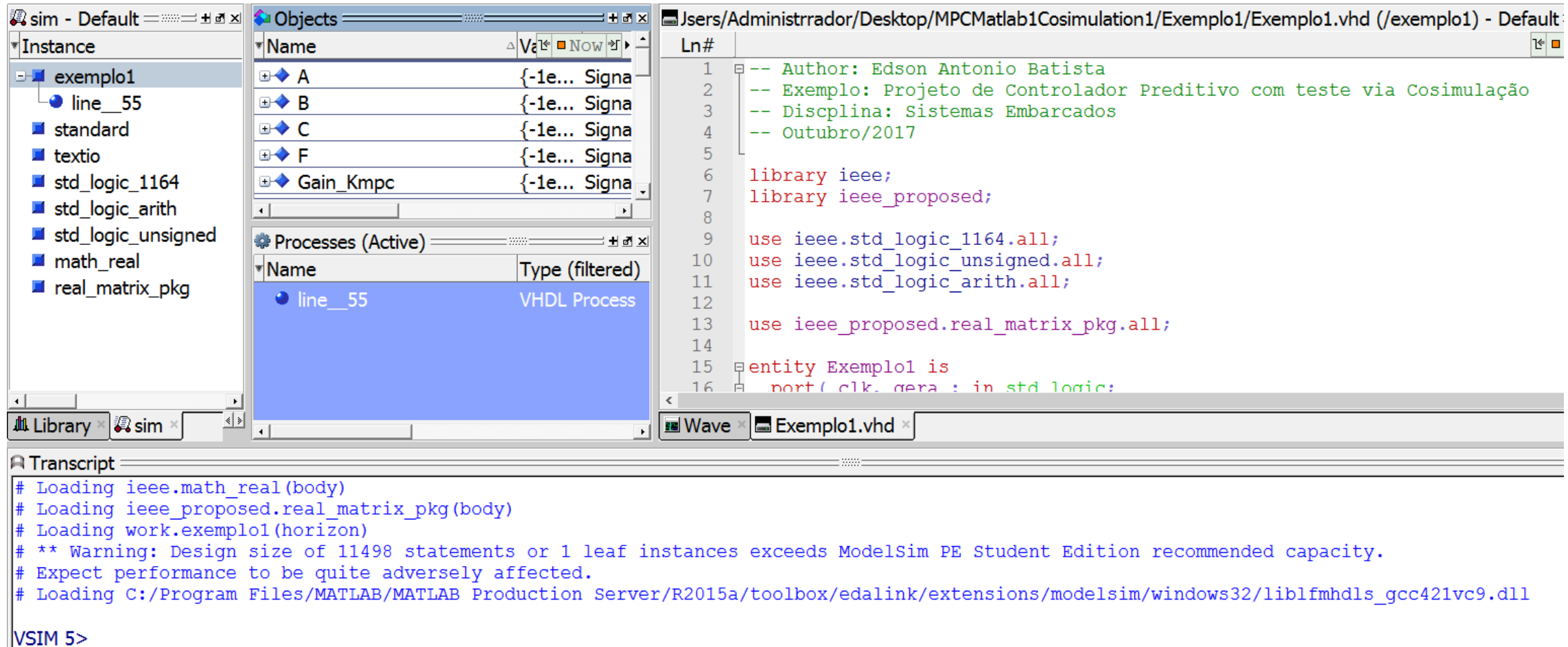
```

# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Loading package std_logic_arith
# -- Loading package STD_LOGIC_UNSIGNED
vsimulink work

```

ModelSim> vsimulink work.Exemplo1

<No Design Loaded> work



The screenshot displays the ModelSim software interface during a VHDL simulation. The top-left pane shows the 'Instance' tree with 'exemplo1' selected. The top-middle pane shows the 'Objects' list with signals A, B, C, F, and Gain_Kmpc. The top-right pane shows the 'Processes (Active)' list with 'line__55' as a VHDL Process. The bottom-left pane shows the 'Library' and 'sim' tabs. The bottom-right pane shows the 'Wave' and 'Exemplo1.vhd' tabs. The main editor window displays the VHDL code for 'Exemplo1.vhd'.

Instance Tree:

- exemplo1
 - line__55
 - standard
 - textio
 - std_logic_1164
 - std_logic_arith
 - std_logic_unsigned
 - math_real
 - real_matrix_pkg

Objects List:

Name	Value	Now
A	{-1e... Signa	
B	{-1e... Signa	
C	{-1e... Signa	
F	{-1e... Signa	
Gain_Kmpc	{-1e... Signa	

Processes (Active):

Name	Type (filtered)
line__55	VHDL Process

VHDL Code (Exemplo1.vhd):

```

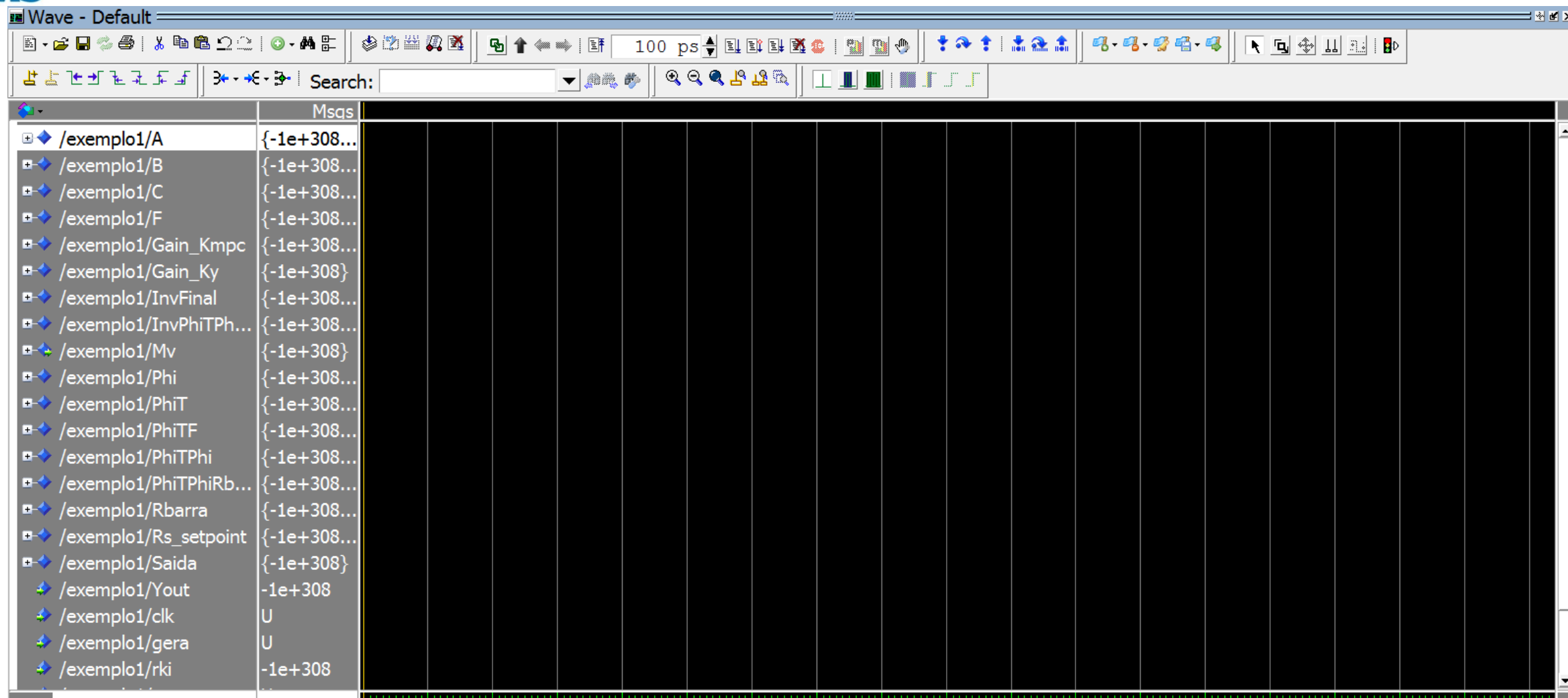
Ln#
1  -- Author: Edson Antonio Batista
2  -- Exemplo: Projeto de Controlador Preditivo com teste via Cosimulação
3  -- Disciplina: Sistemas Embarcados
4  -- Outubro/2017
5
6  library ieee;
7  library ieee_proposed;
8
9  use ieee.std_logic_1164.all;
10 use ieee.std_logic_unsigned.all;
11 use ieee.std_logic_arith.all;
12
13 use ieee_proposed.real_matrix_pkg.all;
14
15 entity Exemplo1 is
16 port ( clk, gera : in std_logic;
```

Transcript:

```

# Loading ieee.math_real(body)
# Loading ieee_proposed.real_matrix_pkg(body)
# Loading work.exemplo1(horizon)
# ** Warning: Design size of 11498 statements or 1 leaf instances exceeds ModelSim PE Student Edition recommended capacity.
# Expect performance to be quite adversely affected.
# Loading C:/Program Files/MATLAB/MATLAB Production Server/R2015a/toolbox/edalink/extensions/modelsim/windows32/liblfmhds_gcc421vc9.dll

VSIM 5>
```



sim - Default

Instance

- exemplo1
 - line__55
 - standard
 - textio
 - std_logic_1164
 - std_logic_arith
 - std_logic_unsigned
 - math_real
 - real_matrix_pkg

Objects

Name	Value
A	{-1e
B	{-1e
C	{-1e
F	{-1e
Gain_Kmpc	{-1e
Gain_Kv	{-1e

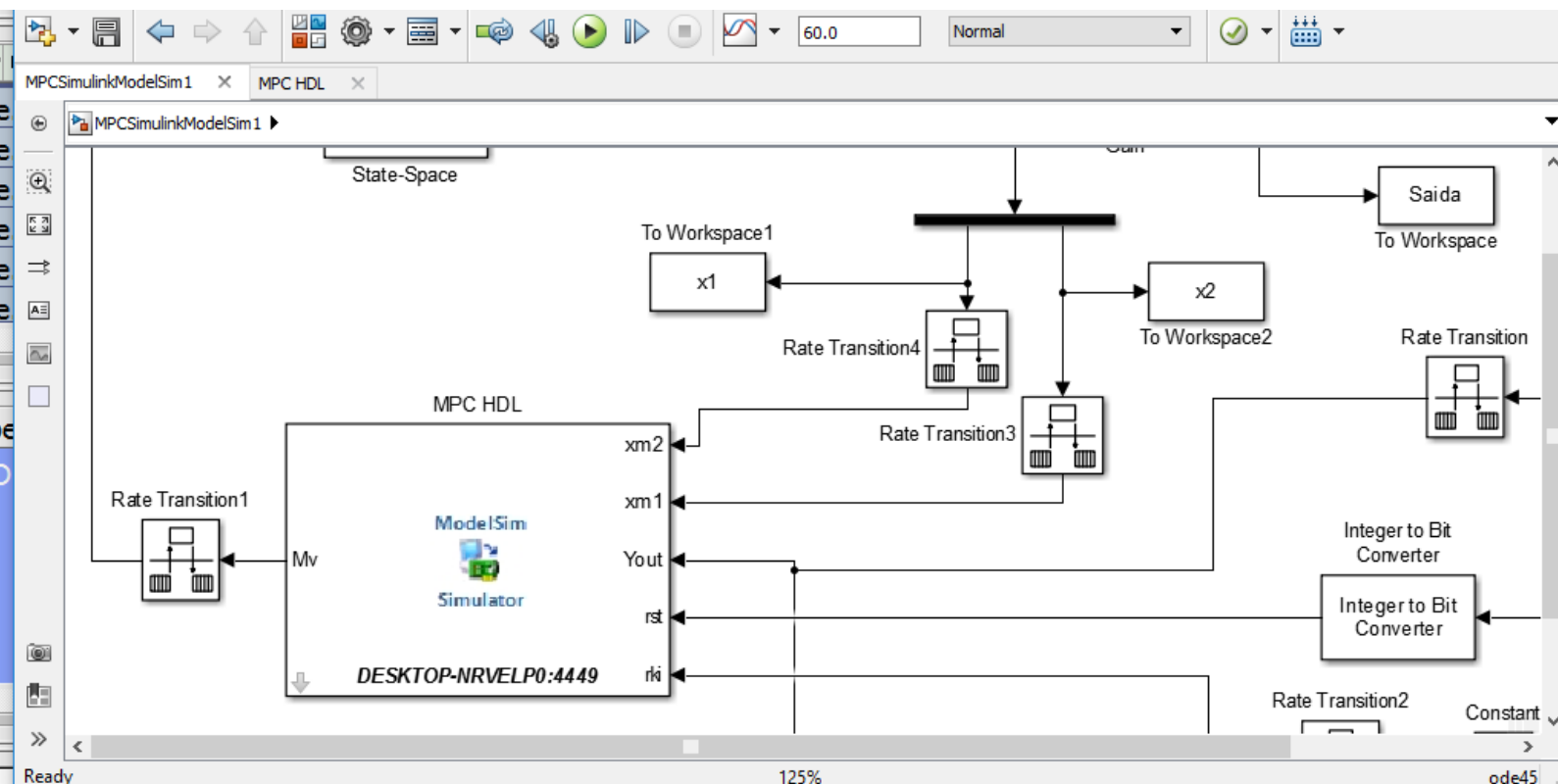
Processes (Active)

Name	Type
line__55	VHD

Library x **sim**

Transcript

```
#
# Attempting to use alternate WLF file "./wlftbj9i1n".
# ** Warning: (vsim-WLF-5001) Could not open WLF file: vsim.wlf
#
# Using alternate file: ./wlftbj9i1n
#
VSIM 6>
```



The screenshot shows the ModelSim HDL simulation environment. The main workspace displays a block diagram of a control system. It includes a 'State-Space' block, a 'MPC HDL' block, and a 'ModelSim Simulator' block. The diagram features several signal lines, rate transitions (Rate Transition1, Rate Transition2, Rate Transition3, Rate Transition4), and output blocks labeled 'x1', 'x2', 'Saida', and 'Integer to Bit Converter'. The status bar at the bottom indicates 'Ready' and '125%' zoom.

sim - Default

Instance

- exemplo1
 - line__55
 - standard
 - textio
 - std_logic_1164
 - std_logic_arith
 - std_logic_unsigned
 - math_real
 - real_matrix_pkg

Objects

Name	Value
A	{-1e
B	{-1e
C	{-1e
F	{-1e
Gain_Kmpc	{-1e
Gain_Kv	{-1e

Processes (Active)

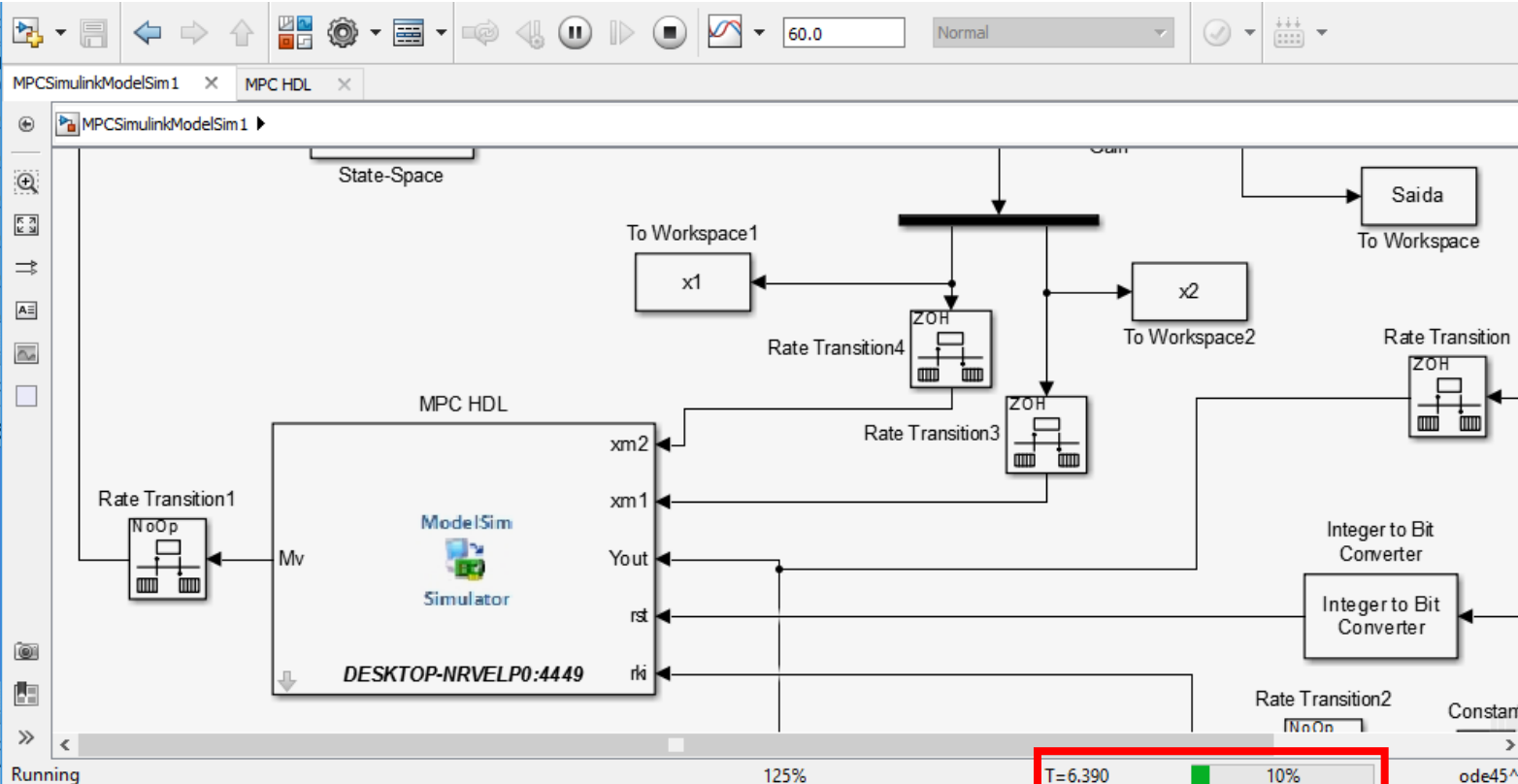
Name	Type
line__55	VHD

Library

Transcript

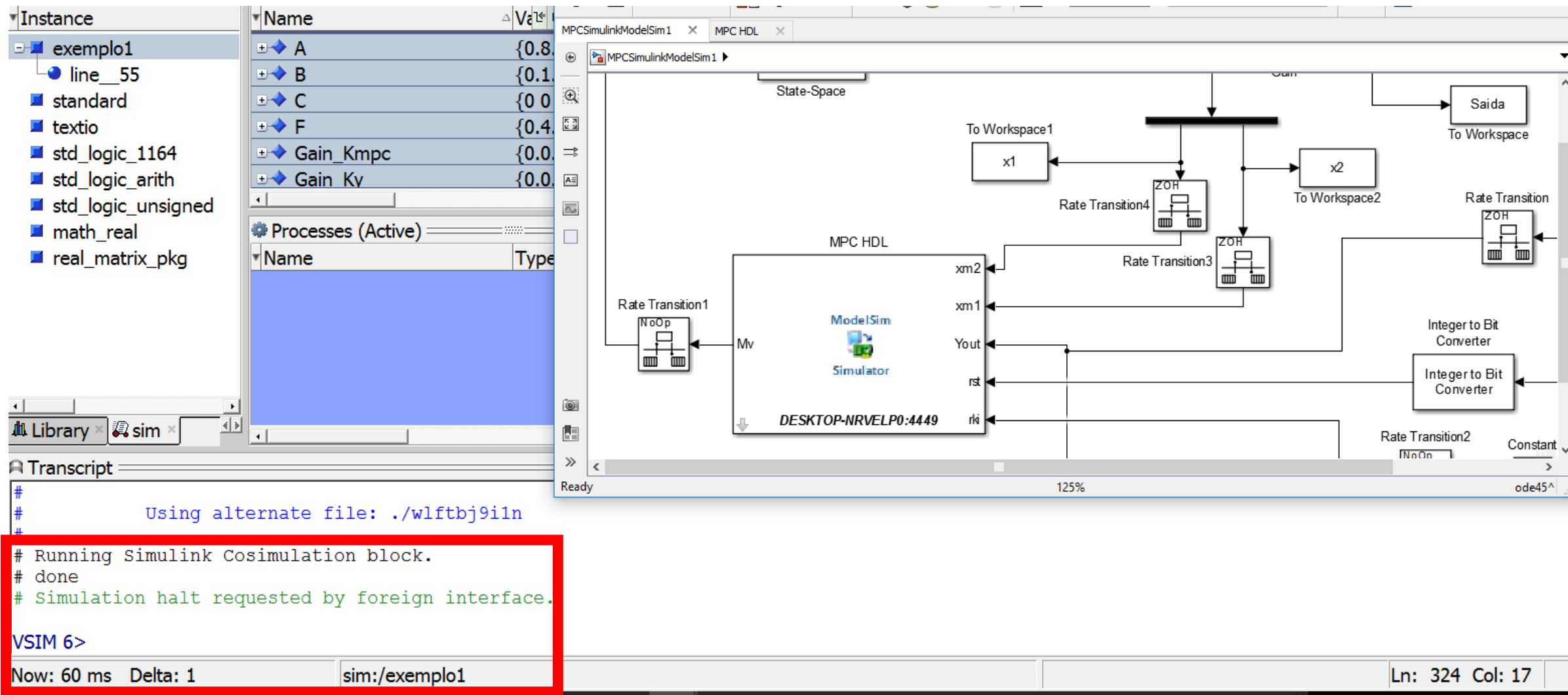
```
# Attempting to use alternate WLF file
# ** Warning: (vsim-WLF-5001) Could not open WLF file: vsim.wlf
#
# Using alternate file: ./wlftbj9i1n
# Running Simulink Cosimulation block.
```

VSIM 6>



The diagram shows a Simulink model for an MPC (Model Predictive Control) system. It includes a central block labeled 'MPC HDL' which is connected to various other blocks. On the left, there's a 'Rate Transition1' block connected to 'Mv'. On the right, there's a 'Rate Transition2' block connected to 'INoOn'. The model also features several 'ZOH' (Zero-Order Hold) blocks and 'Rate Transition' blocks. The output of the model is connected to a 'Saida' block. The model is running on a desktop with the path 'DESKTOP-NRVELP0:4449'.

Running 125% T=6.390 10% ode45^



Wave

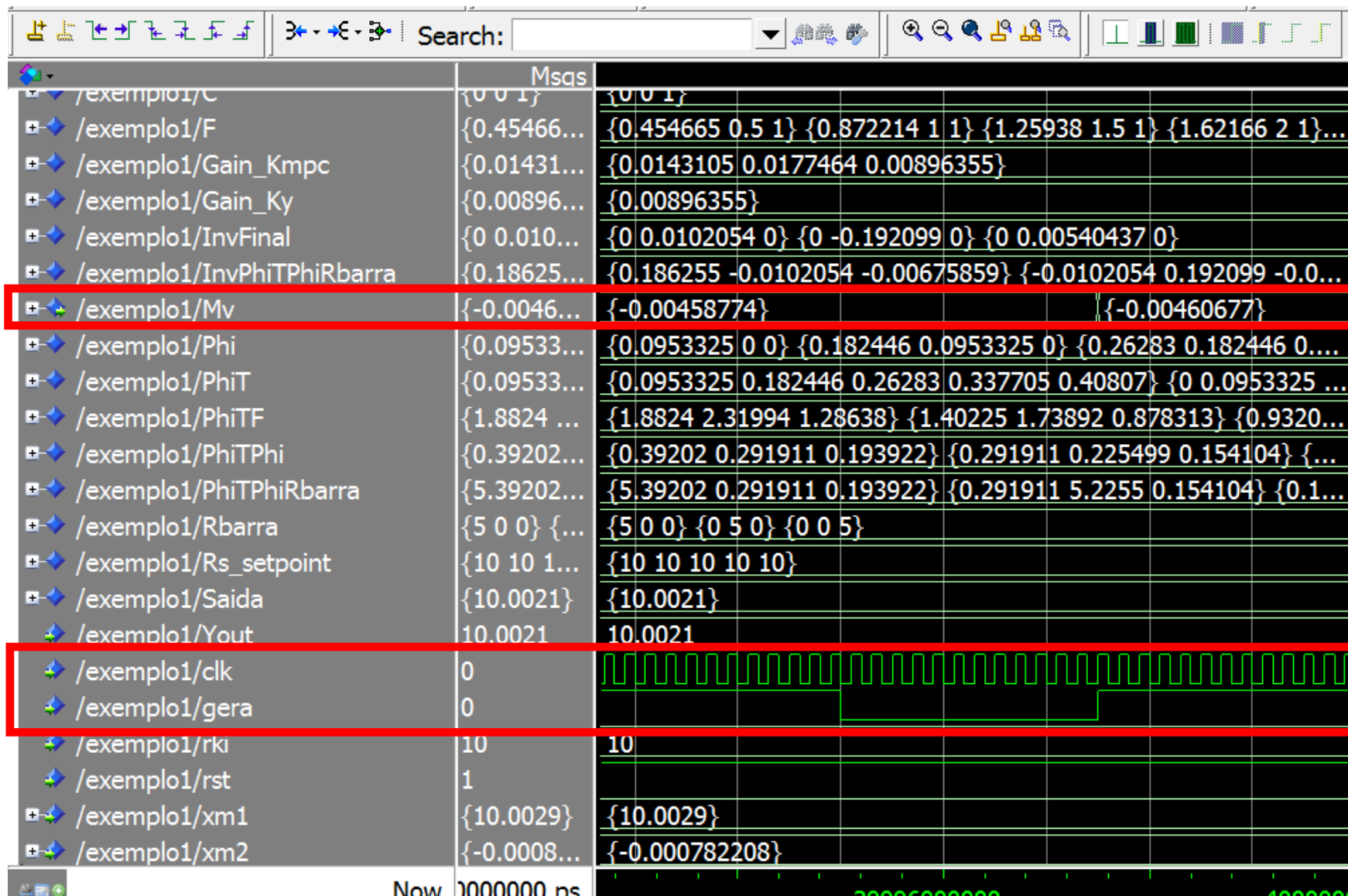
File Edit View Add Format Tools Bookmarks Window Help

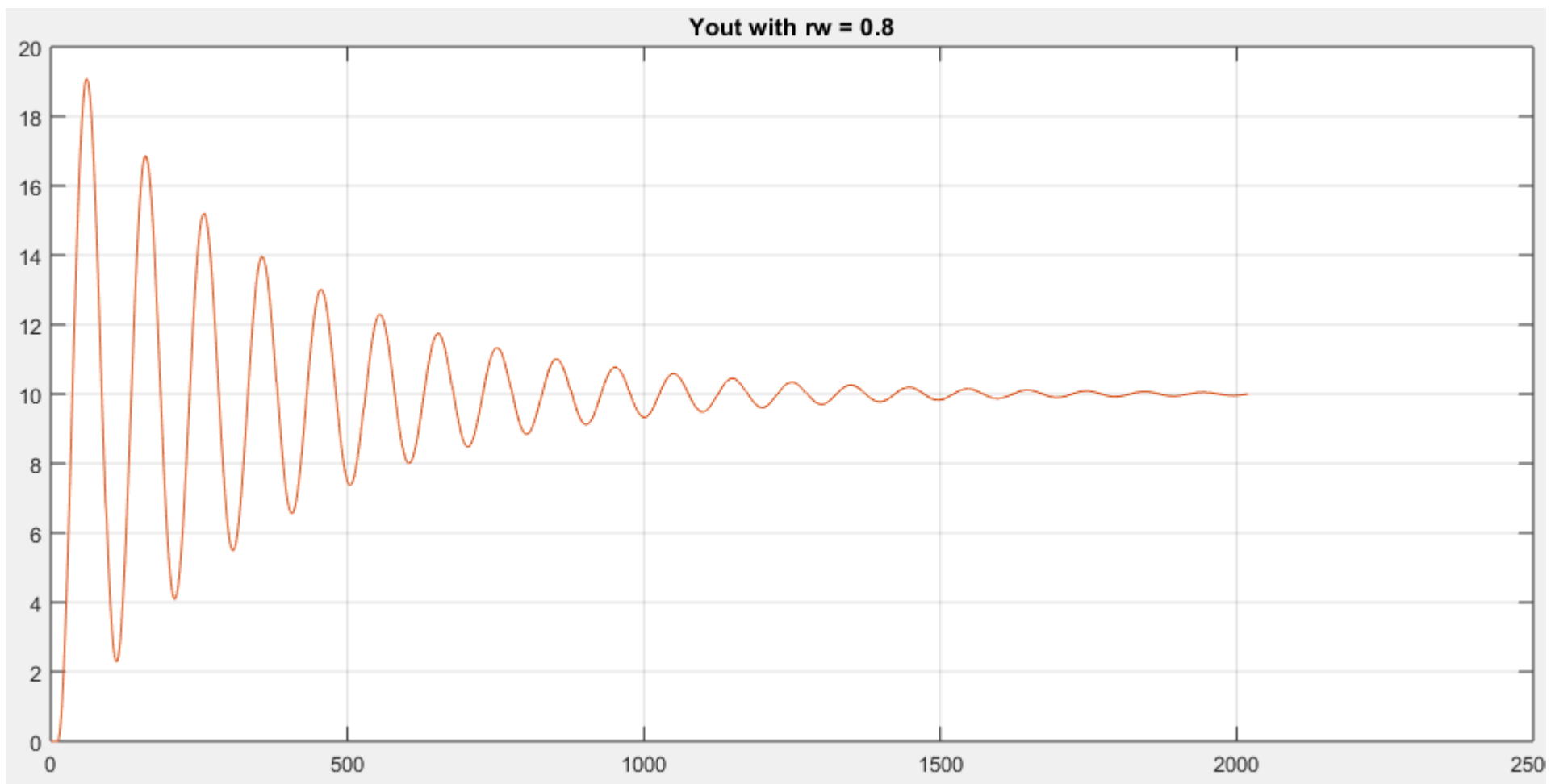
Wave - Default

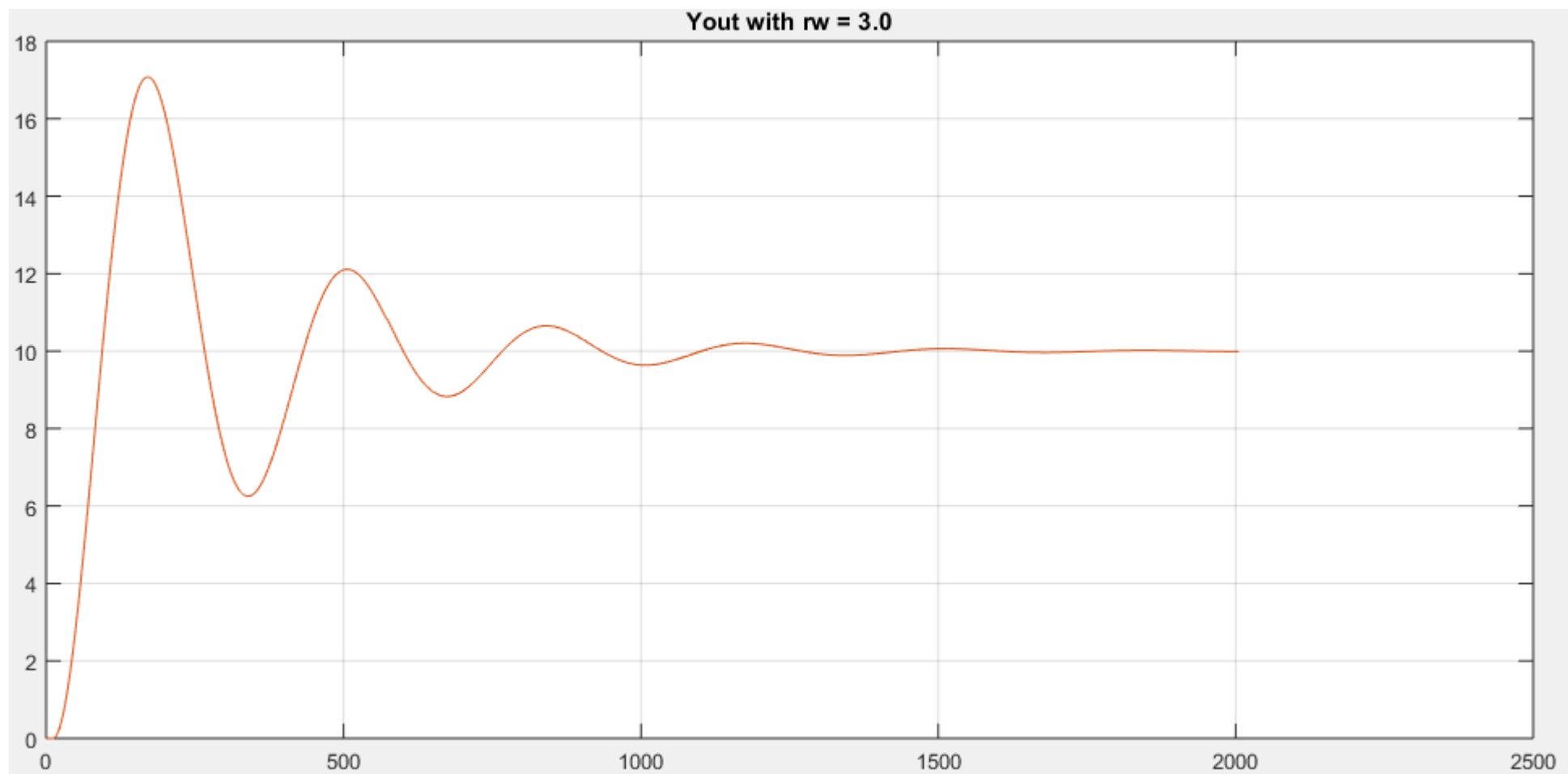
100 ps

Search:

	Msqs	
/exemplo1/A	{0.8187 ...	{0.8187 0 0} {0.09063 1 0} {0.454665 0.5 1}
/exemplo1/B	{0.1813}...	{0.1813} {0.009365} {0.0953325}
/exemplo1/C	{0 0 1}	{0 0 1}
/exemplo1/F	{0.45466...	{0.454665 0.5 1} {0.872214 1 1} {1.25938 1.5 1} {1.62166 2 1} {1.96358 2.5 1}
/exemplo1/Gain_Kmpc	{0.01431...	{0.0143105 0.0177464 0.00896355}
/exemplo1/Gain_Ky	{0.00896...	{0.00896355}
/exemplo1/InvFinal	{0 0.010...	{0 0.0102054 0} {0 -0.192099 0} {0 0.00540437 0}
/exemplo1/InvPhiTPhiRbarra	{0.18625...	{0.186255 -0.0102054 -0.00675859} {-0.0102054 0.192099 -0.00540437} {-0.00675859 -0.00540437 0.196058}
/exemplo1/Mv	{-0.0046...	{-0.00460677}
/exemplo1/Phi	{0.09533...	{0.0953325 0 0} {0.182446 0.0953325 0} {0.26283 0.182446 0.0953325} {0.337705 0.26283 0.182446} {0.40807 0.337705 0.26283}
/exemplo1/PhiT	{0.09533...	{0.0953325 0.182446 0.26283 0.337705 0.40807} {0 0.0953325 0.182446 0.26283 0.337705} {0 0 0.0953325 0.182446 0.26283}
/exemplo1/PhiTF	{1.8824 ...	{1.8824 2.31994 1.28638} {1.40225 1.73892 0.878313} {0.932012 1.16497 0.540608}
/exemplo1/PhiTPhi	{0.39202...	{0.39202 0.291911 0.193922} {0.291911 0.225499 0.154104} {0.193922 0.154104 0.111454}
/exemplo1/PhiTPhiRbarra	{5.39202...	{5.39202 0.291911 0.193922} {0.291911 5.2255 0.154104} {0.193922 0.154104 5.11145}
/exemplo1/Rbarra	{5 0 0} {...	{5 0 0} {0 5 0} {0 0 5}
/exemplo1/Rs_setpoint	{10 10 1...	{10 10 10 10 10}
/exemplo1/Saida	{10.0021}	{10.0021}
/exemplo1/Yout	10.0021	10.0021
/exemplo1/clk	0	
/exemplo1/gera	0	
/exemplo1/rki	10	10







Exercício: A rolagem de um navio pode ser estabilizada com um sistema de controle. Uma tensão aplicada aos atuadores dos lemes de inclinação cria um torque de rolagem que é aplicado ao navio. O navio, em resposta ao torque de rolagem, produz um ângulo de rolagem. Projete um MPC para controlar o ângulo de rolamento.

Observação:

Ângulo de rolagem desejado = 30 graus

Percentual de Overshoot menor que 20%

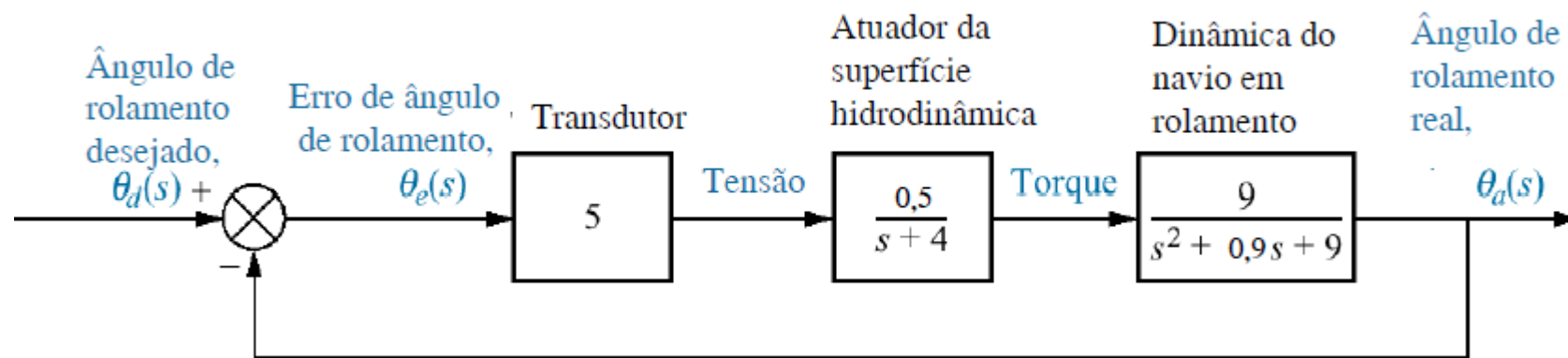
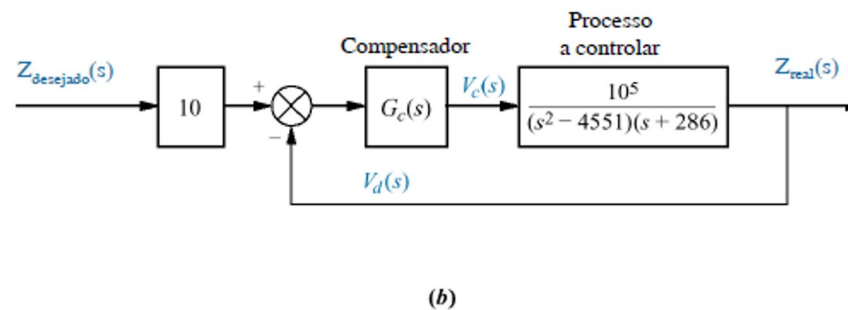
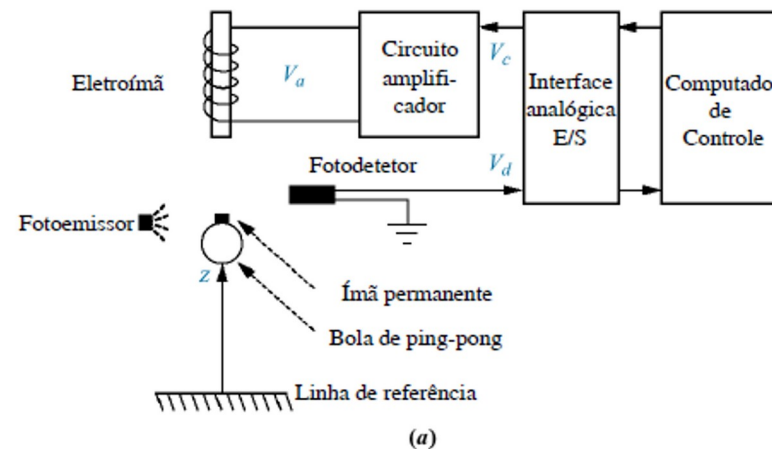


Figura - Diagrama de blocos do sistema de estabilização de rolagem de um navio.

Exercício – Os sistemas de levitação magnética são utilizados atualmente para suspender e propulsionar trens ao longo de trilhos. Um diagrama de um sistema de demonstração de levitação magnética é mostrado na Figura (a). A ação entre um ímã permanente colado à bola de pingue-pongue, o objeto a ser levitado, e um eletroímã fornece a sustentação. A quantidade de elevação pode ser controlada através de V_a aplicada ao eletroímã, como mostrado na Figura (a). A elevação é controlada com a utilização de um par fotodetector para detectar a elevação da bola de pingue-pongue. Admita que o sistema de controle de elevação seja representado pela Figura (b). Com base na planta da Figura abaixo, projete um controlador preditivo que atinja o setpoint. Teste o comportamento do controlador preditivo com diferentes T_s , N_p , N_c e r_w .



MPC para Sistema MIMO

Assumindo que uma planta possui:

- m entradas;
- q saídas;
- n_1 estados

Observação 1: Considerando que **o número de saída é menor ou igual ao número de entrada**, pois se o número de saída for maior que o número de entrada não podemos esperar um controle independente para cada saída (seja levada a erro de regime zero).

Observação 2: Considerando que existe ruído ou perturbação na planta.

$$x_m(k+1) = A_mx_m(k) + B_mu(k) + B_d\omega(k) \quad (1)$$

$$y(k) = C_mx_m(k) \quad (2)$$

Sendo:

$\omega(k)$ é a entrada relacionada ao ruído ou distúrbio. Considerando o distúrbio de entrada, assumido como uma **sequência de ruído branco integrado**. Isso significa que a perturbação de entrada $\omega(k)$ está relacionada a uma sequência de ruído branco de média zero $\epsilon(k)$ pela equação de diferença.

$$\omega(k) - \omega(k-1) = \epsilon(k) \quad (3)$$

Note que na equação (1)

$$x_m(k) = A_mx_m(k-1) + B_mu(k-1) + B_d\omega(k-1) \quad (4)$$

Por definição $\Delta x_m = x_m(k) - x_m(k-1)$ e $\Delta u(k) = u(k) - u(k-1)$

Então subtraindo a equação 4 da equação 1 temos:

$$\Delta x_m(k+1) = A_m\Delta x_m(k) + B_m\Delta u(k) + B_d\epsilon(k) \quad (5)$$

Consequentemente, a saída está relacionada com a variável de estado $\Delta x_m(k)$

$$\Delta y(k+1) = C_m \Delta x_m(k+1) = C_m A_m \Delta x_m(k) + C_m B_m \Delta u(k) + C_m B_d \varepsilon(k)$$

Sendo

$$\Delta y(k+1) = y(k+1) - y(k)$$

Escolhendo um como vetor de variável de estado $x(k) = [\Delta x_m(k)^T \quad y(k)^T]^T$

$$\begin{bmatrix} \Delta x_m(k+1) \\ y(k+1) \end{bmatrix} = \begin{bmatrix} A_m & o_m^T \\ C_m A_m & I_{q \times q} \end{bmatrix} \begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix} + \begin{bmatrix} B_m \\ C_m B_m \end{bmatrix} \Delta u(k) + \begin{bmatrix} B_d \\ C_m B_d \end{bmatrix} \varepsilon(k)$$

$$y(k) = [o_m \quad I_{q \times q}] \begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix} \quad (6)$$

$$\begin{bmatrix} \Delta x_m(k+1) \\ y(k+1) \end{bmatrix} = \begin{bmatrix} A_m & o_m^T \\ C_m A_m & I_{q \times q} \end{bmatrix} \begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix} + \begin{bmatrix} B_m \\ C_m B_m \end{bmatrix} \Delta u(k) + \begin{bmatrix} B_d \\ C_m B_d \end{bmatrix} \varepsilon(k)$$

Sendo:

$$y(k) = [o_m \quad I_{q \times q}] \begin{bmatrix} \Delta x_m(k) \\ y(k) \end{bmatrix} \quad (6)$$

$I_{q \times q}$ a matriz identidade com dimensão igual ao número de saídas

o_m é uma matriz de zeros com dimensão de $q \times n_1$

A_m tem dimensão $n_1 \times n_1$

B_m tem dimensão $n_1 \times m$

C_m tem dimensão $q \times n_1$

Para uma notação simplificada do modelo aumentando temos:

$$x(k+1) = Ax(k) + B\Delta u(k) + B_\varepsilon \varepsilon(k)$$

$$y(k) = Cx(k)$$

Define os vetores Y e ΔU

$$\Delta U = [\Delta u(k_i)^T \quad \Delta u(k_i + 1)^T \quad \dots \quad \Delta u(k_i + N_c - 1)^T]^T$$

$$Y = [y(k_i + 1 | k_i)^T \quad y(k_i + 2 | k_i)^T \quad y(k_i + 3 | k_i)^T \quad \dots \quad y(k_i + N_p | k_i)^T]^T$$

$$x(k_i + 1 | k_i) = Ax(k_i) + B\Delta u(k_i) + B_d\epsilon(k_i)$$

$$\begin{aligned} x(k_i + 2 | k_i) &= Ax(k_i + 1 | k_i) + B\Delta u(k_i + 1) + B_d\epsilon(k_i + 1 | k_i) \\ &= A^2x(k_i) + AB\Delta u(k_i) + B\Delta u(k_i + 1) \\ &\quad + AB_d\epsilon(k_i) + B_d\epsilon(k_i + 1 | k_i) \end{aligned}$$

$$\vdots$$

$$\begin{aligned} x(k_i + N_p | k_i) &= A^{N_p}x(k_i) + A^{N_p-1}B\Delta u(k_i) + A^{N_p-2}B\Delta u(k_i + 1) \\ &\quad + A^{N_p-N_c}B\Delta u(k_i + N_c - 1) + A^{N_p-1}B_d\epsilon(k_i) \\ &\quad + A^{N_p-2}B_d\epsilon(k_i + 1 | k_i) + \dots + B_d\epsilon(k_i + N_p - 1 | k_i). \end{aligned}$$

$$Y = Fx(k_i) + \Phi \Delta U$$

$$\underbrace{\begin{bmatrix} y(k+1) \\ y(k+2) \\ \vdots \\ y(k+n) \end{bmatrix}}_Y = \underbrace{\begin{bmatrix} CA \\ CA^2 \\ CA^3 \\ \vdots \\ CA^{N_p} \end{bmatrix}}_F x(k_i) + \underbrace{\begin{bmatrix} CB & 0 & 0 & \dots & 0 \\ CAB & CB & 0 & \dots & 0 \\ CA^2B & CAB & CB & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ CA^{N_p-1}B & CA^{N_p-2}B & CA^{N_p-3}B & \dots & CA^{N_p-N_c}B \end{bmatrix}}_{\Phi} \Delta U$$

O controle incremental ótimo dentro de uma janela de otimização é dado por:

$$\Delta U = (\Phi^T \Phi + \bar{R})^{-1} \Phi^T (\bar{R}_s r(k_i) - Fx(k_i))$$

$$\Delta U = (\Phi^T \Phi + \bar{R})^{-1} \Phi^T (\bar{R}_s r(k_i) - Fx(k_i))$$

Sendo que a matrix:

- $\Phi^T \Phi$ tem dimensão de $mN_c \times mN_c$
- $\Phi^T F$ tem dimensão $mN_c \times n_1$
- $\Phi^T \bar{R}_s$ é igual a última coluna q de $\Phi^T F$
- A matrix de peso $\bar{R}$ é um bloco de matrizes com dimensão igual de $\Phi^T \Phi$
- Setpoint $r(k_i) = [r_1(k_i) \quad r_2(k_i) \quad \dots r_q(k_i)]^T$ relacionado com q

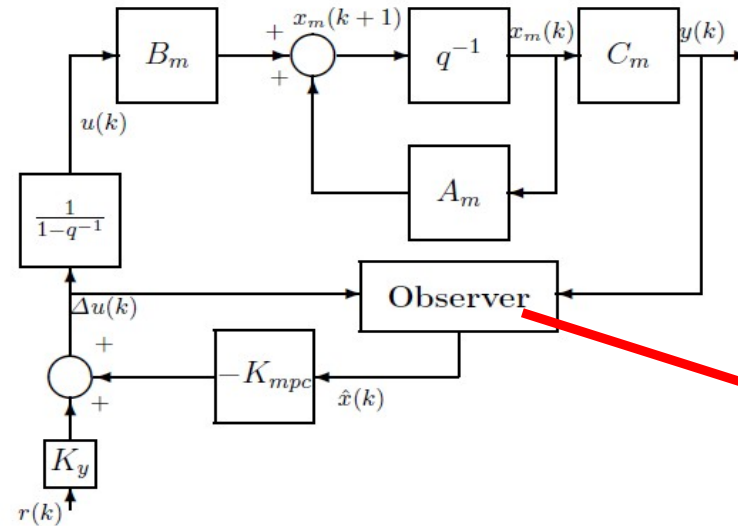
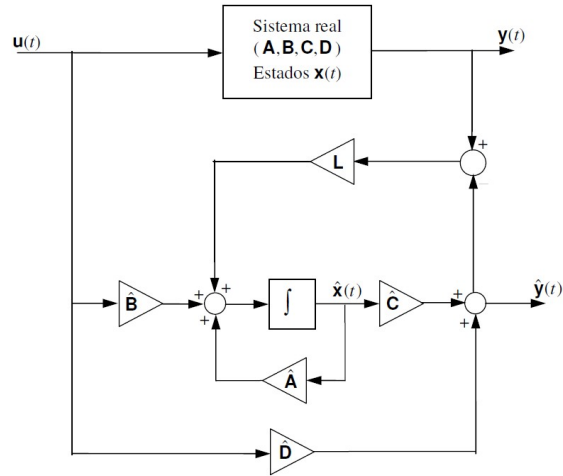
Aplicando o princípio de **controle de horizonte** os m elements em ΔU , forma-se o controle ótimo incremental para Sistema MIMO.

Sendo:

- I_m matriz identidade de dimensão m x m
- o_m matriz de zero de dimensão m x m

$$\begin{aligned} \Delta u(k_i) &= \overbrace{\begin{bmatrix} I_m & o_m & \dots & o_m \end{bmatrix}}^{N_c} (\Phi^T \Phi + \bar{R})^{-1} (\Phi^T \bar{R}_s r(k_i) - \Phi^T Fx(k_i)) \\ &= K_y r(k_i) - K_{mpc} x(k_i), \end{aligned}$$

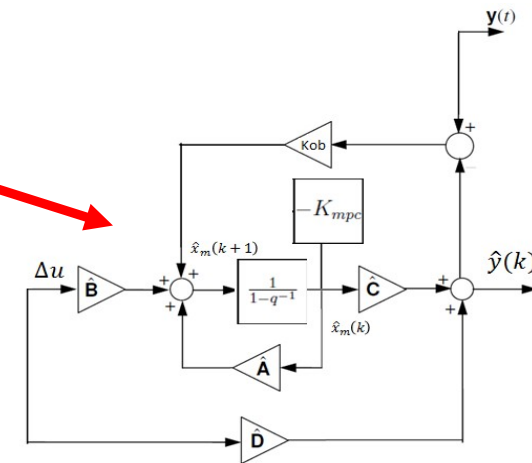
Observador de estado para MPC



$$\hat{x}_m(k+1) = A_m \hat{x}_m(k) + B_m u(k) + K_{ob}(v(k) - \hat{y}(k))$$

$$\Delta u(k) = K_y r(k) - K_{mpc} \hat{x}(k)$$

$$\hat{v}(k) = C_m \hat{x}_m(k)$$



Observador de estado para MPC

Assumindo que o modelo no espaço de estado da planta é dado por

$$x_m(k+1) = A_m x_m(k) + B_m u(k).$$

Pode-se calcular a variável de estado $\hat{x}_m(k)$, $k = 1, 2, \dots$, com uma condição de estado inicial $\hat{x}_m(0)$ e um sinal de entrada $u(k)$:

$$\hat{x}_m(k+1) = A_m \hat{x}_m(k) + B_m u(k).$$

O erro da estimação de estado $\rightarrow \tilde{x}_m(k) = x_m(k) - \hat{x}_m(k)$

$$\begin{aligned}\tilde{x}_m(k+1) &= A_m(x_m(k) - \hat{x}_m(k)) \\ &= A_m \tilde{x}_m(k).\end{aligned}$$

Dado um estado de erro inicial, tem-se

$$\tilde{x}_m(k) = A_m^k \tilde{x}_m(0).$$

A questão é como melhorar a estimativa de $x_m(k)$. A solução é usar um princípio de feedback, onde um sinal de erro é implantado para melhorar a estimativa. O observador é construído usando a equação:

$$\hat{x}_m(k+1) = \overbrace{A_m \hat{x}_m(k) + B_m u(k)}^{\text{model}} + \overbrace{K_{ob}(y(k) - C_m \hat{x}_m(k))}^{\text{correction term}},$$

Sendo
 K_{ob} é a matriz de ganho do observador

Na forma do observador, a variável de estado estimada $\hat{x}_m(k+1)$ possui dois termos:

- O primeiro termo é o modelo original da variável de estado;
- O segundo termo é o termo de correção com base no erro entre a saída medida e a saída prevista usando a estimativa $\hat{x}_m(k)$.

$$\hat{x}_m(k+1) = \overbrace{A_m \hat{x}_m(k) + B_m u(k)}^{\text{model}} + \overbrace{K_{ob}(y(k) - C_m \hat{x}_m(k))}^{\text{correction term}},$$

Para escolher o ganho do observador K_{ob} deve-se analisar a equação do erro de malha fechada. Assim, substituindo $y(k) = C_m x_m(k)$ na “equação acima”

$$\begin{aligned}\tilde{x}_m(k+1) &= A_m \tilde{x}_m(k) - K_{ob} C_m \tilde{x}_m(k) \\ &= (A_m - K_{ob} C_m) \tilde{x}_m(k).\end{aligned}$$

Utilizar a alocação de polos para encontrar a matriz de ganho do observador K_{ob}

MATLAB

```
%-----  
% Transposta de Ae e Ce para gerar o ganho do observador  
At = Ae';  
Ct = Ce';  
%-----  
% Calcula o observador  
polos = [0.02 0.01 0.016];  
Kob = place(At,Ct,polos);
```

TRABALHO 4: Entregar os códigos do Matlab com as respectivas respostas (relatório ou no próprio Código).

1. Dada uma matriz A de ordem $m \times m$, o que é um autovalor de A . Exemplifique a determinação de autovalores de uma matriz A através do Matlab; **(2,0)**
2. Defina o rank de uma matriz. Exemplifique a determinação do rank de uma matriz através do Matlab; **(2,0)**
3. O que é um sistema controlável. Demonstre através do Matlab se um sistema é ou não controlável. **(2,0)**
4. O que é um sistema observável. Dado o sistema;

$$\dot{x} = \begin{bmatrix} -2 & -1 & -3 \\ 0 & -2 & 1 \\ -7 & -8 & -9 \end{bmatrix} x + \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix} u$$

$$y = [4 \quad 6 \quad 8]x$$

Determinar se o sistema é observável utilizando o Matlab; **(2,0)**

5. Dado um sistema digital determine o ganho do observador de estado utilizando o Matlab; **(2,0)**

- Para plantas que operam em determinados valores:

Relacionada com
a lei de controle

$$u^{min} \leq u(k) \leq u^{max}$$

$$\Delta u^{min} \leq \Delta u(k) \leq \Delta u^{max}$$

$$y^{min} \leq y(k) \leq y^{max}$$

Relacionada com
A saída

- Para **incremento do controle**:

$$\begin{aligned} -\Delta U &\leq -\Delta U^{\min} \\ \Delta U &\leq \Delta U^{\max} \end{aligned}$$

- Em formato matricial:

$$\begin{bmatrix} -I \\ I \end{bmatrix} \Delta U \leq \begin{bmatrix} -\Delta U^{\min} \\ \Delta U^{\max} \end{bmatrix}$$

- Para **lei de controle**:

$$\begin{bmatrix} u(k_i) \\ u(k_i + 1) \\ u(k_i + 2) \\ \vdots \\ u(k_i + N_c - 1) \end{bmatrix} = \overbrace{\begin{bmatrix} I \\ I \\ I \\ \vdots \\ I \end{bmatrix}}^{C_1} u(k_i - 1) + \overbrace{\begin{bmatrix} I & 0 & 0 & \dots & 0 \\ I & I & 0 & \dots & 0 \\ I & I & I & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ I & I & I & \dots & I \end{bmatrix}}^{C_2} \begin{bmatrix} \Delta u(k_i) \\ \Delta u(k_i + 1) \\ \Delta u(k_i + 2) \\ \vdots \\ \Delta u(k_i + N_c - 1) \end{bmatrix}$$

- Em formato matricial:

$$\begin{aligned} -(C_1 u(k_i - 1) + C_2 \Delta U) &\leq -U^{\min} \\ (C_1 u(k_i - 1) + C_2 \Delta U) &\leq U^{\max} \end{aligned}$$